

Efficient, UC-secure and Publicly Auditable MPC from OLE & VOLE-in-the-head

Carsten Baum¹  and Chiara-Marie Zok¹ 

Technical University of Denmark, {cabau,chizo}@dtu.dk

Abstract. Secure Multiparty Computation (MPC) computes on private input data, but generally does not guarantee correctness of the output towards third parties. This property, also called public auditability, was first studied explicitly by Baum et al. (SCN 2014). Their work and its follow-ups generate a Non-Interactive Zero-Knowledge proof of correctness of the MPC outcome during the MPC protocol, ensuring validity of the output even if all parties are corrupted.

In this work, we revisit and improve the MPC with Public Auditability blueprint. While the original work uses a version of the SPDZ MPC protocol with expensive lattice-based preprocessing, our construction combines any generic OLE-based preprocessing with a publicly verifiable somewhat linearly homomorphic commitment scheme from VOLE-in-the-head in a non-trivial way. Our commitment scheme relies solely on random oracle calls instead of previously used linearly homomorphic commitments based on structured Public-Key assumptions.

1 Introduction

Secure multi-party computation (MPC) allows a set of mutually distrusting parties $\mathcal{P}_1, \dots, \mathcal{P}_n$ to evaluate a function on their inputs in a privacy-preserving way. When the computation is completed, participants learn nothing except the output of the function. MPC protocols can be used in a variety of contexts such as private voting, auctions or privacy preserving machine learning [14,26,29,1,27] (to just mention some examples). A widely used paradigm to construct secret sharing-based MPC protocols divides computation into a preprocessing and online phase, with the most popular approaches following the BDOZ and SPDZ protocols [21,13,20]. There, during the computationally heavy preprocessing phase, correlated randomness is generated that is then consumed during the lightweight online phase.

In addition to privacy, MPC guarantees that the output of the computation is correct. This is especially important when MPC is used for computations with large impact such as e-voting. However, MPC guarantees hold only to the participating parties and only for corruptions up to the corruption threshold, which is usually strictly smaller than the total number of parties. Publicly Auditable MPC, first introduced in [8], solves this by giving an external party the possibility to audit the transcript after the computation took place. Their MPC protocol

(and follow-up works such as [32]) ensures privacy as long as at least one party is honest, and correctness even if all parties are corrupted. This is achieved in the presence of an external, incorruptible auditor that does not have a secret state and that is not involved in the computation.

Public verifiability can trivially be realized if each party constructs a Non-Interactive Zero-Knowledge proof (NIZK) of correctness while it participates in the MPC protocol. When instantiating this with generic NIZKs [2,12] this leads to suboptimal performance due to the approach being non-black box in the underlying MPC protocol or having to produce the NIZK inside MPC. [8] avoid this by relying on the [21] MPC protocol, that has an information-theoretic online phase where all protocol messages are openings of linear secret sharings. By letting all MPC messages be sent through a bulletin board, which works as a broadcast with memory, the protocol messages can be retained by anyone including the auditor.

[8] then lets each party commit to its shares using linearly homomorphic Pedersen commitments. This allows anyone to establish correctness of the reconstruction of secret sharings and thus the correctness of the MPC output. The construction relies on lattice-based threshold somewhat homomorphic encryption (SHE) for a publicly verifiable preprocessing. The linear homomorphism of the commitments is used to achieve verifiability of the online phase, but this also means that their protocol can only be instantiated for computations over fields as large as Discrete Logarithm (DLOG)-hard groups. In addition, they rely on SHE with computationally costly proofs of verifiable decryption where the plaintext space must be as large as the DLOG-hard group of the commitments.

1.1 Our Contributions

In this work, we improve the [8] blueprint in multiple ways. First, we show that the underlying MPC scheme can be instantiated using recent Pseudorandom Correlation Generator (PCG)-based preprocessing for Oblivious Linear Evaluation (OLE) correlations, reducing the communication overhead due to triple generation. Moreover, we show that the recent Vector-OLE-in-the-Head (VOLEitH) paradigm allows for the construction of suitable linearly homomorphic commitments that make the MPC scheme publicly verifiable without using any assumptions but a Random Oracle. This avoids both expensive verifiable decryption as well as commitments based on public-key assumptions. Finally, we prove our constructions secure in the Universal Composability (UC) framework.

Better UC one-time homomorphic commitments. It is well-known that UC-secure VOLE-based homomorphic commitments can easily be constructed based on a PCG preprocessing [35,10,34]. However, these are designated-verifier commitments that require a private state, making them unsuitable for Auditable MPC. We observe that VOLE-in-the-head ZK proofs implicitly define linearly homomorphic commitments, but where such commitments only can be opened

once. We formalize and analyze their construction in the UC framework and construct the first UC-secure one-time linearly homomorphic commitment scheme¹ from VOLEitH. These commitments can, in turn, be constructed only assuming a Random oracle, and we provide a UC formalization that permits their use in non-interactive protocols based on the Fiat-Shamir transform. Moreover, we show that our commitments can be opened more than once: basically, a sender can send openings *to the future* by committing to the opening using the Random Oracle and thus open the openings at a later point.

Efficient Auditable MPC for any (large) field using OLE. [8] relies on the linear homomorphism and public verifiability of Pedersen commitments, and also that those commitments can be opened repeatedly without weakening security. We simplify the preprocessing, redesign how the MPC protocol uses the linearly homomorphic commitments, and show how to reconcile the one-time property with the required verifiability of the openings. Namely, we observe that it is sufficient if the commitment scheme allows for *two* reliable openings: one to verify all opened secret sharings in the MPC protocol, and another to verify the output. The second opening can be realized from sending openings *to the future* which our commitment scheme permits. Using the linearity of the commitment scheme, we implicitly construct a computationally lightweight NIZK tailored to our MPC scheme that efficiently proves honest behavior of parties.

1.2 Technical Overview

High-level structure of the MPC protocol. We begin by describing the architecture of our MPC protocol from a high level, before showing how the individual parts are constructed. For a visual overview, see Figure 1. All our protocols make use of the global bulletin board $\mathcal{F}_{\text{GBB}}$ for communication and almost all protocols use the global random oracle $\mathcal{F}_{\text{GpRO}}$ for various purposes. The transcript on $\mathcal{F}_{\text{GBB}}$ can be used by any auditor to verify the correctness of the computation as soon as the output is sent to the bulletin board. The MPC protocol follows the offline-online paradigm. The online phase, formalized by functionality $\mathcal{F}_{\text{Online}}$ (see Section 6) is realized through the protocol Π_{Online} .

For a fixed circuit $\mathcal{C}$ the parties first generate all required correlated randomness, and commitments to it, using the functionality $\mathcal{F}_{\text{Offline}}$. This correlated randomness is consumed in the circuit computation. We realize $\mathcal{F}_{\text{Offline}}$ through the protocol Π_{Offline} where we use the OLE functionality $\mathcal{F}_{\text{OLE}}$ to generate Beaver Triples and a functionality $\mathcal{F}_{\text{HOTCom}}$ to commit to random values and triples. To realize $\mathcal{F}_{\text{HOTCom}}$, which formally describes our VOLEitH-based commitments with public verifiability, we first construct a protocol $\Pi_{\text{HOTCommit}}^{1-1}$ that realizes a version of $\mathcal{F}_{\text{HOTCom}}$ (called $\mathcal{F}_{\text{HOTCom}}^i$). This single-sender single-receiver commitment is built on top of $\mathcal{F}_{\text{WVOLE}}$, a functionality that generates *weak* VOLE

¹ One-time UC-secure linearly homomorphic commitments from random oracles have previously been constructed in [18], but our construction yields concretely better parameters due to the use of VOLEitH.

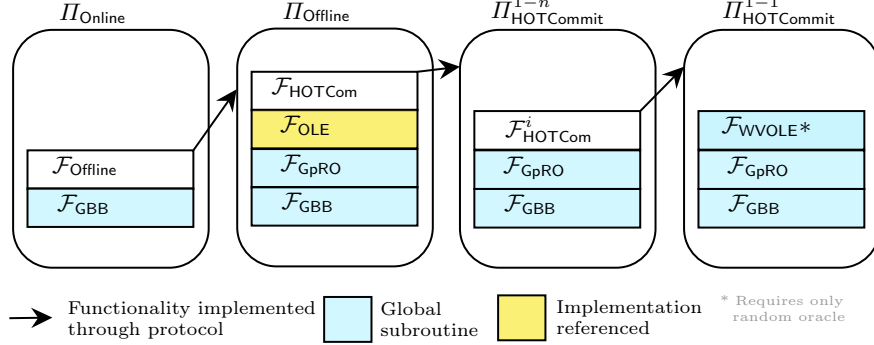


Fig. 1: Overview of protocols and functionalities.

correlations for a sender and a receiver. We then show how to use multiple $\mathcal{F}_{\text{HOTCom}}^i$ instances to realize $\mathcal{F}_{\text{HOTCom}}$ in the single-sender multi-receiver setting based on the protocol $\Pi_{\text{HOTCommit}}^{1-n}$. This two-step construction of $\mathcal{F}_{\text{HOTCom}}$ is necessary to allow senders to reliably open commitments to individual receivers. We finally realize $\mathcal{F}_{\text{WVOLE}}$ by carefully formalizing and analyzing the previous VOLEitH construction in the UC framework. For $\mathcal{F}_{\text{OLE}}$ we give references for existing implementations. Hence our construction only relies on a global random oracle, a bulletin board and OLE.

OLE & MPC. We use a version of the BDOZ MPC protocol [13] for the online phase. In BDOZ, each party $\mathcal{P}_i$ of the n parties $\mathcal{P}_1, \dots, \mathcal{P}_n$ holds a share $x_i \in \mathbb{F}$ of a secret x such that $x = x_1 + \dots + x_n$ where $\mathbb{F}$ is a finite field. To make sure parties do not alter their shares, BDOZ uses pair-wise linearly homomorphic MACs. This prevents parties from revealing inconsistent shares $x'_i \neq x_i$ of the secret, while it still allows all parties to perform linear computations on the shared secrets. BDOZ also uses preprocessed and shared multiplication triples together with Beaver’s circuit randomization technique [11] to perform multiplications on secret-shared data. Recently, Rachuri & Scholl [30] have shown how to implement a preprocessing phase for SPDZ-type protocols using pair-wise Programmable PCGs that generate Oblivious Linear Evaluation correlations over a large field $\mathbb{F}$. We use their approach for preprocessing, but with heavy modifications to achieve public verifiability.

VOLEs & linearly homomorphic MACs. In the BDOZ MACs, the sender (the party holding the share) has the share $x_i \in \mathbb{F}$ and a MAC $w_i \in \mathbb{F}$, while the receiver (the party establishing correctness of the opening) holds $\Delta, v_i \in \mathbb{F}$ such that $w_i = v_i + x_i \Delta$. To reveal the share in a verifiable way, the sender sends x_i, w_i to the receiver, who checks that the correlation holds. For all shares of $\mathcal{P}_i$ towards the same receiver $\mathcal{P}_j$, the receiver uses the same Δ in all correlations (making the MAC scheme linearly homomorphic). Put differently, correctly established MACs on secret shares form a Vector Oblivious Linear Evaluation (VOLE) cor-

relation. However, security of the MACs fails in the public verifiability setting as Δ becomes known to the sender if all parties are corrupted.

Making VOLE MACs into publicly verifiable commitments. Public verifiability for VOLE correlations was recently addressed in [7]. Their work extends the PCG-based ZK proofs of [35,10,34] towards public verifiability of VOLE-based NIZKs. [7] achieve this by observing that a weaker form of VOLE is sufficient for VOLE-based MACs used in NIZKs, viewing these MACs as commitments. The sender obtains x_i, w_i already during the commitment phase, and can conduct arbitrary (at this point unverifiable) openings of commitments. Once all operations on commitments and their openings are completed, Δ is generated as a random oracle output. Finally, based on this choice of Δ , the receiver learns v_i which allows it to verify the openings.

In [7] the focus lies on achieving VOLE-based NIZK proofs. We formalize the properties of the commitment scheme that [7] actually achieves, and prove that their construction yields one-time linear UC-secure commitments. Towards this, we have to explicitly apply the Fiat-Shamir transform to construct the commitment scheme instead of directly compiling to NIZKs as in previous works. As the one-time property is insufficient for use in MPC, we use that, during an opening, the sender can commit to its opening x_i, w_i using a regular UC commitment instead of sending it directly to the receiver. This gives the sender a limited way to make openings even after the verifier’s key Δ has been publicly revealed. We formalize this property, which we call *delayed openings*, and use it in our construction.

Having only single-receiver commitments is insufficient for use in MPC protocols, where we want each party to be committed to its shares *consistently* towards all other parties. We therefore use a consistency check based on a universal hash, combined with a global bulletin board that we use as broadcast channel. To reliably avoid cheating even if all parties are corrupted, we determine the challenge used in the check using the Fiat-Shamir transform applied to all commitments previously made. In addition, we show how the check integrates with our commitment scheme with its constraints on verifiable openings.

Commitments in MPC. We then integrate our new commitment scheme into the MPC protocol. There, the parties will use OLEs to generate secret sharings of multiplication triples. We then let parties post commitments to their shares on a public bulletin board. This will also be used by parties to open commitments during evaluation of the MPC protocol, and we check correctness of the multiplication triples akin to [28]. All verification steps of the MPC scheme, as well as the actual computation, can be combined into a single verifiable opening. However, in order to achieve input independence (as in previous works), we only reveal the output of the computation once all consistency checks passed.

Universal Composition and Fiat-Shamir. We formalize lower-level primitives such as weak VOLE and one-time commitments in UC, give a composable con-

struction and analyze all security guarantees modularly. This clashes with the use of the Fiat-Shamir transform when generating challenges in many ways: for example, to generate a challenge that tests if multiplication triples are well-formed, the challenge must be computed based on the state of the UC functionality which holds the shares. However, normal UC functionalities do not offer such a mechanism, and a (global) random oracle cannot hash such a state either. While this could in theory be resolved by combining all primitives into a single functionality, this approach would strongly contradict the modularity of UC.

We therefore carefully model a *challenge generation* mechanism into our functionalities where parties (and the auditor) can obtain unpredictable challenges for higher-level protocols based on the functionality’s state. However, any rushing adversary (or in particular any adversary if all parties are corrupted) can choose its inputs to the functionality, observe which challenge this generates, and adaptively change its choice of inputs. This mimics a real-world attack where an attacker simulates the impact of its messages onto random oracle outputs before posting messages to the bulletin board. Our UC modeling permits such attacks by allowing attackers to obtain this information or even reset functionalities.

Communication complexity and Open problems. In order to quantify the cost of our protocol, we have analyzed its communication complexity in terms of bits sent per party and total number of rounds and separated them by offline and online phase. The offline phase can be completed in 5 rounds while the online phase takes $4 + \mathcal{D}$ rounds, where $\mathcal{D}$ is the circuit depth. For details see Appendix C. While our work addresses some drawbacks of the state-of-the-art, other questions remain open. For example, our construction does not yet work for binary circuits. We describe limitations and open problems in Appendix A.

1.3 Related Work

Cunningham et al. [19] extended [8] by the use of so-called Commitment Enhanced Secret Sharing. The authors introduce the notion of completely identifiable auditability by adding linear homomorphic commitments to each additive share. [19] uses Pedersen commitments so the construction is not post-quantum secure. Schoenmakers and Veeningen [32] use a threshold version of Paillier and generic NIZK, making their work also not post-quantum secure. The authors achieve *universally verifiable* MPC, which extends public auditability to prove correctness of an encryption of a secret result. By relying on a generic NIZK, both the proof size and prover runtime cannot be optimized for the specific MPC scheme. In [31], the authors present an enhanced version of [8,19] that uses lattice-based commitments and additionally supports lower corruption thresholds towards robustness. Their work heavily relies on NIZKs during the offline phase to show correctness of lattice-based commitments, ciphertexts and multiplications and requiring a CRS for the commitment scheme. In comparison, our offline phase is essentially equivalent to the OLE-based preprocessing of [30] and works using any UC-secure OLE. Parties simply commit to the outcomes and run a very lightweight and standard multiplication triple check.

Baldimtsi et al. [4] define the notion of crowd-verifiable zero-knowledge (CVZR) proofs to achieve what they call end-to-end verifiable MPC such that cheating can be detected even if the setup assumption of a well-formed CRS does not hold. Similar to [8], [32] and [19] Baldimtsi et al. consider separation of input and computation parties but in contrast to prior work they also consider corruption of input parties. Our construction does not support separation of input and computation parties. They prove their construction secure in the random oracle model without using trapdoor commitments. They state that their construction as is, is secure when at least one computation party is honest but can be easily altered to achieve public audibility following the notion of [8] when the computation result is publicly decrypted. However, they show that their approach is limited to only certain securely computed functions, and they require super-polynomial security assumptions. In our work, we boost single-receiver commitments to multi-receiver commitments. To combat cheating adversaries that provide inconsistent shares [22] also used pairwise homomorphic commitments. Their construction is secure in the dishonest majority setting and can be instantiated using minimal assumptions. In summary, all previous works either are not post-quantum secure, only support a subset of all possible functions or do not support full corruption. Our construction instead only relies on random oracles, works for all circuits and is secure even if all parties are corrupted. We provide a comparison in Table 1.

	Ours	[8]	[4]	[19]	[22]	[31]	[30]	[32]
Auditable	✓	✓	(✓)	✓	X	✓	X	✓
Composable	UC	UC	UC	UC	UC	UC	UC	X
Assumptions (except RO)	OLE	Lattice+ DLOG	subexp. TPKE	Lattice+ DLOG	PRG+ OT	Lattice	pPCGs	Paillier
Setup	X	CRS	X	CRS	X	CRS	X	TPKE
PQ	✓	X	X	X	✓	✓	✓	X

Table 1: Comparison of related work. All constructions that do not support full corruption are secure in the dishonest majority setting. TPKE = Threshold Public Key Encryption, pPCG = programmable PCG.

2 Preliminaries and Notation

We use $[x..y]$ to denote the interval $\{x, x+1, \dots, y\}$ and write $[x]$ for $[1..x]$. We write vectors in lower-case bold letters such as $\mathbf{x}$, and matrices as upper-case bold letters such as $\mathbf{A}$. For a matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$ we use $\mathbf{A}_i$ to denote the i 'th row and $\mathbf{A}^j$ for the j 'th column. $\mathbf{A}[i, j]$ denotes the element at row i and column j of the matrix $\mathbf{A}$. Similarly, we use $\mathbf{x}[i]$ or x_i to denote the i th element of $\mathbf{x}$.

We use λ for the security parameter. Parties in this work are usually denoted using the symbol $\mathcal{P}$. We write v^i to indicate that v is either sent by or sent to $\mathcal{P}_i$. For a value v with index id we write v_{id} and with v_{id}^i we denote a value v with index id that was sent either to or by $\mathcal{P}_i$. In our protocols and functionalities,

we use lists (using notation `li`) and maps (using notation `mp`) to store values. If elements have to be addressable by an ID they are stored inside maps and otherwise in lists. Flags (which are essentially publicly known variables with notation `fl`) are used to indicate states of functionalities and protocols to ensure that certain operations can only be performed when preliminary actions are completed. Additional preliminaries can be found in Appendix B.

2.1 Linear Codes

Let $\mathbb{F}$ be a finite field and $n_{\mathcal{C}}, k_{\mathcal{C}} \in \mathbb{N}$ be integers. A *linear code* $\mathcal{C} \subset \mathbb{F}^{n_{\mathcal{C}}}$ is a $k_{\mathcal{C}}$ -dimensional vector subspace of $\mathbb{F}^{n_{\mathcal{C}}}$, $1 \leq k_{\mathcal{C}} \leq n_{\mathcal{C}}$. Defining

$$d_{\mathcal{C}} = \min\{\#\text{non-zero indices of } \mathbf{c} \mid \mathbf{c} \in \mathcal{C} \setminus \{\mathbf{0}\}\},$$

we say that $\mathcal{C}$ is a linear code over $\mathbb{F}$ with length $n_{\mathcal{C}}$, dimension $k_{\mathcal{C}}$ and (minimum) distance $d_{\mathcal{C}}$ or $[n_{\mathcal{C}}, d_{\mathcal{C}}, k_{\mathcal{C}}]$ -code for short.

A matrix $\mathbf{G} \in \mathbb{F}^{k_{\mathcal{C}} \times n_{\mathcal{C}}}$ is a *generator matrix* of $\mathcal{C}$ if $\mathbf{x}^T \mathbf{G}$ generates $\mathcal{C}$, i.e. $\mathcal{C} = \{\mathbf{x}^T \mathbf{G} \mid \mathbf{x} \in \mathbb{F}^{k_{\mathcal{C}}}\}$. If $\mathbf{G}$ is of the form $[\mathbf{I}_{k_{\mathcal{C}}} | \mathbf{A}]$, where $\mathbf{I}_{k_{\mathcal{C}}}$ is the $k_{\mathcal{C}} \times k_{\mathcal{C}}$ identity matrix, then $\mathbf{G}$ is called *systematic*.

2.2 UC & Functionalities

We model protocols and their security in this work using the Universal Composability (UC) framework [16]. See Appendix B.2 for a short UC introduction.

Notation. Throughout this work, we will denote functionalities using $\mathcal{F}$, protocols using Π , the adversary as $\mathcal{A}$ and simulators as $\mathcal{S}$. The operations of a functionality act as abstract interfaces that can be invoked by parties and the adversary, independent of any protocol implementation. Those interfaces are usually denoted in **Bold Text** inside a functionality. A first interaction with a functionality $\mathcal{F}$ for such a interface can always either be triggered by an honest party, or by the adversary $\mathcal{A}$ playing the role of a dishonest party. Always stating this fact, however, would make our functionalities harder to read. To simplify notation, we write that a message can be sent by a possibly corrupted party, meaning that formally $\mathcal{A}$ sends the message on behalf of the corrupted party.

Functionalities. We use ideal functionalities for random oracles $\mathcal{F}_{\text{GPRO}}$ and public bulletin boards $\mathcal{F}_{\text{GBB}}$, which can be found in Figure 3 and Figure 2 respectively. Both are modeled as global subroutines. Note that $\mathcal{F}_{\text{GBB}}$ as a bulletin board allows external parties to read from it at any point. It naturally acts as a broadcast channel during a protocol, and we therefore sometimes write *broadcasting* when we mean that a party is writing to $\mathcal{F}_{\text{GBB}}$. We use the formulation of [17] to model our global random oracle.

In their approach, the authors only let functionalities (machines with `pid =  $\perp$`) program the oracle. The calling functionality can initially be assigned control of

the oracle and is then referred to as the controller of the session. Only registered controllers of sid can program points that have not been queried before. In this formulation of the global random oracle the programming access is controlled by the calling functionalities. To not further complicate our functionalities we model this access structure as a macro MACROPRG described in Figure 4. Throughout the paper, we implicitly assume that every functionality first invokes **Init Control** and performs programming to $\mathcal{F}_{\text{GpRO}}$ via the predefined macro. For clarity and brevity, this is not repeated in the description of individual functionalities.

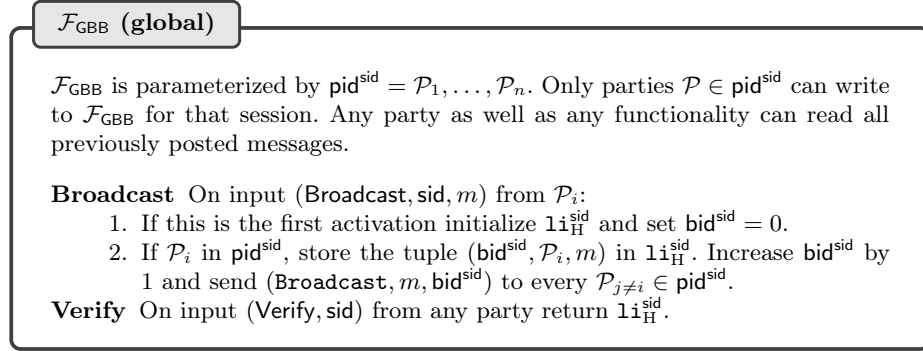


Fig. 2: Ideal global bulletin board functionality.

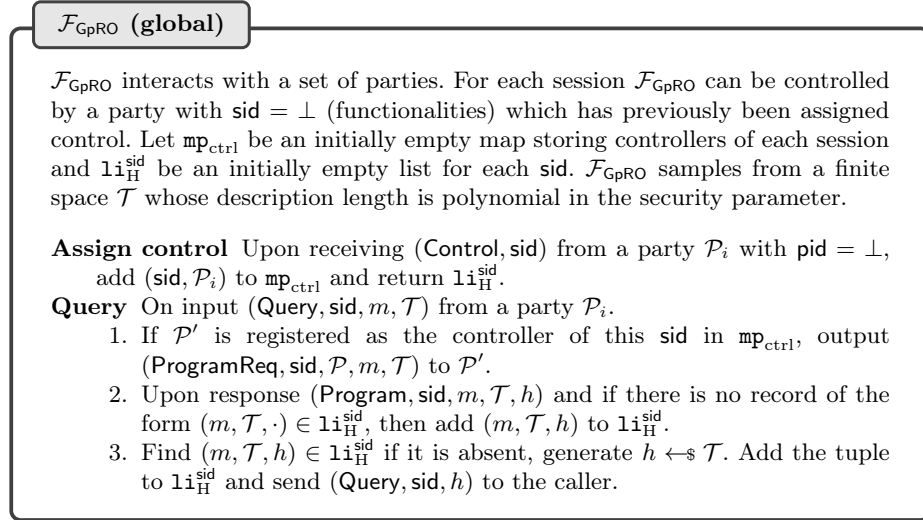


Fig. 3: Global Random Oracle functionality $\mathcal{F}_{\text{GpRO}}$ following [17].

For $\mathcal{F}_{\text{GpRO}}$ we use the following, well-known fact (see e.g. [5]):

MACROPRG

If any of the processes return $\perp$ abort the execution inside the calling functionality. Every functionality keeps track of initially empty map $\mathbf{li}_H^{\text{sid}}$. We only allow programming from the ideal process adversary ($\mathcal{S}$) with same sid as the calling functionality.

Init control send (Control, sid) to $\mathcal{F}_{\text{GPRO}}$ and wait for its response m . If $m = (\text{Ok}, \text{Hist})$ and $\text{Hist} \neq \perp$ output $\perp$.

Program On input (ProgramReq, $\text{sid}, m, \mathcal{T}$) from $\mathcal{F}_{\text{GPRO}}$. Send (Program, $\text{sid}, m, \mathcal{T}, h$) to $\mathcal{S}$ and wait for its response h . Forward (Program, $\text{sid}, m, \mathcal{T}, h$) to $\mathcal{F}_{\text{GPRO}}$.

Fig. 4: Macro for programming the random oracle

Proposition 1 (Random oracle graph game). *Let $H: \{0,1\}^* \rightarrow \mathcal{T}$ be a random oracle and consider following game with an adversary $\mathcal{A}$. The game keeps track of a directed graph $G = (V, E)$, initially empty, and proceeds as follows: The adversary can (repeatedly) query H at some x . Let $H(x) = y$.*

- *If $y \in V$ but there is no edge $(x', y) \in E$, then the adversary wins. (Preimage)*
- *If an edge $(x', y) \in E$ exists with $x' \neq x$, then the adversary wins. (Collision)*
- *Else, add nodes x and y to V and add an edge $e = (x, y)$ from x to y to E .*

Let $\mathcal{A}$ be an adversary which makes at most Q queries to H . Then the probability that $\mathcal{A}$ wins is bounded by $Q^2/|\mathcal{T}|$.

In addition, our protocols also use an Oblivious Linear Evaluation (OLE) functionality (see Figure 5). There exists a variety of protocols to efficiently implement this primitive, such as [23,24]. Protocols based on Pseudorandom Correlation Generators have been proposed, following the breakthrough work of [15]. They implement OLE with sublinear communication (in the correlation length ℓ) where the shares of honest parties are pseudorandom.

$\mathcal{F}_{\text{OLE}}$

$\mathcal{F}_{\text{OLE}}$ interacts between two parties $\mathcal{P}_S$ and $\mathcal{P}_R$. The functionality is parameterized by the finite field $\mathbb{F}$ and output length ℓ .

Extend On input (Extend, $\text{sid}, \mathbf{u}$) from $\mathcal{P}_S$ and input (Extend, $\text{sid}, \mathbf{x}$) from $\mathcal{P}_R$ where $\mathbf{u}, \mathbf{x} \in \mathbb{F}^\ell$.

1. If $\mathcal{P}_R$ is corrupt wait for $\mathcal{A}$ to send $\mathbf{v}$. Otherwise sample $\mathbf{v} \leftarrow \mathbb{F}^\ell$.
2. If $\mathcal{P}_S$ is honest, compute $\mathbf{w} = \mathbf{u} \cdot \mathbf{x} + \mathbf{v}$. If $\mathcal{P}_S$ is corrupt wait for $\mathcal{A}$ to send $\mathbf{w}$ and recompute $\mathbf{v}$ accordingly.
3. Output $\mathbf{w}$ to $\mathcal{P}_S$ and $\mathbf{v}$ to $\mathcal{P}_R$.

Fig. 5: OLE functionality $\mathcal{F}_{\text{OLE}}$.

2.3 VOLEs and VOLE-based MACs

Given a field $\mathbb{F}$, a VOLE correlation of length ℓ is a pair of random variables $(\mathbf{w}, \mathbf{u}) \in \mathbb{F}^\ell \times \mathbb{F}^\ell$ and $(\mathbf{v}, \Delta) \in \mathbb{F}^\ell \times \mathbb{F}$, subject to the constraint that $w_i = v_i + u_i \cdot \Delta$. The correlation exists between two parties, the sender $\mathcal{P}_S$ and receiver $\mathcal{P}_R$. $\mathcal{P}_S$ is given $(\mathbf{w}, \mathbf{u})$, while $\mathcal{P}_R$ learns $(\mathbf{v}, \Delta)$. It is an instantiation of the correlation generated by $\mathcal{F}_{\text{OLE}}$ where all x_i are Δ .

VOLE can be used to build a simple, information-theoretic MAC scheme with homomorphic properties for use in MPC protocols, such as described in [13]. There, the secret held by the sender is $\mathbf{u}$ while the MACs are defined by $\mathbf{w}$. $\Delta, \mathbf{v}$ are the keys held by the verifier. To see that the scheme is linearly homomorphic, observe that for any $\alpha, \delta \in \mathbb{F}$ and VOLE correlations $w_1 = v_1 + u_1 \Delta$ and $w_2 = v_2 + u_2 \Delta$, it holds that

$$\underbrace{\alpha w_1 + w_2}_{\text{computed by } \mathcal{P}_S} = \underbrace{\alpha v_1 + v_2 - \Delta \delta}_{\text{computed by } \mathcal{P}_R} + \underbrace{(\alpha u_1 + u_2 + \delta) \Delta}_{\text{computed by } \mathcal{P}_S}$$

is a valid VOLE correlation for the secret $\alpha u_1 + u_2 + \Delta$ whose MACs and keys can be locally computed by $\mathcal{P}_S, \mathcal{P}_R$ respectively.

To open a secret u , the sender sends both u, w to $\mathcal{P}_R$, which in turn checks that $w = u \cdot \Delta + v$. The MAC w cannot be forged for a different secret $u' \neq u$ with probability larger than $1/|\mathbb{F}|$ (see e.g. [9, Section 3.1] for a proof):

Lemma 1. *Let v, Δ be a VOLE commitment to u with MAC w . Assume that $\mathcal{P}_S$ sends $u', w' \in \mathbb{F}$ to $\mathcal{P}_R$ such that $u \neq u'$. Let $\mathcal{P}_R$ accept if $w' = u' \Delta + v$, and otherwise reject. Then $\mathcal{P}_R$ accepts with probability at most $1/|\mathbb{F}|$.*

3 Formalizing Auditable MPC

We now recap the UC formalization of Publicly Verifiable (Auditable) Secure Multiparty Computation as first presented in [8].

In Figure 6 we present the ideal functionality $\mathcal{F}_{\text{Online}}$. In $\mathcal{F}_{\text{Online}}$ each party can input private values $x^i \in \mathbb{F}$ for a fixed field $\mathbb{F}$ and compute a predetermined circuit using these values. The circuit can be built from multiplication and linear gates corresponding to the same operations in $\mathbb{F}$. Moreover, the parties $\mathcal{P}$ can, upon agreement, open the outputs of the computation. In addition, an auditor $\mathcal{P}_A$ is an additional party that can interact with the functionality and verify the correctness of the computation output y after it is opened. The auditor will, on input y , output **accept** if the circuit $\mathcal{C}$ on input $x^1, \dots, x^n$ outputs y . If the value y is not identical to the output of $\mathcal{C}$, the auditor will output **reject**.

Parties. The functionality interacts with a set of n parties $\mathcal{P}_1, \dots, \mathcal{P}_n$ that provide inputs and evaluate the circuit $\mathcal{C}$. The adversary $\mathcal{A}$ can corrupt arbitrarily many of the parties. If at least one party is honest correctness and input privacy

$\mathcal{F}_{\text{Online}}$

$\mathcal{F}_{\text{Online}}$ interacts with an adversary $\mathcal{A}$, a set of computation parties $\mathcal{P}$ and an auditor $\mathcal{P}_A$. Let $\mathbb{F}$ be a field.

Initialize On input $(\text{Init}, \text{sid}, \mathcal{C})$ from all parties in $\mathcal{P}$, where $\mathcal{C}$ is a circuit with n inputs, one output, and addition and multiplication gates over $\mathbb{F}$:

1. $\mathcal{A}$ sends the set of corrupted parties $\tilde{\mathcal{P}} \subseteq \mathcal{P}$.

Input On input $(\text{Input}, \text{sid}, i, \text{id}_x, x)$ from $\mathcal{P}_i$ with $x \in \mathbb{F}$ and $(\text{Input}, \text{sid}, i, \text{id}_x)$ from all other parties.

1. Store (id_x, x) and ignore all messages $(\text{Input}, \text{sid})$ with the same id_x .
2. If all parties are corrupted, send $(\text{Input}, \text{sid}, i, \text{id}_x, x)$ to $\mathcal{A}$.

Compute On input $(\text{Compute}, \text{sid})$ from all parties $\mathcal{P}_i$:

1. Assign inputs to input gates of $\mathcal{C}$. If some input gates are unassigned, output $(\text{Abort}, \text{sid})$.
2. Compute $y_c = \mathcal{C}(x_1, \dots, x_n)$.
3. Send y_c to $\mathcal{A}$ and wait for $\mathcal{A}$ to respond with y^* . If $|\tilde{\mathcal{P}}| = n$ all values $y^* \in \mathbb{F} \cup \{\perp\}$ are valid. If $|\tilde{\mathcal{P}}| < n$ only $y^* \in \{y_c, \perp\}$ is valid.
4. Send $(\text{Output}, \text{sid}, y^*)$ to all $\mathcal{P}_i$ if y^* is valid, and $(\text{Abort}, \text{sid})$ otherwise.

Audit On input $(\text{Audit}, \text{sid}, y)$ from $\mathcal{P}_A$ and if **Compute** has finished:

1. If $y = y_c = y^*$ send $(\text{Accept}, \text{sid})$ to $\mathcal{P}_A$.
2. If $y^* = \perp$ send $(\text{AuditNotPossible}, \text{sid})$ to $\mathcal{P}_A$.
3. If $y \neq y_c$ or $y \neq y^*$ send $(\text{Reject}, \text{sid})$ to $\mathcal{P}_A$.

Fig. 6: Functionality $\mathcal{F}_{\text{Online}}$ of Auditable MPC.

are guaranteed. If all parties are dishonest only correctness can be guaranteed and the auditor $\mathcal{P}_A$ can verify the output after the computation of $\mathcal{C}$ is finished. We model $\mathcal{P}_A$ as an incorruptible party.

Realizing $\mathcal{F}_{\text{Online}}$. To maintain a state that is shared by all protocols we use session identifiers sid throughout our construction. When realizing $\mathcal{F}_{\text{Online}}$, all steps of the computation are sent to the bulletin board, such that $\mathcal{P}_A$ can later get the transcript of the computation. We present the protocol in Section 6.

4 One-time Homomorphic Commitments

In this section, we build a one-time homomorphic commitment scheme from a weak form of VOLE. Namely, we use a special VOLE functionality, whose ideas were introduced in [7], with publicly revealed Δ to implement VOLE-based commitments. The commitment functionality $\mathcal{F}_{\text{HOTCom}}$ that we realize is described in Figures 7 and 8.

We now describe the functionality in more detail. The functionality $\mathcal{F}_{\text{HOTCom}}$ is a multi-receiver commitment scheme, run between a sender $\mathcal{P}_S$, a set of receivers $\mathcal{P}_R$ and an auditor $\mathcal{P}_A$. After initialization using **Init**, the sender may create mc commitments in multiple rounds using **Commit**. Linear operations can then be applied to previously committed values using **Add** and **Linear**. Commitments

$\mathcal{F}_{\text{HOTCom}}$ (Part 1)

$\mathcal{F}_{\text{HOTCom}}$ interacts with the sender $\mathcal{P}_S$, a set of receivers $\mathcal{P}_R$, an auditor $\mathcal{P}_A$ and an adversary $\mathcal{A}$ who is allowed to corrupt every party except $\mathcal{P}_A$. If $\mathcal{A}$ or a party sends (Abort, sid) it halts.

The functionality is parameterized by a field $\mathbb{F}_p$. Let fl_{open} be a flag that is initially 0 and $\text{li}_{\text{gen}}, \text{li}_{\text{open}}, \text{li}_{\text{chal}}$ be lists and $\text{mp}_{\text{com}}, \text{mp}_{\text{chal}}$ be maps - all are initially empty. Challenges are generated from a finite set $\mathcal{T}$.

Init Upon input (Init, sid, mc) from $\mathcal{P}_S$ set $\text{li}_{\text{gen}} := \{\text{cid}_1, \dots, \text{cid}_{\text{mc}}\}$ and output (Init, sid, mc) to all parties in $\mathcal{P}_R$ and $\mathcal{A}$. Let $\mathcal{U}$ be the set of all $\mathbb{F}_p$ -linear functions in $\text{cid}_1, \dots, \text{cid}_{\text{mc}}$.

Commit On input (Commit, sid, $(\text{cid}_j, m_j)_{j \in [k]}$) from $\mathcal{P}_S$, if $\text{fl}_{\text{open}} = 0$, $\{\text{cid}_j\}_{j \in [k]} \subseteq \text{li}_{\text{gen}}$, $m_j \in \mathbb{F}_p$ and $(\text{cid}_j, *) \notin \text{mp}_{\text{com}}$ for all $j \in [k]$:

1. Let (state, *) be the last item on li_{chal} or state = $\perp$ if it is empty. If $\mathcal{P}_S$ is corrupted, wait for message (ChalSel, sid, c) and let **ch** be as in $(\text{li}_{\text{gen}} \| (\text{state}, (\text{cid}_j, m_j)_{j \in [k]})) \| c, \text{ch} \in \text{mp}_{\text{chal}}$. If sender is honest, set $\text{ch} \xleftarrow{\$} \mathcal{T}$. Add $(\text{state} \| (\text{cid}_j, m_j)_{j \in [k]}, \text{ch})$ to li_{chal} .
2. Add the value $(\text{cid}_j, (m_j, \text{id}_j))$ for all $j \in [k]$ to mp_{com} where id_j is the identity function in $\mathcal{U}$.
3. Send (Receipt, sid, $(\text{cid}_j)_{j \in [k]}, \text{ch}$) to all $\mathcal{P}_i \in \mathcal{P}_R$, $\mathcal{P}_S$ and $\mathcal{A}$.

Challenge Test On input (TestChallenge, sid, mc, L) from $\mathcal{A}$ where $L = (\text{cid}_j, m_j)_{j \in [k]}$ is a list with elements with each $\text{cid}_j \in T = \{\text{cid}_1, \dots, \text{cid}_{\text{mc}}\}$ and if $\mathcal{P}_S$ is corrupted:

1. If $(T \| L \| i, *) \in \text{mp}_{\text{chal}}$ set $c = i + 1$ for the largest i , otherwise $c = 0$.
2. Sample $\text{ch} \xleftarrow{\$} \mathcal{T}$, add $(T \| L \| c, \text{ch})$ to mp_{chal} and send (ch, c) to $\mathcal{A}$.

Hint On input (Hint, sid, $(\text{cid}_j)_{j \in [k]}, i$) from $\mathcal{P}_S$, if $(\text{cid}_j, (m_j, *)) \in \text{mp}_{\text{com}}$ for every $j \in [k]$, $\text{fl}_{\text{open}} = 0$ and $i \in \{0, \dots, |\mathcal{P}_R|\}$:

1. If $\mathcal{P}_S$ is corrupt, wait for input (Hint, sid, $(x_j)_{j \in [k]}$) from $\mathcal{A}$ where $x_j \in \mathbb{F}_p$. Otherwise set $x_j = m_j$.
2. For each $j \in [k]$ if $x_j := m_j$ add $(\text{cid}_j, \neg\text{delay}, x_j, 1, i)$ to li_{open} , otherwise add $(\text{cid}_j, \neg\text{delay}, x_j, 0, i)$.
3. If $i = 0$ send (Hint, sid, $(\text{cid}_j, x_j)_{j \in [k]}$) to all $\mathcal{P}_i \in \mathcal{P}_R$ and $\mathcal{A}$.
4. If $i \neq 0$ send (PrivHint, sid, $(\text{cid}_j, x_j)_{j \in [k]}$) to $\mathcal{P}_i$.

Delayed Hint On input (DelHint, sid, $(\text{cid}_j)_{j \in [k]}$) from $\mathcal{P}_S$, if $(\text{cid}_j, (m_j, *)) \in \text{mp}_{\text{com}}$ for every $j \in [k]$ and if $\text{fl}_{\text{open}} = 0$:

1. If $\mathcal{P}_S$ is corrupt, wait for input (Hint, sid, $(x_j)_{j \in [k]}$) from $\mathcal{A}$ where $x_j \in \mathbb{F}_p$. Otherwise set $x_j = m_j$.
2. For every $j \in [k]$ if $x_j = m_j$ add $(\text{cid}_j, \text{delay}, x_j, 1)$ to li_{open} , otherwise add $(\text{cid}_j, \text{delay}, x_j, 0)$.
3. Send (DelHint, sid, $(\text{cid}_j)_{j \in [k]}$) to all parties in $\mathcal{P}_R$.

Fig. 7: Ideal functionality for one-time homomorphic commitments.

and linear operations can be done until the sender calls **Open**. Thereafter, these operations are not permitted anymore as a special flag fl_{open} is then set. Commitments and operations on them are internally stored in a special map mp_{com}

$\mathcal{F}_{\text{HOTCom}}$ (Part 2)

Add On input $(\text{Add}, \text{sid}, (\text{cid}_j)_{j \in [3]})$ from $\mathcal{P}_S$ and if $\text{fl}_{\text{open}} = 0$, check if $(\text{cid}_3, *) \notin \text{mp}_{\text{com}}$, $\text{cid}_3 \notin \text{li}_{\text{gen}}$ and $\{(\text{cid}_1, (m_1, f_1)), (\text{cid}_2, (m_2, f_2))\} \subseteq \text{mp}_{\text{com}}$:

1. Store tuple $(\text{cid}_3, (m_1 + m_2, f_1 + f_2))$ in mp_{com} .
2. Send $(\text{Add}, \text{sid}, (\text{cid}_j)_{j \in [3]})$ to all $\mathcal{P}_i \in \mathcal{P}_R$ and $\mathcal{A}$.

Linear On input $(\text{Linear}, \text{sid}, \text{cid}_1, \text{cid}_2, \alpha, \delta)$ from $\mathcal{P}_S$ and if $\text{fl}_{\text{open}} = 0$, if $(\text{cid}_2, *) \notin \text{mp}_{\text{com}}$, $\text{cid}_2 \notin \text{li}_{\text{gen}}$ and $(\text{cid}_1, (m_1, f_1)) \in \text{mp}_{\text{com}}$:

1. Store $(\text{cid}_2, \alpha \cdot m_1 + \delta, \alpha \cdot f_1 + \delta)$ in mp_{com} .
2. Send $(\text{Linear}, \text{sid}, \text{cid}_1, \text{cid}_2, \alpha, \delta)$ to all $\mathcal{P}_i \in \mathcal{P}_R$ and $\mathcal{A}$.

Open On input $(\text{Open}, \text{sid})$ from $\mathcal{P}_S$ and if $\text{fl}_{\text{open}} = 0$: Set $\text{fl}_{\text{open}} \leftarrow 1$ and send $(\text{Open}, \text{sid})$ to all parties.

Delayed Open On input $(\text{Open}, \text{sid})$ from $\mathcal{P}_S$ and if $\text{fl}_{\text{open}} = 1$:

1. Let $L := \{(\text{cid}_j, x_j) \mid (\text{cid}_j, \text{delay}, x_j, b_j) \in \text{li}_{\text{open}}\}$.
2. Replace each $(\text{cid}_j, \text{delay}, x_j, b_j) \in \text{li}_{\text{open}}$ with $(\text{cid}_j, \neg \text{delay}, x_j, b_j, 0)$.
3. Set $\text{fl}_{\text{open}} \leftarrow 2$ and send $(\text{Open}, \text{sid}, L)$ to each party in $\mathcal{P}_R$.

Verify On input $(\text{Verify}, \text{sid}, (\text{cid}_j, m_j)_{j \in [k]})$ from any party $\mathcal{P}_i \in \mathcal{P}_R$ and if $\text{fl}_{\text{open}} \neq 0$:

- Return $(\text{Verified}, \text{sid})$ if $(\text{cid}_j, \neg \text{delay}, m_j, 1, 0) \in \text{li}_{\text{open}}$ or $(\text{cid}_j, \neg \text{delay}, m_j, 1, i) \in \text{li}_{\text{open}}$ for every $j \in [k]$
- Otherwise return $(\text{VerifyFailed}, \text{sid})$.

Audit On input $(\text{Audit}, \text{sid}, (\text{cid}_j, f_j)_{j \in [k]})$ by $\mathcal{P}_A$, if $\text{cid}_j \in \mathcal{U}$ and $\text{fl}_{\text{open}} = 2$:

1. Let $L := \{(\text{cid}_j, m_j) \mid (\text{cid}_j, \neg \text{delay}, m_j, 1, 0) \in \text{li}_{\text{open}}\}$.
2. Return $(\text{Audit}, \text{sid}, L)$ if $(\text{cid}_j, \neg \text{delay}, m_j, 1, 0) \in \text{li}_{\text{open}}$ and $(\text{cid}_j, (m_j, f_j)) \in \text{mp}_{\text{com}}$, for every $j \in [k]$. Otherwise return $(\text{AuditFailed}, \text{sid})$.

Challenges On input $(\text{Challenges}, \text{sid})$ from any party or $\mathcal{A}$ output $L = ((\text{cid}_j), \text{ch}_k) \mid ((\text{cid}_j, m_j), \text{ch}_k) \in \text{li}_{\text{chal}}$.

Fig. 8: Ideal functionality for one-time homomorphic commitments, continued.

based on unique commitment IDs cid . Some of the commitment IDs are reserved for using **Commit** and stored in a list li_{gen} .

Instead of multiple rounds of openings of commitments, the functionality permits so-called *hints*. A hint allows an honest party to reveal a committed value in a verifiable way, while a dishonest sender can change the hint outputs at will. However, since hints are verifiable, such a change to the outputs is visible after **Open** is called. Hints can be public and private, meaning that output can either be opened to an individual receiver or all receivers in a verifiable way. *Private hints* are only verifiable by the receiver and no other party.

A sender can also *send hints to the future* by using **Delayed Hints** before calling **Open**. Revealing these hints through **Delayed Open** can only be used after **Open**. The (dishonest) sender can always decide not to reveal such a hint at this point, but it cannot change the value that is revealed anymore. Also, once such a delayed hint is revealed it is verifiable by all parties, just as regular

hints. $\mathcal{F}_{\text{HOTCom}}$ keeps track of hints and openings in the list li_{open} , including information about delayed hints now being visible or receivers of hints.

Audits. The auditor $\mathcal{P}_A$ acts as an additional receiver that is incorruptible. It can, however, not obtain private hints. In comparison to the other parties, it can only verify hints after **Delayed Open** is called. At that point, it can verify both regular and delayed hints.

Generating challenges. The protocol implementing $\mathcal{F}_{\text{HOTCom}}$ consists of random oracle calls and messages sent to a bulletin board $\mathcal{F}_{\text{GBB}}$. As such, this permits generating unpredictable challenges (to be used by protocols using $\mathcal{F}_{\text{HOTCom}}$ as a building block) using the Fiat-Shamir paradigm where the messages on the Bulletin Board are hashed. However, this allows a corrupt sender, ahead of time, to *simulate* protocol transcripts before posting to $\mathcal{F}_{\text{GBB}}$, thus seeing what Fiat-Shamir challenges the simulated transcripts lead to. To capture this advantage, we additionally allow the attacker to use the functionality such that it shows which challenges its input choices generate. That is, we *explicitly* model challenge generation into the functionality, as well as explicit challenge testing which the adversary can do using **Challenge Test**.

During **Commit**, the dishonest sender can then pick any such tested challenge that it observed before. Based on the state, challenges can then be read by other parties such as the auditor using **Challenges**. The challenge space (essentially, the output space of the underlying random oracle) is defined by $\mathcal{T}$. To ensure consistency, mappings between functionality states and generated challenges are stored in a map mp_{chal} . The different challenges that can be derived publicly given the current and previous states of the functionality are stored in li_{chal} .

4.1 Weak VOLE

To implement $\mathcal{F}_{\text{HOTCom}}$ we use a *weak* form of VOLE, which we describe in $\mathcal{F}_{\text{WVOLE}}$ in Figure 9. It implements a form of VOLE correlation generation, described in Section 2.3 to allow linear commitments. However, it does not generate the correlation shares of the sender and the receiver simultaneously. Instead, initially $\mathcal{P}_S$ learns its shares of the correlation, as well as Δ . The choice of Δ is dependent on the message that it intends to send on the broadcast channel, which is modeled by the additional input m . $\mathcal{P}_R$ learns Δ, m , but initially not $\mathbf{Q}$. However, once the parties enter **Deliver** then $\mathbf{Q}$ is forwarded to $\mathcal{P}_R$. Note that every choice of m leads to a different Δ , and an adversary can test which m generates which Δ without generating interaction via **Delta Test**. This reflects that the implementation later generates Δ as the output of a hash function evaluated on the protocol transcript and m . The functionality stores the (m, Δ) that is chosen by the adversary in the flag fl_{Δ} . The functionality does not keep any secret from the sender if it is dishonest. In addition, $\mathcal{F}_{\text{WVOLE}}$ allows an auditor to obtain the same information as $\mathcal{P}_R$ at any later point. It is precisely the use of m that ties $\mathbf{Q}, \Delta$ to an external context (such as other messages on a broadcast channel) making audit meaningful even if both $\mathcal{P}_S, \mathcal{P}_R$ are corrupted.

To allow using $\mathcal{F}_{\text{WVole}}$ in a Fiat-Shamir context by other protocols, the functionality also generates an additional challenge **ch** once $\mathbf{U}, \mathbf{V}$ are fixed by the sender *but before the VOLE challenge Δ is chosen*. This allows higher-level protocols to derive an unpredictable value **ch** once inputs are fixed. This is in line with an actual implementation where $\mathbf{U}, \mathbf{V}$ can already be extracted from random oracle-based commitments before the opening takes place later, for which Δ is necessary. The challenge **ch** will later be computed by hashing RO commitments containing $\mathbf{U}, \mathbf{V}$. Since the adversary could simulate how multiple choices of $\mathbf{U}, \mathbf{V}$ and commitments thereof generate different hash function outputs and thus **ch** values, we allow the adversary to test output challenges using **Challenge Test** before inputting them to the functionality. As all this information is available as a public transcript and **ch** can also be derived by other parties, the functionality allows third parties to generate challenges using **Challenge**. As in $\mathcal{F}_{\text{HOTCom}}$, we use a map mp_{chal} to keep track of all challenges generated to the adversary and store the ultimately chosen challenge in the flag fl_{ch} .

A previous (interactive) version of $\mathcal{F}_{\text{WVole}}$ was presented in [7], where the authors also show how to instantiate it. We adjust their protocols to fit our functionality. The asynchronous generation of $\Delta, \mathbf{Q}$ can easily be realized by running the original protocol from [7], but where Δ is now chosen based on a random oracle evaluated on the sender’s message. $\mathcal{P}_A$ ’s operations can be permitted by letting the parties communicate via a broadcast channel. This way, $\mathcal{P}_A$ can obtain $\Delta, \mathbf{Q}$ and other information the same way as $\mathcal{P}_R$ does in the protocol. These modifications to the protocol and the security proof of [7] can be found in Appendix D.

4.2 From weak VOLE to single-receiver one-time commitments

We will now describe a protocol that implements $\mathcal{F}_{\text{HOTCom}}$ in the $\mathcal{F}_{\text{WVole}}$ -hybrid model for $|\mathcal{P}_R| = 1$, i.e. for one receiver, working over $\mathbb{F}_p$ for large values of p . For small p , we refer to Appendix A.

How to set up commitments. Between sender $\mathcal{P}_S$ and receiver $\mathcal{P}_R$, we assume the existence of one instance of $\mathcal{F}_{\text{WVole}}$ where $\mathcal{P}_S$ is the sender and $\mathcal{P}_R$ acts as receiver. The parties wish to commit to at most mc messages, where k_C is the dimension of an underlying linear code $\mathcal{C}$ and we set $\ell = \lceil \text{mc}/k_C \rceil$. The parties run **Init** on $\mathcal{F}_{\text{WVole}}$ and $\mathcal{P}_S$ learns $\mathbf{U}, \mathbf{V}$. Each of these matrices have 2ℓ rows, where we use $\mathbf{U}$ to define the commitments to messages and $\mathbf{V}$ to perform consistency checks. We refer to the commitments from $\mathbf{U}$ as “level 1” VOLEs, and the VOLEs in the consistency check (which additionally use $\mathbf{V}$) as “level 0” VOLEs.

More concretely, sender uses $\mathbf{U} \in \mathbb{F}_p^{2\ell \times k_C}$ to define a “level 1” VOLE with a yet unknown challenge Δ_1 . The upper half of $\mathbf{U}$, i.e. the first ℓ rows, will be used for the message part of the VOLE (the degree-1 coefficient, which we call $\mathbf{U}_1$) commitments, while its lower half of $\mathbf{U}$ will act as the MAC (the degree 0 coefficient of each VOLE, also called $\mathbf{U}_2$). That means, the “level 1” VOLEs are

The functionality interacts with a sender $\mathcal{P}_S$, a receiver $\mathcal{P}_R$, an adversary $\mathcal{A}$, an auditor $\mathcal{P}_A$ and the functionality $\mathcal{F}_{\text{GPRO}}$. Upon initialization, $\mathcal{A}$ can corrupt any of $\mathcal{P}_S, \mathcal{P}_R$ or even both, but never $\mathcal{P}_A$. $\mathcal{F}_{\text{WVOLE}}$ is parametrized by a prime p as well as an $[nc, kc, dc]_p$ linear code $\mathcal{C}$ over $\mathbb{F}_p$ with generator matrix $\mathbf{G} \in \mathbb{F}_p^{kc \times nc}$ and a set $[N] \subseteq \mathbb{F}_p$. Let $\mathcal{T}$ be a set of possible challenges and $\mathbf{fl}_{\text{ch}}, \mathbf{fl}_{\Delta}$ be flags that are initially $\perp$. Let $\mathbf{mp}_{\text{chal}}$ be a map that is initially empty.

Init Upon first input $(\text{Init}, \text{sid}, \ell)$ from $\mathcal{P}_S$:

1. If $\mathcal{P}_S$ is honest, then sample $\mathbf{U} \xleftarrow{\$} \mathbb{F}_p^{\ell \times kc}$, $\mathbf{V} \xleftarrow{\$} \mathbb{F}_p^{\ell \times nc}$, and $\mathbf{ch} \xleftarrow{\$} \mathcal{T}$, set $\mathbf{fl}_{\text{ch}} \leftarrow (\ell, \mathbf{ch})$ and send $(\text{Sender}, \text{sid}, \mathbf{U}, \mathbf{V}, \mathbf{ch})$ to $\mathcal{P}_S$.
2. If $\mathcal{P}_S$ is corrupt, receive $(\text{Sender}, \text{sid}, \mathbf{U}, \mathbf{V}, c)$ from $\mathcal{A}$ where $(\ell \parallel \mathbf{U} \parallel \mathbf{V} \parallel c, \mathbf{ch}) \in \mathbf{mp}_{\text{chal}}$. Then set $\mathbf{fl}_{\text{ch}} \leftarrow (\ell, \mathbf{ch})$.
3. Finally, send $(\text{Init}, \text{sid}, \ell, \mathbf{ch})$ to $\mathcal{P}_R$.

Init Done Upon first input $(\text{InitDone}, \text{sid}, m)$ from $\mathcal{P}_S$ and if $\mathbf{fl}_{\text{ch}} \neq \perp$:

1. Set $\Delta \leftarrow \text{getDelta}(\text{sid}, \ell, \mathbf{ch}, m)$ and $\mathbf{fl}_{\Delta} \leftarrow (m, \Delta)$.
2. Finally, send $(\text{Done}, \text{sid}, m, \Delta)$ to both $\mathcal{P}_S$ and $\mathcal{P}_R$.

Deliver Upon first input $(\text{Deliver}, \text{sid})$ from $\mathcal{P}_S$ and if $\mathbf{fl}_{\Delta} \neq \perp$:

1. Compute $\mathbf{Q} := \mathbf{V} + \mathbf{U}\mathbf{G}\text{diag}(\Delta)$ and send $(\text{Corr}, \text{sid}, \mathbf{Q})$ to $\mathcal{P}_R$.

Challenge Test On input $(\text{TestChallenge}, \text{sid}, \ell, \mathbf{U}, \mathbf{V})$ from $\mathcal{A}$ where $\mathbf{U} \in \mathbb{F}_p^{\ell \times kc}$ and $\mathbf{V} \in \mathbb{F}_p^{\ell \times nc}$:

1. If $(\ell \parallel \mathbf{U} \parallel \mathbf{V} \parallel *, *) \in \mathbf{mp}_{\text{chal}}$ then let $i = c + 1$ where c is the highest such value. Otherwise, let $c = 0$.
2. Sample $\mathbf{ch} \xleftarrow{\$} \mathcal{T}$, store $(\ell \parallel \mathbf{U} \parallel \mathbf{V} \parallel c, \mathbf{ch})$ in $\mathbf{mp}_{\text{chal}}$ and send $(c, \mathbf{ch})$ to $\mathcal{A}$.

Delta Test On input $(\text{TestDelta}, \text{sid}, \ell, \mathbf{ch}, m)$ from $\mathcal{A}$ where $\mathbf{ch} \in \mathcal{T}$ and $m \in \{0, 1\}^*$: Return $\text{getDelta}(\text{sid}, \ell, \mathbf{ch}, m)$.

Audit Upon receiving $(\text{Audit}, \text{sid})$ from $\mathcal{P}_A$:

- If no $(\text{Corr}, \text{sid})$ message was sent to $\mathcal{P}_R$, then return $(\text{AuditFailed}, \text{sid})$.
- Otherwise return $(\text{Corr}, \text{sid}, \ell, \mathbf{ch}, m, \Delta, \mathbf{Q})$ where $\mathbf{fl}_{\text{ch}} = (\ell, \mathbf{ch}), \mathbf{fl}_{\Delta} = (m, \Delta)$.

Challenge Upon receiving $(\text{Challenge}, \text{sid})$ from any party or $\mathcal{A}$ let $\mathbf{fl}_{\text{ch}} = (\ell, \mathbf{ch})$ and return $(\text{Challenge}, \text{sid}, \mathbf{ch})$.

Procedure getDelta(sid, ℓ , $\mathbf{ch}$, m):

1. If $(\ell \parallel \mathbf{ch} \parallel m, \Delta') \in \mathbf{mp}_{\text{chal}}$ then set $\Delta := \Delta'$
2. Otherwise sample $\Delta \xleftarrow{\$} [N]^{nc}$ and store $(\ell \parallel \mathbf{ch} \parallel m, \Delta)$ in $\mathbf{mp}_{\text{chal}}$.
3. Return Δ .

Fig. 9: Ideal Functionality $\mathcal{F}_{\text{WVOLE}}^{p, N, \mathcal{C}}$ for auditable weak VOLE.

formed by the equation $\mathbf{U}_1 X + \mathbf{U}_2$ modulo p where X is the indeterminate, to be replaced later by Δ_1 . After **Init** terminates, the sender therefore holds the secrets and MACs of *random* level-1 VOLEs which it can use to commit.

For each linear operation on the level-1 VOLE commitments, sender and receiver keep track of the linear functions which they apply to the level-1 VOLE commitments generated as mentioned above. Given an input m to be committed to and an unused index $(i, j) \in [\ell] \times [kc]$ in $\mathbf{U}_1$ (i.e. an unused level-1 VOLE),

$\Pi_{\text{HOTCommit}}^{1-1}$ (Part 1)

The protocol is run by a sender $\mathcal{P}_S$, a receiver $\mathcal{P}_R$ and an auditor $\mathcal{P}_A$. It is parametrized by a prime p as well as the $[nc, kc, dc]_p$ linear code $\mathcal{C}$ over $\mathbb{F}_p$. We define the generator matrix $\mathbf{G} \in \mathbb{F}_p^{kc \times nc}$ for $\mathcal{C}$ as well as a set $S_\Delta \subseteq \mathbb{F}_p$. There exists an instance $\mathcal{F}_{\text{WVole}}$ of $\mathcal{F}_{\text{WVole}}^{p, N, \mathcal{C}}$ between $\mathcal{P}_S$ and receiver $\mathcal{P}_R$ which $\mathcal{P}_A$ can also access. Moreover, all parties have access to a random oracle $\mathcal{F}_{\text{GPRO}}$ and bulletin board $\mathcal{F}_{\text{GBB}}$. Let $\mathcal{T}$ be a challenge space. In the following, when we say that a party aborts then it also broadcasts (Abort, sid).

Each party maintains initially empty lists li_{gen} , li_{chal} and maps mp_{com} , mp_{open} . We denote by $f_j \in \mathbb{F}_p[X_1, \dots, X_{\ell k_C}]$ the polynomial $f_j(x_1, \dots, x_{\ell k_C}) = x_j$. For any degree-1 polynomial f let $f^{(0)}$ denote the polynomial that consists of the degree-0 monomials of f and $f^{(1)}$ be the sum of the degree-1 monomials of f .

Init On first input (Init, sid, mc) to $\mathcal{P}_S$:

1. Let $\ell := \lceil mc/k_C \rceil$. Send (Init, sid, 2ℓ) to $\mathcal{F}_{\text{WVole}}$. $\mathcal{P}_S$ obtains $\mathbf{U} \in \mathbb{F}_p^{2\ell \times k_C}$, $\mathbf{V} \in \mathbb{F}_p^{2\ell \times nc}$ and $\overline{\text{ch}} \in \mathcal{T}$ from $\mathcal{F}_{\text{WVole}}$.
2. Set $\mathbf{V}_1 := \mathbf{V}_{[1..\ell]}$, $\mathbf{V}_2 := \mathbf{V}_{[\ell+1..2\ell]}$, $\mathbf{U}_1 := \mathbf{U}_{[1..\ell]}$ and $\mathbf{U}_2 := \mathbf{U}_{[\ell+1..2\ell]}$. We denote by $\mathbf{u}_1, \dots, \mathbf{u}_\ell$ the ℓ rows of $\mathbf{U}_2$.
3. For each $j \in [\ell k_C]$ add cid_j to its list li_{gen} .

$\mathcal{P}_R$, upon obtaining (Init, sid, 2ℓ , $\overline{\text{ch}}$), for each $j \in [\ell k_C]$ adds cid_j to li_{gen} .

Commit On input (Commit, sid, $(\text{cid}_{i_j}, m_j)_{j \in [k]}$) to $\mathcal{P}_S$ where each $\text{cid}_{i_j} \in \text{li}_{\text{gen}}$ and $m_j \in \mathbb{F}_p$:

1. $\mathcal{P}_S$ for each $j \in [k]$ writes $x = i_j \bmod \ell$ and $y := (i_j - x)/\ell$. Then it broadcasts $d_j := m_j - \mathbf{U}[x+1, y+1]$.
2. $\mathcal{P}_S$ for each $j \in [k]$ locally adds $(\text{cid}_{i_j}, (m_j, f_{i_j}))$ to mp_{com} , while $\mathcal{P}_R$ upon receiving the broadcast message adds $(\text{cid}_{i_j}, (d_{i_j}, f_{i_j}))$ to mp_{com} .
3. Both parties remove each cid_{i_j} from their list li_{gen} .
4. Both parties send (Query, sid, $\overline{\text{ch}} \parallel \text{mp}_{\text{com}}, \mathcal{T}$) to $\mathcal{F}_{\text{GPRO}}$ and obtain ch . Let $L = (\text{cid}_j \mid (\text{cid}_j, *) \in \text{mp}_{\text{com}})$. Then they add $(\overline{\text{ch}} \parallel L, \text{ch})$ to li_{chal} and output ch .

Fig. 10: Protocol $\Pi_{\text{HOTCommit}}^{1-1}$ implementing $\mathcal{F}_{\text{HOTCom}}$ for 1 receiver.

$\mathcal{P}_S$ announces a correction value $d = x - \mathbf{U}_1[i, j]$ to be broadcast. d allows $\mathcal{P}_R$ to later correct its share of the level-1 VOLE commitment (i, j) to the secret x using the additive homomorphism of VOLE commitments.

Hinting and opening. To *hint* towards the receiver, the sender will simply reveal the committed value of the level-1 commitment but not the corresponding MAC in $\mathbf{U}_2$ (or the linear combination applied to these MACs). To *open* towards the receiver, $\mathcal{P}_S$ first reveals all the MACs of the opened level-1 commitments to the receiver, which we denote for simplicity here as A . The sender then uses the random oracle on input A and the transcript to generate the big-field VOLE challenge Δ_1 . It can compute all the level-1 VOLE commitment keys, namely the matrix $\mathbf{S} = \mathbf{U}_1 \Delta_1 + \mathbf{U}_2$ and give them to the receiver. Note that in a regular definition of VOLE, $\mathbf{S}$ is generated by the VOLE protocol and hidden from $\mathcal{P}_S$.

$\Pi_{\text{HOTCommit}}^{1-1}$ (Part 2)

Hint On input $(\text{Hint}, \text{sid}, (\text{cid}_j)_{j \in [k]}, i)$ to $\mathcal{P}_S$, if $(\text{cid}_j, (m_j, f_j)) \in \text{mp}_{\text{com}}$ for every $j \in [k]$ and if **Open** has not run yet, $\mathcal{P}_S$ does the following:

If $i = 0$ broadcast $(\text{cid}_j, m_j)_{j \in [k]}$ and add $(\text{cid}_j, (m_j, f_j, \text{pub}))$ to mp_{open} .

If $i = 1$

1. Sample $r_j \in \{0, 1\}^\lambda$ and compute $g_j := f_j^{(0)} + f_j^{(1)}(\mathbf{u}_1, \dots, \mathbf{u}_\ell)$.
2. For $j \in [k]$ send $(\text{Query}, \text{sid}, (m_j \| g_j \| r_j), \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ to obtain c_j .
3. Broadcast $(\text{cid}_j, c_j)_{j \in [k]}$ and send $(\text{cid}_j, m_j)_{j \in [k]}$ to $\mathcal{P}_R$.
4. Add $(\text{cid}_j, (m_j, g_j, r_j, c_j, f_j, \text{priv}))$ to mp_{open} .

$\mathcal{P}_R$, upon receiving $(\text{cid}_j, m_j)_{j \in [k]}$ from $\mathcal{P}_S$ via broadcast, if $(\text{cid}_j, (d_j, f_j)) \in \text{mp}_{\text{com}}$ for every $j \in [k]$, if **Open** has not run yet and if no $(\text{cid}_j, *) \in \text{mp}_{\text{open}}$, add each $(\text{cid}_j, (m_j, d_j, f_j, \text{pub}))$ to mp_{open} .

$\mathcal{P}_R$, upon receiving $(\text{cid}_j, m_j)_{j \in [k]}$ from $\mathcal{P}_S$ and the broadcast $(\text{cid}_j, c_j)_{j \in [k]}$, if $(\text{cid}_j, (d_j, f_j)) \in \text{mp}_{\text{com}}$ for every $j \in [k]$, if **Open** has not run yet and if no $(\text{cid}_j, *) \in \text{mp}_{\text{open}}$, add $(\text{cid}_j, (m_j, c_j, d_j, f_j, \text{priv}))$ to mp_{open} for $j \in [k]$.

Delayed Hint On input $(\text{DelHint}, \text{sid}, (\text{cid}_j)_{j \in [k]})$ to $\mathcal{P}_S$, if $(\text{cid}_j, (m_j, f_j)) \in \text{mp}_{\text{com}}$ for every $j \in [k]$ and if **Open** has not run yet, $\mathcal{P}_S$ does the following:

1. Sample $r_j \in \{0, 1\}^\lambda$ and compute $g_j := f_j^{(0)} + f_j^{(1)}(\mathbf{u}_1, \dots, \mathbf{u}_\ell)$.
2. Send $(\text{Query}, \text{sid}, m_j \| g_j \| r_j, \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ and obtain the response c_j for each $j \in [k]$. Then broadcast $(\text{cid}_j, c_j)_{j \in [k]}$.
3. Add $(\text{cid}_j, (m_j, g_j, r_j, c_j, f_j, \text{delay}))$ to mp_{open} .

$\mathcal{P}_R$, upon receiving the broadcast $(\text{cid}_j, c_j)_{j \in [k]}$, if $(\text{cid}_j, (d_j, f_j)) \in \text{mp}_{\text{com}}$ for every $j \in [k]$, if **Open** has not run yet and if no $(\text{cid}_j, *) \in \text{mp}_{\text{open}}$, adds $(\text{cid}_j, (c_j, d_j, f_j, \text{delay}))$ to mp_{open} for each $j \in [k]$.

Add On input $(\text{Add}, \text{sid}, (\text{cid}_j)_{j \in [3]})$ to $\mathcal{P}_S$, if no call to **Open** was made yet, check if $(\text{cid}_3, *) \notin \text{mp}_{\text{com}}$, $\text{cid}_3 \notin \text{li}_{\text{gen}}$ and $\{(\text{cid}_1, (A, f_1)), (\text{cid}_2, (B, f_2))\} \subseteq \text{mp}_{\text{com}}$. If so:

- Each party adds $(\text{cid}_3, (A + B, f_1 + f_2))$ to mp_{com} .

Linear On input $(\text{Linear}, \text{sid}, \text{cid}_1, \text{cid}_2, \alpha, \beta)$ to $\mathcal{P}_S$, if **Open** was not called yet, check if $(\text{cid}_2, *) \notin \text{mp}_{\text{com}}$, $\text{cid}_2 \notin \text{li}_{\text{gen}}$ and $(\text{cid}_1, (A, f)) \in \text{mp}_{\text{com}}$. If so:

- Set $g = \alpha f + \beta$
- Each party adds $(\text{cid}_2, (\alpha A + \beta, g))$ to mp_{com} .

Fig. 11: Protocol $\Pi_{\text{HOTCommit}}^{1-1}$ implementing $\mathcal{F}_{\text{HOTCom}}$ for 1 receiver

The receiver, upon receiving A and $\mathbf{S}$, first recomputes Δ_1 . With this information and the other messages from the sender, it can apply all the differences d to the matrix $\mathbf{S}$, thus correcting all level-1 VOLE commitment keys of random messages to the actual messages. Then it applies all linear operations f_j that were supposed to lead to the messages m_j revealed earlier by the sender in *hint*. $\mathcal{P}_R$ checks that the resulting keys are consistent with the opened MACs in A .

At this point, the commitment protocol is not secure: the sender could have lied when generating $\mathbf{S}$, which it generated *after* Δ_1 is chosen, and the receiver does

Open On first input (Open, sid) $\mathcal{P}_S$ does the following:

1. Compute $g_j := f_j^{(0)} + f_j^{(1)}(\mathbf{u}_1, \dots, \mathbf{u}_\ell)$ and define

$$A_{\text{pub}} := \{(\text{cid}_j, m_j, g_j, f_j) \mid (\text{cid}_j, (m_j, f_j)) \in \text{mp}_{\text{open}}\}$$

$$A_{\text{priv}} := \{(\text{cid}_j, c_j, f_j) \mid (\text{cid}_j, (m_j, g_j, r_j, c_j, f_j, \text{priv})) \in \text{mp}_{\text{open}}\}$$

$$A_{\text{delay}} := \{(\text{cid}_j, c_j, f_j) \mid (\text{cid}_j, (m_j, g_j, r_j, c_j, f_j, \text{delay})) \in \text{mp}_{\text{open}}\}$$

$$B_{\text{priv}} := \{(\text{cid}_j, m_j, g_j, r_j) \mid (\text{cid}_j, (m_j, g_j, r_j, c_j, f_j, \text{priv})) \in \text{mp}_{\text{open}}\}$$

2. $\mathcal{P}_S$ sends (Query, sid, $\overline{\text{ch}} \| A_{\text{pub}} \| A_{\text{priv}} \| A_{\text{delay}}, p$) to $\mathcal{F}_{\text{GPRO}}$ and obtains Δ_1 .
3. $\mathcal{P}_S$ computes $\mathbf{S} := \mathbf{U}_2 + \mathbf{U}_1 \Delta_1$ and $\mathbf{D} := \mathbf{V}_2 + \mathbf{V}_1 \Delta_1$.
4. $\mathcal{P}_S$ broadcasts $(\mathbf{S}, \mathbf{D}, A_{\text{pub}}, A_{\text{delay}})$ and sends B_{priv} to $\mathcal{P}_R$.
5. Let m be the concatenation of all broadcast messages in order. Then send (InitDone, sid, m) and (Deliver, sid) to $\mathcal{F}_{\text{WVOLE}}$.

Upon receiving B_{priv} from $\mathcal{P}_S$, $\mathcal{P}_R$ does the following:

1. Obtain (Done, sid, m, Δ_0) from $\mathcal{F}_{\text{WVOLE}}$.
2. Check that $\mathbf{S}, \mathbf{D}, A_{\text{pub}}, A_{\text{delay}}$ as contained in m was broadcast on $\mathcal{F}_{\text{GBB}}$. If not, then abort.
3. For each $(\text{cid}_j, m_j, g_j, f_j) \in A_{\text{pub}}$ if $(\text{cid}_j, (m_j, d_j, f_j, \text{pub})) \in \text{li}_{\text{open}}$ for every $j \in [k]$ replace each $(\text{cid}_j, (m_j, d_j, f_j, \text{pub}))$ in mp_{open} with $(\text{cid}_j, (m_j, g_j, d_j, f_j, \text{pub}))$.
4. For each $(\text{cid}_j, m_j, g_j, r_j) \in B_{\text{priv}}$, if $(\text{cid}_j, (m_j, c_j, d_j, f_j, \text{priv})) \in \text{mp}_{\text{open}}$ for every $j \in [k]$ send (Query, sid, $m_j \| g_j \| r_j, \mathcal{T}$) to $\mathcal{F}_{\text{GPRO}}$ and check if the response is c_j . If so, then replace each $(\text{cid}_j, (m_j, c_j, d_j, f_j, \text{priv}))$ in mp_{open} with $(\text{cid}_j, (m_j, g_j, c_j, d_j, f_j, \text{priv}))$. Otherwise abort.
5. Obtain (Corr, sid, $\mathbf{Q}$) from $\mathcal{F}_{\text{WVOLE}}$ where $\mathbf{Q} \in \mathbb{F}_p^{2\ell \times n_c}$. Set $\mathbf{Q}_1 := \mathbf{Q}_{[1..\ell]}$ and $\mathbf{Q}_2 := \mathbf{Q}_{[\ell+1..2\ell]}$.

Delayed Open On second input (Open, sid) $\mathcal{P}_S$ computes and broadcasts

$$A_{\text{delay, open}} := \{(\text{cid}_j, m_j, g_j, r_j, f_j) \mid (\text{cid}_j, (m_j, g_j, r_j, c_j, f_j, \text{delay})) \in \text{mp}_{\text{open}}\}$$

Upon observing $A_{\text{delay, open}}$ on $\mathcal{F}_{\text{GBB}}$ broadcast by $\mathcal{P}_S$, $\mathcal{P}_R$ does the following:

1. For each $(\text{cid}_j, m_j, g_j, r_j, f_j) \in A_{\text{delay, open}}$, if $(\text{cid}_j, (c_j, d_j, f_j, \text{delay})) \in \text{mp}_{\text{open}}$ for every $j \in [k]$ and $(\text{cid}_j, c_j, f_j) \in A_{\text{delay}}$ send (Query, sid, $m_j \| g_j \| r_j, \mathcal{T}$) to $\mathcal{F}_{\text{GPRO}}$ and check if the response is c_j .
2. If c_j is correct then replace each $(\text{cid}_j, (c_j, d_j, f_j, \text{delay}))$ in mp_{open} with $(\text{cid}_j, (m_j, g_j, r_j, d_j, f_j, \text{delay}))$. Otherwise abort.

Challenges Upon input (Challenges, sid) to any party, send (Challenge, sid) to $\mathcal{F}_{\text{WVOLE}}$. If it returns $\perp$ return $\perp$, otherwise obtain $\overline{\text{ch}}$. Then go through every **Commit** broadcast on $\mathcal{F}_{\text{GBB}}$, beginning with the first posted, and compute li_{chal} as $\mathcal{P}_R$ does in **Commit** using $\overline{\text{ch}}$. Then output li_{chal} .

Fig. 12: Protocol $\Pi_{\text{HOTCommit}}^{1-1}$ implementing $\mathcal{F}_{\text{HOTCom}}$ for 1 receiver

Verify On input $(\text{Verify}, \text{sid}, (\text{cid}'_j, m'_j)_{j \in [k]})$ to $\mathcal{P}_R$ where all $(\text{cid}'_j, \dots) \in \text{mp}_{\text{open}}$, if **Open** was finished and $\mathcal{P}_R$ did not abort it does the following:

1. Compute

$$\begin{aligned} A_{\text{pub}} &:= \{(\text{cid}_j, m_j, g_j, f_j) \mid (\text{cid}_j, (m_j, g_j, d_j, f_j, \text{pub})) \in \text{mp}_{\text{open}}\} \\ A_{\text{priv}} &:= \{(\text{cid}_j, c_j, f_j) \mid (\text{cid}_j, (m_j, g_j, c_j, d_j, f_j, \text{priv})) \in \text{mp}_{\text{open}}\} \\ A_{\text{delay}} &:= \left\{ (\text{cid}_j, c_j, f_j) \mid \begin{array}{l} (\text{cid}_j, (m_j, g_j, r_j, d_j, f_j, \text{delay})) \in \text{mp}_{\text{open}} \text{ or} \\ (\text{cid}_j, (c_j, d_j, f_j, \text{delay})) \in \text{mp}_{\text{open}} \end{array} \right\} \end{aligned}$$

2. Send $(\text{Query}, \text{sid}, \overline{\text{ch}} \| A_{\text{pub}} \| A_{\text{priv}} \| A_{\text{delay}}, p)$ to $\mathcal{F}_{\text{GPRO}}$ and obtain Δ_1 .
3. Let $\Gamma \in \mathbb{F}_p^{\ell \times k_C}$ be such that

$$\Gamma[i, j] := \begin{cases} d_{(j-1)\ell+i-1} & \text{if } (\text{cid}'_j, (m'_j, g_j, d_j, f_j, \text{pub})) \in \text{mp}_{\text{open}} \\ & \text{or } (\text{cid}'_j, (m'_j, g_j, c_j, d_j, f_j, \text{priv})) \in \text{mp}_{\text{open}} \\ & \text{or } (\text{cid}'_j, (m'_j, g_j, r_j, d_j, f_j, \text{delay})) \in \text{mp}_{\text{open}} \\ 0 & \text{otherwise} \end{cases}$$

4. Let f_j be all the polynomials used to define Γ .
5. Set $\mathbf{S}' := \mathbf{S} + \Gamma \Delta_1$ and denote row i as $\mathbf{s}'_j$
6. For each $(\text{cid}'_j, (m'_j, g_j, d_j, f_j, \text{pub}))$, $(\text{cid}'_j, (m'_j, g_j, c_j, d_j, f_j, \text{priv}))$ and $(\text{cid}'_j, (m'_j, g_j, r_j, d_j, f_j, \text{delay}))$ in mp_{open} check that

$$g_j = f_j^{(0)} + f_j^{(1)}(\mathbf{s}'_1, \dots, \mathbf{s}'_\ell) - m_j \Delta_1$$

7. Check that $\mathbf{D} = \mathbf{Q}_2 + \mathbf{Q}_1 \Delta_1 - \mathcal{C}(\mathbf{S}) \cdot \text{diag}(\Delta_0)$

Audit On input $(\text{Audit}, \text{sid}, (\text{cid}'_j, f'_j)_{j \in [k]})$ $\mathcal{P}_A$ does the following:

1. If $\mathcal{P}_S$ or $\mathcal{P}_R$ broadcast $(\text{Abort}, \text{sid})$ then return $(\text{AuditFailed}, \text{sid})$.
2. Send $(\text{Audit}, \text{sid})$ to $\mathcal{F}_{\text{WVole}}$. If it returns $(\text{AuditFailed}, \text{sid})$ return $(\text{AuditFailed}, \text{sid})$, otherwise store $\ell, \overline{\text{ch}}, m, \Delta_0, \mathbf{Q}$.
3. If m is not identical with the messages broadcast by $\mathcal{P}_S$ in the respective order then return $(\text{AuditFailed}, \text{sid})$.
4. Set $A_{\text{priv}} := \emptyset$ and recompute li_{gen} like $\mathcal{P}_R$ in **Init**.
5. For each **Commit**, **Hint**, **Delayed Hint**, **Add**, **Linear** perform the actions that $\mathcal{P}_R$ performs.
6. For each **Hint** with $i = 1$ and a broadcast message $(\text{cid}_j, c_j)_{j \in [k]}$ if $(\text{cid}_j, (d_j, f_j)) \in \text{mp}_{\text{com}}$ and if no $(\text{cid}_j, \dots) \in \text{li}_{\text{open}}$ add all (cid_j, c_j, f_j) to A_{priv} .
7. Run steps 2 and 3 of $\mathcal{P}_R$ in **Open** and all steps in **Delayed Open**.
8. Recompute $A_{\text{pub}}, A_{\text{delay}}$ like $\mathcal{P}_R$ does during **Verify**. If $(\text{cid}'_j, m'_j, \dots, f'_j) \notin A_{\text{pub}} \cup A_{\text{delay}}$ for some cid'_j return $(\text{AuditFailed}, \text{sid})$.
9. Run **Verify** with input $(\text{cid}'_j, m'_j)_{j \in [k]}$ and A_{priv} as computed above. Output $(\text{Audit}, \text{sid}, (\text{cid}'_j, m'_j)_{j \in [k]})$ if it accepts, otherwise output $(\text{AuditFailed}, \text{sid})$.

Fig. 13: Protocol $\Pi_{\text{HOTCommit}}^{1-1}$ implementing $\mathcal{F}_{\text{HOTCom}}$ for 1 receiver

not have any guarantee yet that $\mathbf{S}$ is well-formed. By changing $\mathbf{S}$ the sender could have forged the level-1 VOLE keys of the receiver, easily allowing it to pass the MAC check outlined above. We therefore now explain how the receiver uses $\mathbf{Q}$ and Δ_0 generated by $\mathcal{F}_{\text{WVOLE}}$ to verify that $\mathbf{S}$ was formed correctly.

Checking the VOLE key matrix $\mathbf{S}$. The receiver can check the correctness of $\mathbf{S}$ based on its share of the correlation that $\mathcal{F}_{\text{WVOLE}}$ outputs. We call the correlation formed by $\mathcal{F}_{\text{WVOLE}}$ the “level-0” VOLEs. Recall that the receiver obtains $\mathbf{Q} = \mathbf{U}\mathbf{G}\text{diag}(\Delta_0) + \mathbf{V}$, by definition of $\mathcal{F}_{\text{WVOLE}}$. This is a subspace-VOLE correlation, where all of $\mathbf{U}$ is now the committed secret: the secrets of the initial random level-1 VOLEs ($\mathbf{U}_1$) are identical to the first ℓ rows of the level-0 VOLE correlation. Moreover, the MACs ($\mathbf{U}_2$) of the level-1 VOLEs are identical to the lower half of the level-0 VOLE correlation. Thus, they both form the secret of the level-0 VOLE, and $\mathbf{S}$ is a publicly known linear transform of the rows of the secret of the level-0 VOLEs, i.e. of $\mathbf{U}$. Since this is a row-wise linear transform on $\mathbf{U}$, we can check correctness of $\mathbf{S}$ using the level-0 VOLE correlation.

Towards this, the sender additionally sends $\mathbf{D} = \mathbf{V}_2 + \mathbf{V}_1\Delta_1$ alongside $\mathbf{S}$. This applies the same linear transform which generated $\mathbf{S}$ from $\mathbf{U}$, but this time on the MACs of the level-0 VOLEs (i.e. on $\mathbf{V}$). Then, the receiver obtains $(\text{Done}, \text{sid}, m, \Delta_0)$ and $(\text{Corr}, \text{sid}, \mathbf{Q})$ from $\mathcal{F}_{\text{WVOLE}}$. Denoting the first ℓ rows of $\mathbf{Q}$ as $\mathbf{Q}_1$ and lower ℓ rows as $\mathbf{Q}_2$, the receiver can compute $\mathbf{Q}_2 + \mathbf{Q}_1\Delta_1$. If $\mathbf{S}, \mathbf{D}$ were formed correctly, then it must hold that

$$\underbrace{\mathbf{Q}_2 + \mathbf{Q}_1\Delta_1}_{\text{level-0 VOLE Key}} = \underbrace{\mathbf{V}_2 + \mathbf{V}_1\Delta_1}_{\text{level-0 VOLE MAC } \mathbf{D}} + \underbrace{\mathcal{C}(\mathbf{S})}_{\text{level-0 VOLE message}} \cdot \text{diag}(\Delta_0)$$

This equation is checked by the receiver, which broadcasts $(\text{Abort}, \text{sid})$ if the check fails. Security follows because $\mathbf{S}$ is encoded in the linear code $\mathcal{C}$ using the public generator matrix $\mathbf{G}$ during the check, while $\mathbf{Q}$ of $\mathcal{F}_{\text{WVOLE}}$ is also correlated using $\mathcal{C}$. To pass the check with an inconsistent input, one can show that $\mathcal{P}_{\text{S}}$ must essentially guess Δ_0 in at least $d_{\mathcal{C}}$ positions before it is chosen.

Complications due to delayed and private hints. While this description describes the general commitment protocol that we construct, it must also perform **Hint** with a private opening and **Delayed Hint** as specified in $\mathcal{F}_{\text{HOTCom}}$. While implementing this complicates the protocol description on a formal level, the ideas for achieving these properties are simple: For any **Hint** with a private opening the sender hashes the message m , together with some randomness r , into a commitment c using the random oracle $\mathcal{F}_{\text{CpRO}}$. c is broadcast, while m, r are sent to the receiver. Later, c is part of the challenge to derive Δ_1 (thus binding $\mathcal{P}_{\text{S}}$ to m) and, upon receiving the MAC in private, the receiver can check the MAC and validity of the opening of the commitment c . This also allows third parties to recompute Δ_1 as only c is used to derive it, which is crucial for auditing. **Delayed Hint** works similarly: there, the message m whose opening is delayed will be committed to in a commitment c that is broadcast. Then, when the delayed hint is opened during **Delayed Open**, the message and randomness (as well as

the MAC) are revealed. The receiver can again check if the commitment indeed contains m and that the MAC is valid. This time, however, m as well as the MAC will be broadcast, as they are not private outputs.

Audits and Challenges. To perform **Audit**, the auditor will collect all messages to be verified from the broadcast channel. It then uses $\mathcal{F}_{\text{WVOLE}}$'s **Audit** to obtain the same $\mathbf{Q}$ matrix of the level-0 VOLE correlation that $\mathcal{P}_R$ obtained (as well as Δ_0) and then basically performs the same operations and checks that $\mathcal{P}_R$ did during the protocol to apply linear homomorphisms and check openings. One crucial difference is that $\mathcal{P}_A$ does not have access to *private* openings done via **Private Hint**, but it has access to the commitments that contain the messages (which are broadcast). This is sufficient to recompute Δ_1 .

To ensure that Δ_1 is unpredictable to the adversary, the computation involves the use of the challenge $\overline{\mathbf{ch}} \in \{0,1\}^{2\lambda}$ generated by the hybrid functionality. This is only generated after the inputs of the sender to $\mathcal{F}_{\text{WVOLE}}$ are fixed. By construction, we ensure that the challenge Δ_0 of the level-0 VOLE correlation is only chosen after all other messages (and also Δ_1) are fixed. Figures 10, 11, 12 and 13 contain the full protocol.

Theorem 1. *Let λ be the security parameter, $\mathcal{C}$ be an $[n_C, k_C, d_C]$ -linear code over $\mathbb{F}_p$ and $N < p$. The protocol $\Pi_{\text{HOTCommit}}^{1-1}$ securely implements $\mathcal{F}_{\text{HOTCom}}$ against any PPT static adversary $\mathcal{A}$ corrupting at most both $\mathcal{P}_S, \mathcal{P}_R$ but never $\mathcal{P}_A$, in the $\mathcal{F}_{\text{WVOLE}}^{p,N,C}$ -hybrid model with access to global subroutines $\mathcal{F}_{\text{GBB}}, \mathcal{F}_{\text{GpRO}}$. $\mathcal{A}$ makes at most Q queries to the Random Oracle. The distinguishing advantage of $\mathcal{Z}$ is at most $Q/2^\lambda + Q/N^{d_C} + Q/p + Q^2/2^{2\lambda} + Q^2/p$, and we use $\mathcal{T} = 2^{2\lambda}$ as challenge space for $\mathcal{F}_{\text{WVOLE}}$.*

The proof can be found in Appendix E. The Q^2/p -term in the theorem is only an artifact of the simplified construction of the protocol: it comes from random oracle calls on inputs from $\mathbb{F}_p$ that have to generate a challenge. In practice, this can be resolved by always requiring 2-3 values to be committed simultaneously.

The proof, while lengthy, is relatively straightforward: we provide two simulation strategies, depending on the sender being corrupted or not. If the sender is honest, then messages and commitments sent during **Commit**, **Hint** or **Delayed Hint** are essentially random. During **Open**, we carefully choose the g_j to fit based on the simulated messages and outputs of the ideal functionality. If the sender is dishonest, then we use the extraction strategy from [7], which works because the challenges Δ_1, Δ_0 are unpredictable to the sender and because the transcript influences Δ_0 generated by $\mathcal{F}_{\text{WVOLE}}$ as well. We use **Challenge Test** of $\mathcal{F}_{\text{WVOLE}}$ to simulate random oracle responses for choices of $\mathbf{U}, \mathbf{V}$.

4.3 From single-receiver to multi-receiver one-time commitments

We now extend the previous protocol to the multi-receiver setting where $|\mathcal{P}_R| > 1$ using the protocol $\Pi_{\text{HOTCommit}}^{1-n}$. It is realized in the $\mathcal{F}_{\text{GBB}}, \mathcal{F}_{\text{GpRO}}, \mathcal{F}_{\text{HOTCom}}$ -hybrid

model and described in Figure 20 and Figure 21 in Appendix F. $\Pi_{\text{HOTCom}}^{1-n}$ runs an instance of $\mathcal{F}_{\text{HOTCom}}$ with every receiver, committing to the same value everywhere. To show consistency, it uses the linearly homomorphic properties of the commitment scheme.

In more detail, the sender $\mathcal{P}_S$ initializes one instance of $\mathcal{F}_{\text{HOTCom}}$ with each party $\mathcal{P}_i \in \mathcal{P}_R$, which we denote as $\mathcal{F}_{\text{HOTCom}}^i$. During **Commit**, **Hint**, **Delayed Hint**, **Open**, **Add**, **Linear** and **Verify** $\mathcal{P}_S$ will send the same message to each $\mathcal{F}_{\text{HOTCom}}^i$. For **Hint** with a private receiver the sender only has to send the message to $\mathcal{F}_{\text{HOTCom}}^i$ with receiver $\mathcal{P}_i$. Every message in **Delayed Hint** will be revealed during **Delayed Open**.

During **Commit** the sender could commit to different messages between two or more instances of $\mathcal{F}_{\text{HOTCom}}^i$. This is easy to catch for all messages that are opened through **Hint** or **Delayed Open** as $\mathcal{P}_S$ will send the correct value to $\mathcal{F}_{\text{GBB}}$ before revealing it at each $\mathcal{F}_{\text{HOTCom}}^i$. Each $\mathcal{P}_i$ can then compare the value they received from $\mathcal{F}_{\text{HOTCom}}^i$ with the value that was sent to $\mathcal{F}_{\text{GBB}}$. To further ensure that the values that are sent via **Private Hint** are consistent, the parties perform a **Consistency Check** procedure during **Open**: there, they sample a universal hash function which is applied to all committed values. Afterwards, they open the outcome of the universal hash. If any difference is observed, then the commitments are inconsistent.

To prevent leakage of secrets, **Consistency Check** uses an additional value r to which the sender committed herself to during **Init**. In order to sample a universal hash function, the parties use the challenge value from the most recent **Commit** to derive as many challenge values as there are commitment values. Later, any auditor $\mathcal{P}_A$ can verify the correctness of any commitment opening by inspecting $\mathcal{F}_{\text{GBB}}$ and verifying outputs with each $\mathcal{F}_{\text{HOTCom}}^i$. To implement **Challenges**, note that each $\mathcal{F}_{\text{HOTCom}}^i$ outputs a challenge upon commitment to the sender and receiver, but makes the value also accessible to any other party. Every party $\mathcal{P}_i \in \mathcal{P}_R$ obtains the challenges of all functionalities and hashes these together using $\mathcal{F}_{\text{GpRO}}$. In Appendix F we prove the following theorem:

Theorem 2. *In the $(\mathcal{F}_{\text{GBB}}, \mathcal{F}_{\text{GpRO}}, \mathcal{F}_{\text{HOTCom}}^{1-1})$ -hybrid model the protocol $\Pi_{\text{HOTCom}}^{1-n}$ implements $\mathcal{F}_{\text{HOTCom}}$ over the field $\mathbb{F}_p$ with statistical security against any static active adversary corrupting at most all of $\mathcal{P}_S, \mathcal{P}_R$ but never $\mathcal{P}_A$. The adversary makes at most Q queries to $\mathcal{F}_{\text{GpRO}}$, and the distinguishing advantage at most $Q/p + Q/2^\lambda + Q^2/2^{2\lambda}$. The challenge space is $\mathcal{T} = \{0, 1\}^{2\lambda}$.*

The proof strategy again distinguishes between an honest and dishonest sender. If the sender is honest, then we simply program the $\mathcal{F}_{\text{HOTCom}}^i$ functionalities to make the correct outputs and use the hiding property of the universal hash to simulate the consistency check. If the sender is corrupted, then the adversary can cheat by either committing to different values to each party or by sending hints that differ from the commitments. While the latter is easy to catch, the former is caught through the consistency check which uses that the challenge for

the check is unpredictable by using a challenge value generated by $\mathcal{F}_{\text{HOTCom}}$. To make this consistent, we emulate random oracle responses for challenge queries by outputting what $\mathcal{F}_{\text{HOTCom}}$ outputs. This is guaranteed to work as the simulator controls challenge generation of the $\mathcal{F}_{\text{HOTCom}}^i$ instances, whose challenges are unpredictable to the attacker before actually committing to a value (thus allowing to program). This simulation also succeeds if all receivers are corrupt.

5 Offline Phase

During the online phase of the MPC protocol, pre-generated random values and triples are used to evaluate the circuit. Therefore, we now introduce the functionality $\mathcal{F}_{\text{Offline}}$ in Figure 14 that generates these with UC security. Note that the triples generated by the functionality might not be well-formed. However, this will be corrected for during the online phase.

We now give a short description of $\mathcal{F}_{\text{Offline}}$. Upon initialization in **Initialize**, the parties have to decide how many sharings for random values and triples can be generated later, which is correlated to the number of input and multiplication gates of the circuit $\mathcal{C}$ that is evaluated by a protocol using $\mathcal{F}_{\text{Offline}}$. For each of these challenges, the offline functionality will reserve certain IDs which refer to them internally. These IDs can be used by the parties to *reliably* open the sharings later. These are either triple IDs (`tid`) or IDs for input sharings (`rid`). During **Input** and **Triples** random additive sharings will be generated among all parties. The adversary is allowed to pick his shares and make triples incorrect using additive terms. Moreover, sharings that exist can have linear transforms applied to them using commands in the functionality. However, the ID of the result must always differ from IDs reserved for inputs and triples for technical reasons. The functionality stores the shares internally in the map `mpval`.

Finally, parties are allowed to open their sharings either privately towards a party (`type = 0`) or in public (`type = 1`) using **Open**. In either case, an attacker is allowed to add an additive error to the revealed value, but this cheating is noted by the functionality. Parties can also decide to delay an opening to a later stage (`type = 2`), and release such a delayed sharing later on. This is similar to the **Open** and **Delayed Open** of $\mathcal{F}_{\text{HOTCom}}$, but where now all parties can be senders. Opened values are stored internally in the list `liopen`, together with attributes if they are private or delayed openings.

At this point, the opened values might be wrong and cannot yet be verified. **Release** upon first call will make all opened values verifiable. Thereafter, a call to **Full Release** makes openings also verifiable to the auditor. Similar to $\mathcal{F}_{\text{HOTCom}}$ the functionality permits opening values in private, which cannot be verified by the auditor. The state of verifiability is stored in the flag `flver`.

To ease use with other protocols in the Fiat-Shamir setting, the functionality generates challenge values upon committing by the participants and an adversary can test which challenges its controlled parties' inputs would generate before

committing using **Triples**. We keep track of the number of generated challenges using $ccount$. If not all parties are corrupted, the attacker must choose a challenge consistent with the remaining state which we enforce using an extra min variable. To allow simulation of the protocol in the head of the attacker if all parties are corrupted, such an attacker can also *reset* the functionality at any point. This resets $ccount$ as well, allowing it to also set the challenge to a value related to previous states. The same challenge values can later be queried by the auditor. As before, we use mp_{chal} to keep track of all generated challenges and store the ultimately chosen challenge in the flag fl_{ch} .

5.1 Offline Protocol

The offline phase is implemented through $\Pi_{Offline}$ shown in Figure 16 Figure 17. The protocol generates authenticated random values and pairwise authenticated partial triples, in a style similar to [30]. Just as in the functionality, each secret in the protocol is additively shared. Each party $\mathcal{P}_i$ is a sender in an instance of $\mathcal{F}_{HOTCom}$ where it is committed to for its share and where the functionality has n receivers. The shares can be opened and verified via $\mathcal{F}_{HOTCom}$, and linear operations can be applied as well. Finally, the auditor $\mathcal{P}_A$ can verify outputs by using the **Audit** interfaces of all $\mathcal{F}_{HOTCom}$ instances to obtain the shares and comparing with its input.

However, one difficulty is that $\mathcal{F}_{Offline}$ must output potentially faulty triples (a^i, b^i, c^i) such that, if no party cheats, $\sum a^i \cdot \sum b^i = \sum c^i$. We achieve this by using an instance of $\mathcal{F}_{OLE}$ between each ordered pair of parties $\mathcal{P}_i, \mathcal{P}_j$. Here, we follow [30] and many earlier works that each c^i can be computed as

$$c^i = a^i \cdot b^i + \sum_{j < i} a^j \cdot b^i + \sum_{j < i} a^i \cdot b^j$$

While $a^i \cdot b^i$ can be computed by each $\mathcal{P}_i$ locally, it uses $\mathcal{F}_{OLE}$ together with every $\mathcal{P}_j$ to obtain additive shares of $a^j \cdot b^i$, $a^i \cdot b^j$ which it adds to $a^i \cdot b^i$ before committing to the sum. In Appendix G we prove the following statement:

Theorem 3. *In the $(\mathcal{F}_{GBB}, \mathcal{F}_{HOTCom}, \mathcal{F}_{OLE})$ -hybrid model, the protocol $\Pi_{Offline}$ securely implements $\mathcal{F}_{Offline}$ with security against any static active adversary corrupting at most all parties in $\mathcal{P}$ but never $\mathcal{P}_A$. If the adversary makes at most Q queries to $\mathcal{F}_{GpRO}$ then the distinguishing advantage is at most $Q^2/2^{2\lambda}$. The provided challenge space is $\mathcal{T} = \mathbb{F}_p^*$.*

To prove the statement, we provide different simulators depending on there being an honest party (except $\mathcal{P}_A$) or if all parties are corrupted. If an honest party exists, then the simulation strategy mainly follows [30] with small adjustments for the commitments. If all parties are corrupted, we alter the strategy to generate unpredictable challenges for the attacker based on $\mathcal{F}_{GpRO}$ queries. Here, we can emulate consistent $\mathcal{F}_{GpRO}$ outputs by using that its inputs, combined with the inputs to all $\mathcal{F}_{HOTCom}^i$, fix all inputs to the $\mathcal{F}_{Offline}$ functionality necessary to compute the challenge. We then use **reset** until the adversary finishes a transcript to obtain more challenges if needed for the simulation.

$\mathcal{F}_{\text{Offline}}$ (Part 1)

$\mathcal{F}_{\text{Offline}}$ interacts with a set of parties $\mathcal{P}$, an auditor $\mathcal{P}_A$ and an adversary $\mathcal{A}$. $\mathcal{A}$ may corrupt a subset of parties $\tilde{\mathcal{P}} \subseteq \mathcal{P}$ but never $\mathcal{P}_A$. Let $\mathbb{F}$ be a field. The adversary or any party may send $(\text{Abort}, \text{sid})$ at any point, whereupon $\mathcal{F}_{\text{Offline}}$ sends $(\text{Abort}, \text{sid})$ to all parties and aborts. The provided challenge space is $\mathcal{T} = \mathbb{F}_p^*$. Let $\text{fl}_{\text{ch}}, \text{fl}_{\text{ver}}$ be flags that are initially $\perp$, li_{open} be an initially empty list and $\text{mp}_{\text{val}}, \text{mp}_{\text{chal}}$ be empty maps. If all parties are corrupt and $\mathcal{A}$ sends **reset**, reset mp_{val} and fl_{ch} .

Initialize On first input $(\text{Init}, \text{sid}, \text{in}, \text{m})$ from all parties in $\mathcal{P}$:

1. Store in for the number of random values and m for the triples.
2. Set $\text{tid} = 1$, $\text{rid} = 1$, $\text{ccount} = 0$, $\text{fl}_{\text{ver}} = \perp$ and $\text{fl}_{\text{ch}} = \perp$.

Input On first input $(\text{Input}, \text{sid})$ from all parties in $\mathcal{P}$ and if $\text{fl}_{\text{ver}} = \perp$:

1. For each $l \in [\text{in}]$ and all honest parties $\mathcal{P}_i \in \mathcal{P} \setminus \tilde{\mathcal{P}}$ choose $(r_l^i)_{l \in [\text{in}]} \xleftarrow{\$} \mathbb{F}$.
2. Wait for $\mathcal{A}$ to send $(\text{rid}_l, r_l^i)_{l \in [\text{in}]}$ for every $\mathcal{P}_i \in \tilde{\mathcal{P}}$.
3. Set $r_l = \sum r_l^i$ and store $(\text{rid}_l, r_l)_{l \in [\text{in}]}$ in mp_{val} . Send $(\text{ShareInput}, \text{sid}, (\text{rid}, r^i))_{l \in [\text{in}]}$ to each $\mathcal{P}_i$.

Triples On first input $(\text{Triples}, \text{sid})$ from all parties in $\mathcal{P}$ and if $\text{fl}_{\text{ver}} = \perp$:

1. For all honest parties $\mathcal{P}_i \in \mathcal{P} \setminus \tilde{\mathcal{P}}$ sample $(a_l^i, b_l^i)_{l \in [m]} \xleftarrow{\$} \mathbb{F}$. Increase ccount and set $\text{min} = \text{ccount}$.
2. If $|\tilde{\mathcal{P}}| = 0$
 - (a) Set $a = \sum_{i \in \mathcal{P}} a_l^i$, $b = \sum_{i \in \mathcal{P}} b_l^i$ and sample $c_l^i \xleftarrow{\$} \mathbb{F}$ subject to the constraint that $c_l = \sum_{i \in \mathcal{P}} c_l^i$.
 - (b) Append $(\text{tid}_a, a), (\text{tid}_b, b), (\text{tid}_c, c)$ to mp_{val} and sample $\text{fl}_{\text{ch}} \xleftarrow{\$} \mathcal{T}$.
3. If $|\tilde{\mathcal{P}}| > 0$
 - (a) Wait for $\mathcal{A}$ to send $(\text{Values}, \text{sid}, ((a^i, b^i, c^i)_{i \in \tilde{\mathcal{P}}}, \Delta_c, (\delta_a^j, \delta_b^j)_{j \in \mathcal{P} \setminus \tilde{\mathcal{P}}}))$ and let **state** be a copy of mp_{val} .
 - (b) Set $a = \sum_{i \in \mathcal{P}} a^i$, $b = \sum_{i \in \mathcal{P}} b^i$ and $c = a \cdot b + \sum_{i \in \mathcal{P} \setminus \tilde{\mathcal{P}}} a^i \cdot \delta_a^i + b^i \cdot \delta_b^i + \Delta_c$.
 - (c) For all honest parties $\mathcal{P}_i \in \mathcal{P} \setminus \tilde{\mathcal{P}}$ sample $c^i \xleftarrow{\$} \mathbb{F}$ subject to the constraint that $c = \sum_{i \in \mathcal{P}} c^i$.
 - (d) Append $(\text{tid}_a, a), (\text{tid}_b, b), (\text{tid}_c, c)$ to **state**.
 - (e) Sample **ch** $\xleftarrow{\$} \mathcal{T}$, append $(\text{state} \parallel \text{ccount}, \text{ch})$ to mp_{chal} and send $(\text{ch}, \text{ccount})$ to $\mathcal{A}$.
 - (f) If $\mathcal{A}$ sends **reject** then increase ccount and go to step a. If $\mathcal{A}$ sends (accept, t) and $t \geq \text{min}$ find $(\text{state}' \parallel t, \text{ch})$ in mp_{chal} , add $(\text{tid}_a, a), (\text{tid}_b, b), (\text{tid}_c, c)$ from **state'** to mp_{val} and set $\text{fl}_{\text{ch}} = \text{ch}$.
4. Send $(\text{ShareTriple}, \text{sid}, (\text{tid}, a^i, b^i, c^i), \text{fl}_{\text{ch}})$ to each party $\mathcal{P}_i \in \mathcal{P} \setminus \tilde{\mathcal{P}}$.

Fig. 14: Ideal functionality $\mathcal{F}_{\text{Offline}}$

6 Online Protocol

We present the protocol Π_{Online} in Figure 27 and Figure 28 for the online phase. The protocol uses $\mathcal{F}_{\text{Offline}}$ to share inputs, create multiplication triples, perform linear operations and reconstruct shared values. Public messages are sent through

$\mathcal{F}_{\text{Offline}}$ (Part 2)

- Linear** On input (Linear, sid, $\text{id}_1, \text{id}_2, \alpha, \delta$) from all parties, where $(\text{id}_1, v_1) \in \text{mp}_{\text{val}}$, $(\text{id}_2, *) \notin \text{mp}_{\text{val}}$ and $\text{id}_2 \neq \text{rid}_*, \text{tid}_*$, $\text{fl}_{\text{ver}} = \perp$ and $\alpha, \delta \in \mathbb{F}$: Set $v_2 = \alpha \cdot v_1 + \delta$ and store (id_2, v_2) in mp_{val} .
- On input (Linear, sid, $\text{id}_1, \text{id}_2, \text{id}_3$) from all parties, where $\{(\text{id}_1, v_1)(\text{id}_2, v_2)\} \subseteq \text{mp}_{\text{val}}$, $(\text{id}_3, *) \notin \text{mp}_{\text{val}}$ and $\text{id}_3 \neq \text{rid}_*, \text{tid}_*$ and $\text{fl}_{\text{ver}} = \perp$: Set $v_3 = v_1 + v_2$ and add $\text{id}_1, \text{id}_2, \text{id}_3$
- Open** On input (Open, sid, id, type, i) from all parties, if $\text{fl}_{\text{ver}} = \perp$, $(\text{id}, v) \in \text{mp}_{\text{val}}$, type $\in [0..2]$ and $i \in [0..n]$:
1. If type = 1 or type = 0 and $\mathcal{P}_i \in \tilde{\mathcal{P}}$ send (Open, sid, id, i, v) to $\mathcal{A}$.
 2. Wait for $\mathcal{A}$ to send $\Delta \in \mathbb{F}, \delta \in \{0, 1\}$. If $\Delta \neq 0$ or $\delta = 0$ set $c = 0$, otherwise set $c = 1$.
 3. Append (id, type, $v + \Delta, i, c$) to li_{open} .
 4. If type = 0 and $\mathcal{P}_i \notin \tilde{\mathcal{P}}$ send (Open, sid, $i, v + \Delta$) to $\mathcal{P}_i$.
 5. If type = 1 send (Open, sid, $i, v + \Delta$) to all parties.
 6. If type = 2 send (DOpen, sid) to all parties.
- Release** On input (Release, sid) from all parties and if $\text{fl}_{\text{ver}} = \perp$: Set $\text{fl}_{\text{ver}} = 1$ and send (Release, sid) to each party.
- Full Release** On input (Release, sid) from all parties and if $\text{fl}_{\text{ver}} = 1$:
1. For each $(\text{id}, 2, v, i) \in \text{li}_{\text{open}}$ send (Release, sid, id, v) to the adversary.
 2. Upon input (Ok, sid) from the adversary, replace each $(\text{id}, 2, v, i)$ on li_{open} with $(\text{id}, 1, v, i)$ and let L be the list of all (id, v) .
 3. Set $\text{fl}_{\text{ver}} = 2$ and send (Release, sid, L) to each party.
- Verify** On input (Verify, sid, L) from $\mathcal{P}_i$ where for each $\text{id} \in L$ there is a value $(\text{id}, *, v, *, c) \in \text{li}_{\text{open}}$ and if $\text{fl}_{\text{ver}} \neq \perp$:
1. If $(\text{id}, 1, *, *) \in \text{li}_{\text{open}}$ or $(\text{id}, 0, *, i) \in \text{li}_{\text{open}}$ for each $\text{id} \in L$ output accept, otherwise reject.
- Challenge** On input (Challenge, sid) by $\mathcal{P}_A$ return fl_{ch} .
- Audit** On input (Audit, sid, $(\text{id}_l, m_l, f_l, b_l)_{l \in [k]}$) from $\mathcal{P}_A$ where $\text{fl}_{\text{ver}} = 2$ and each f_l is a linear function in $\text{tid}_j, \text{rid}_j$:
1. If $(\text{id}_l, 1, m_l, *) \in \text{li}_{\text{open}}$ for all $l \in [k]$ then return accept, else reject.

Fig. 15: Ideal functionality $\mathcal{F}_{\text{Offline}}$

$\mathcal{F}_{\text{GGB}}$ and randomness is created using the challenge mechanism of $\mathcal{F}_{\text{Offline}}$. During the initialization triples and random values are created by $\mathcal{F}_{\text{Offline}}$, and these are additively shared among the parties. For each input gate of the circuit one random sharing is needed and for each multiplication gate two triples will be created. Since triples may not be correct when returned by $\mathcal{F}_{\text{Offline}}$, they are then verified using the **Triple Check** procedure. For each triple that is verified, one additional triple has to be consumed in the process.

To check correctness of a triple, we use the method due to [28]. During that procedure a random value τ is generated based on previous messages send to $\mathcal{F}_{\text{GGB}}$. To verify one triple x, y, z using another a, b, c , the parties compute and open $\rho = \tau \cdot x + a$ and $\sigma = y + b$. Using these, they compute and open

$$v = \tau \cdot z - c + \sigma \cdot a + \rho \cdot b - \sigma \cdot \rho$$

Π_{Offline} (Part 1)

The protocol runs among n parties $\mathcal{P}$. All parties have access to the global sub-routines $\mathcal{F}_{\text{GpRO}}$ and $\mathcal{F}_{\text{GBB}}$. Each $\mathcal{P}_i$ initializes an instance of $\mathcal{F}_{\text{HOTCom}}$, such that it is the sender while every other party is in the set of receivers. We denote this instance as $\mathcal{F}_{\text{HOTCom}}^i$, and each has challenge space $\{0, 1\}^{2^\lambda}$. Additionally each ordered pair of parties has one instance of $\mathcal{F}_{\text{OLE}}$ running between them. When a party does not send an expected message then broadcast (**Broadcast**, *Abort*) and stop the protocol. Each $\mathcal{P}_i$ sets $\text{fl}_{\text{coms}} = 0$ and $c = \perp$.

Initialize On input $(\text{Init}, \text{sid}, \text{in}, m)$ every party $\mathcal{P}_i$ sends $(\text{Init}, \text{sid}, \text{in} + 3m)$ to all instances of $\mathcal{F}_{\text{HOTCom}}$.

Input On input $(\text{Input}, \text{sid})$ and if **Initialize** has been executed and $\text{fl}_{\text{coms}} = 0$, $\mathcal{P}_i$ does the following in times in parallel:

1. Sample $r^i \xleftarrow{\$} \mathbb{F}$ and send $(\text{Commit}, \text{sid}, (\text{rid}, r^i))$ to $\mathcal{F}_{\text{HOTCom}}^i$.
2. If all commits are finished, increase fl_{coms} and output $(\text{rid}_l, r_l^i)_{l \in [n]}$.

Triples On input $(\text{Triples}, \text{sid})$, if **Initialize** has been executed and $\text{fl}_{\text{coms}} = 1$, $\mathcal{P}_i$ does the following m times in parallel:

1. For all $l \in [m]$ sample a^i, b^i uniformly at random and send $(\text{Commit}, \text{sid}, (\text{tid}_j, j^i)_{j \in \{a, b\}})$ to $\mathcal{F}_{\text{HOTCom}}^i$.
2. For each ordered set of parties $(\mathcal{P}_i, \mathcal{P}_j)$, $\mathcal{P}_i$ sends $(\text{Extend}, \text{sid}, a^i)$ and $\mathcal{P}_j$ sends $(\text{Extend}, \text{sid}, b^j)$ to $\mathcal{F}_{\text{OLE}}$. $\mathcal{P}_i$ receives $w^{i,j} = a^i \cdot b^j - v^{i,j}$ while $\mathcal{P}_j$ receives $v^{i,j}$.
3. Repeat the aforementioned step with inputs $(\text{Extend}, \text{sid}, a^j)$ from $\mathcal{P}_j$ and $(\text{Extend}, \text{sid}, b^i)$ from $\mathcal{P}_i$. $\mathcal{P}_j$ obtains $w^{i,j}$ while $\mathcal{P}_i$ gets $v^{i,j}$.
4. For each $l \in [m]$ party $\mathcal{P}_i$ computes $c^i = a^i \cdot b^i + \sum_{j \neq i} w^{i,j} + v^{i,j}$.
5. Each party $\mathcal{P}_i$ sends $(\text{Commit}, \text{sid}, (\text{tid}_c, c^i))$ to $\mathcal{F}_{\text{HOTCom}}^i$.
6. From the last commit response, each party $\mathcal{P}_i$ obtains the challenge value $\overline{\text{ch}}^j$ from each $\mathcal{F}_{\text{HOTCom}}^j$.
7. Send $(\text{Query}, \text{sid}, (\overline{\text{ch}}^1 || \dots || \overline{\text{ch}}^{|P|}), \mathbb{F}_p^*)$ to $\mathcal{F}_{\text{GpRO}}$ and obtain ch .
8. Output $((\text{tid}_l, a_l^i, b_l^i, c_l^i, l)_{l \in [m]}, \text{ch})$. Increase fl_{coms} .

Linear On input $(\text{Linear}, \text{sid}, \text{id}_1, \text{id}_2, \alpha, \beta)$ where $\alpha, \beta \in \mathbb{F}$ $\mathcal{P}_i$ sends $(\text{Linear}, \text{sid}, \text{id}_1, \text{id}_2, \alpha, \beta)$ to $\mathcal{F}_{\text{HOTCom}}^i$. On input $(\text{Linear}, \text{sid}, \text{id}_1, \text{id}_2, \text{id}_3)$, $\mathcal{P}_i$ sends $(\text{Add}, \text{sid}, \text{id}_1, \text{id}_2, \text{id}_3)$ to $\mathcal{F}_{\text{HOTCom}}^i$.

Fig. 16: Protocol Π_{Offline} to implement the preprocessing phase.

which is expected to be 0. If the triples are not correct, then v can falsely open to 0 with probability $\frac{1}{|\mathbb{F}|-1}$ as explained in [28]. Once the triple check was successful, parties can share their inputs using **Input**. For any $\mathcal{P}_i$ to share a value x , they will require a random value r , which was created during **Init** and is privately opened to $\mathcal{P}_i$. $\mathcal{P}_i$ will then send $\delta = x + r$ to $\mathcal{F}_{\text{GBB}}$, allowing parties to publicly correct the sharing of r to x .

To evaluate the circuit $\mathcal{C}$ the parties use the **Linear** and **Multiply** commands. As the shares have linear properties, computing linear gates of $\mathcal{C}$ is straightforward. For multiplication we use multiplication triples as proposed by Beaver [11]: When x, y and a, b, c are additively shared and $c = a \cdot b$ then $z = \alpha \cdot b + \beta \cdot a - \alpha \cdot \beta + c$

Π_{Offline} (Part 2)

Open On input $(\text{Open}, \text{sid}, \text{id}, \text{type}, j)$ where id was committed to using $\mathcal{F}_{\text{HOTCom}}$, $\text{type} \in [0..2]$ and $j \in [0..n]$, $\mathcal{P}_i$ does the following:

- If $\text{type} \in \{0, 1\}$ send $(\text{Hint}, \text{sid}, \text{id}, j)$ to $\mathcal{F}_{\text{HOTCom}}^i$.
- If $\text{type} = 1$ or $\text{type} = 0$ and $i = j$ then obtain $(\text{Hint}, \text{sid}, \text{id}, v^j)$ from all $\mathcal{F}_{\text{HOTCom}}^j$, compute $v = \sum_{j=1}^n v^j$ where v^i is the value $\mathcal{P}_i$ committed to for id in $\mathcal{F}_{\text{HOTCom}}^i$ and output v .
- If $\text{type} = 2$ send $(\text{DelHint}, \text{sid}, \text{id})$ to $\mathcal{F}_{\text{HOTCom}}^i$ and wait for $(\text{DelHint}, \text{sid}, \text{id})$ from every $\mathcal{F}_{\text{HOTCom}}^j$.

Release On input $(\text{Release}, \text{sid})$ each $\mathcal{P}_i$ sends $(\text{Open}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}^i$. If any $\mathcal{F}_{\text{HOTCom}}^i$ outputs $(\text{Abort}, \text{sid})$ then abort.

Full Release On input $(\text{Release}, \text{sid})$ each party $\mathcal{P}_i$ does the following:

1. Send $(\text{Open}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}^i$. If any $\mathcal{F}_{\text{HOTCom}}^j$ aborts then abort.
2. Upon obtaining $(\text{Open}, \text{sid}, L^j)$ from each $\mathcal{F}_{\text{HOTCom}}^j$ where $L^j = \{(\text{id}_l, v_l^j)\}_{l \in [k]}$, set $v_l = \sum_{j=1}^n \text{id}_l^j$.
3. Output $\{(\text{id}_l, v_l)\}_{l \in [k]}$.

Verify On input $(\text{Verify}, \text{sid}, (\text{cid}_l)_{l \in [k]}, h)$, $\mathcal{P}_i$ does the following:

1. Let v_l^j be the shares of cid_l that $\mathcal{P}_i$ obtained previously. If any v_l^j is undefined then output reject.
2. Send $(\text{Verify}, \text{sid}, (\text{cid}_l, v_l^j)_{l \in [k]})$ to each $\mathcal{F}_{\text{HOTCom}}^j$.
3. If any $\mathcal{F}_{\text{HOTCom}}^j$ returns $(\text{VerifyFailed}, \text{sid})$ then output reject.

Audit On input $(\text{Audit}, \text{sid}, (\text{cid}_l, m_l, f_l)_{l \in [k]}, h)$, $\mathcal{P}_A$ does the following:

1. Send $(\text{Audit}, \text{sid}, (\text{cid}_l, f_l)_{l \in [k]})$ to each $\mathcal{F}_{\text{HOTCom}}^i$. If any $\mathcal{F}_{\text{HOTCom}}^i$ returns $(\text{AuditFailed}, \text{sid})$ output reject.
2. If all return $(\text{Audit}, \text{sid}, L^j)$ with $L^j = (\text{id}_l, v_l^j)$ then set $m'_l = \sum_{j=1}^n v_l^j$. If any $m_l \neq m'_l$ output reject, otherwise output accept.

Challenge On input $(\text{Challenge}, \text{sid})$ to $\mathcal{P}_A$ send $(\text{Challenges}, \text{sid})$ to every $\mathcal{F}_{\text{HOTCom}}^i$ instance. In the returned list take the most recent challenge that is associated with **Triples** for each $\mathcal{F}_{\text{HOTCom}}^i$ instance ch^i . Concatenate all ch^i values and query $\mathcal{F}_{\text{CpRO}}$, output what it outputs.

Fig. 17: Protocol Π_{Offline} to implement the preprocessing phase.

is equivalent to $z = x \cdot y$ when computing and opening $\alpha = x - a$ and $\beta = y - b$. For all linear computations, the correctness of openings can be verified using $\mathcal{F}_{\text{Offline}}$, hence our computation yields a correct y if no party cheated. Before outputting the result of the circuit, **Verify** is called to check if the adversary cheated in the computation or during openings (thus implying correctness of triples). After **Output** was successful and y is public, any auditor $\mathcal{P}_A$ can use **Audit** of $\mathcal{F}_{\text{Offline}}$ to verify the computation. The full protocol can be found in 27 and 28 in Appendix H, as well as the security proof.

Theorem 4. *In the $(\mathcal{F}_{\text{GBB}}, \mathcal{F}_{\text{Offline}})$ -hybrid model, the protocol Π_{Online} implements $\mathcal{F}_{\text{Online}}$ with statistical security against any static active adversary. Assuming the adversary sends reject or reset at most Q times to $\mathcal{F}_{\text{Offline}}$, the distinguishing advantage is at most $\frac{Q}{|\mathbb{F}| - 1}$.*

References

1. Adida, B.: Helios: Web-based open-audit voting. In: van Oorschot, P.C. (ed.) *USENIX Security 2008*. pp. 335–348. USENIX Association (Jul / Aug 2008), http://www.usenix.org/events/sec08/tech/full_papers/adida/adida.pdf
2. Ames, S., Hazay, C., Ishai, Y., Venkatasubramanian, M.: Ligerio: Lightweight sub-linear arguments without a trusted setup. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) *ACM CCS 2017*. pp. 2087–2104. ACM Press (Oct / Nov 2017). <https://doi.org/10.1145/3133956.3134104>
3. Badertscher, C., Canetti, R., Hesse, J., Tackmann, B., Zikas, V.: Universal composition with global subroutines: Capturing global setup within plain UC. In: Pass, R., Pietrzak, K. (eds.) *TCC 2020, Part III*. LNCS, vol. 12552, pp. 1–30. Springer, Cham (Nov 2020). https://doi.org/10.1007/978-3-030-64381-2_1
4. Baldimtsi, F., Kiayias, A., Zacharias, T., Zhang, B.: Crowd verifiable zero-knowledge and end-to-end verifiable multiparty computation. In: Moriai, S., Wang, H. (eds.) *ASIACRYPT 2020, Part III*. LNCS, vol. 12493, pp. 717–748. Springer, Cham (Dec 2020). https://doi.org/10.1007/978-3-030-64840-4_24
5. Baum, C., Beullens, W., Braun, L., Delpech de Saint Guilhem, C., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: Shorter, tighter, FAESTer: Optimizations and improved (QROM) analysis for VOLE-in-the-head signatures. In: Kalai, Y.T., Kamara, S.F. (eds.) *CRYPTO 2025, Part VI*. LNCS, vol. 16005, pp. 124–156. Springer, Cham (Aug 2025). https://doi.org/10.1007/978-3-032-01887-8_5
6. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In: Chung, K.M., Sasaki, Y. (eds.) *ASIACRYPT 2024, Part I*. LNCS, vol. 15484, pp. 463–493. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0875-1_15
7. Baum, C., Braun, L., Delpech de Saint Guilhem, C., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In: Handschuh, H., Lysyanskaya, A. (eds.) *CRYPTO 2023, Part V*. LNCS, vol. 14085, pp. 581–615. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38554-4_19
8. Baum, C., Damgård, I., Orlandi, C.: Publicly auditable secure multi-party computation. In: Abdalla, M., De Prisco, R. (eds.) *SCN 14*. LNCS, vol. 8642, pp. 175–196. Springer, Cham (Sep 2014). https://doi.org/10.1007/978-3-319-10879-7_11
9. Baum, C., Dittmer, S., Scholl, P., Wang, X.: Sok: vector OLE-based zero-knowledge protocols. *DCC* **91**(11), 3527–3561 (2023). <https://doi.org/10.1007/s10623-023-01292-8>
10. Baum, C., Malozemoff, A.J., Rosen, M.B., Scholl, P.: Mac’n’cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In: Malkin, T., Peikert, C. (eds.) *CRYPTO 2021, Part IV*. LNCS, vol. 12828, pp. 92–122. Springer, Cham, Virtual Event (Aug 2021). https://doi.org/10.1007/978-3-030-84259-8_4
11. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: Feigenbaum, J. (ed.) *CRYPTO’91*. LNCS, vol. 576, pp. 420–432. Springer, Berlin, Heidelberg (Aug 1992). https://doi.org/10.1007/3-540-46766-1_34
12. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable zero knowledge with no trusted setup. In: Boldyreva, A., Micciancio, D. (eds.) *CRYPTO 2019, Part III*. LNCS, vol. 11694, pp. 701–732. Springer, Cham (Aug 2019). https://doi.org/10.1007/978-3-030-26954-8_23

13. Bendlin, R., Damgård, I., Orlandi, C., Zakarias, S.: Semi-homomorphic encryption and multiparty computation. In: Paterson, K.G. (ed.) EUROCRYPT 2011. LNCS, vol. 6632, pp. 169–188. Springer, Berlin, Heidelberg (May 2011). https://doi.org/10.1007/978-3-642-20465-4_11
14. Bogetoft, P., Christensen, D.L., Damgård, I., Geisler, M., Jakobsen, T., Krøigaard, M., Nielsen, J.D., Nielsen, J.B., Nielsen, K., Pagter, J., Schwartzbach, M.I., Toft, T.: Secure multiparty computation goes live. In: Dingledine, R., Golle, P. (eds.) FC 2009. LNCS, vol. 5628, pp. 325–343. Springer, Berlin, Heidelberg (Feb 2009). https://doi.org/10.1007/978-3-642-03549-4_20
15. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudorandom correlation generators from ring-LPN. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 387–416. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_14
16. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: 42nd FOCS. pp. 136–145. IEEE Computer Society Press (Oct 2001). <https://doi.org/10.1109/SFCS.2001.959888>
17. Canetti, R., Chen, M.: Universally composable succinct vector commitments and applications. In: Theory of Cryptography Conference. pp. 642–674. Springer (2025)
18. Cascudo, I., Damgård, I., David, B., Döttling, N., Dowsley, R., Giacomelli, I.: Efficient UC commitment extension with homomorphism for free (and applications). In: Galbraith, S.D., Moriai, S. (eds.) ASIACRYPT 2019, Part II. LNCS, vol. 11922, pp. 606–635. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-34621-8_22
19. Cunningham, R.K., Fuller, B., Yakubov, S.: Catching MPC cheaters: Identification and openability. In: Shikata, J. (ed.) ICITS 17. LNCS, vol. 10681, pp. 110–134. Springer, Cham (Nov / Dec 2017). https://doi.org/10.1007/978-3-319-72089-0_7
20. Damgård, I., Keller, M., Larraia, E., Pastro, V., Scholl, P., Smart, N.P.: Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits. In: Crampton, J., Jajodia, S., Mayes, K. (eds.) ESORICS 2013. LNCS, vol. 8134, pp. 1–18. Springer, Berlin, Heidelberg (Sep 2013). https://doi.org/10.1007/978-3-642-40203-6_1
21. Damgård, I., Pastro, V., Smart, N.P., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 643–662. Springer, Berlin, Heidelberg (Aug 2012). https://doi.org/10.1007/978-3-642-32009-5_38
22. Frederiksen, T.K., Pinkas, B., Yanai, A.: Committed MPC - maliciously secure multiparty computation from homomorphic commitments. In: Abdalla, M., Dahab, R. (eds.) PKC 2018, Part I. LNCS, vol. 10769, pp. 587–619. Springer, Cham (Mar 2018). https://doi.org/10.1007/978-3-319-76578-5_20
23. Gilboa, N.: Two party RSA key generation. In: Wiener, M.J. (ed.) CRYPTO’99. LNCS, vol. 1666, pp. 116–129. Springer, Berlin, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_8
24. Keller, M., Orsini, E., Scholl, P.: MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) ACM CCS 2016. pp. 830–842. ACM Press (Oct 2016). <https://doi.org/10.1145/2976749.2978357>
25. Keller, M., Pastro, V., Rotaru, D.: Overdrive: Making SPDZ great again. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part III. LNCS, vol. 10822, pp. 158–189. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78372-7_6

26. Knott, B., Venkataraman, S., Hannun, A., Sengupta, S., Ibrahim, M., van der Maaten, L.: Crypten: Secure multi-party computation meets machine learning. *Advances in Neural Information Processing Systems* **34**, 4961–4973 (2021)
27. Küsters, R., Liedtke, J., Müller, J., Rausch, D., Vogt, A.: Ordinos: A verifiable tally-hiding E-voting system. In: 2020 IEEE European Symposium on Security and Privacy. pp. 216–235. IEEE Computer Society Press (Sep 2020). <https://doi.org/10.1109/EuroSP48549.2020.00022>
28. Lindell, Y., Nof, A.: A framework for constructing fast MPC over arithmetic circuits with malicious adversaries and an honest-majority. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) *ACM CCS 2017*. pp. 259–276. ACM Press (Oct / Nov 2017). <https://doi.org/10.1145/3133956.3133999>
29. Nair, D.G., Binu, V., Kumar, G.S.: An improved e-voting scheme using secret sharing based secure multi-party computation. *arXiv preprint arXiv:1502.07469* (2015)
30. Rachuri, R., Scholl, P.: Le mans: Dynamic and fluid MPC for dishonest majority. In: Dodis, Y., Shrimpton, T. (eds.) *CRYPTO 2022, Part I. LNCS*, vol. 13507, pp. 719–749. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15802-5_25
31. Rivinius, M., Reisert, P., Rausch, D., Küsters, R.: Publicly accountable robust multi-party computation. In: 2022 IEEE Symposium on Security and Privacy. pp. 2430–2449. IEEE Computer Society Press (May 2022). <https://doi.org/10.1109/SP46214.2022.9833608>
32. Schoenmakers, B., Veeningen, M.: Universally verifiable multiparty computation from threshold homomorphic cryptosystems. In: Malkin, T., Kolesnikov, V., Lewko, A.B., Polychronakis, M. (eds.) *ACNS 2015. LNCS*, vol. 9092, pp. 3–22. Springer, Cham (Jun 2015). https://doi.org/10.1007/978-3-319-28166-7_1
33. Shoup, V.: *A computational introduction to number theory and algebra*. Cambridge university press (2009)
34. Weng, C., Yang, K., Katz, J., Wang, X.: Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In: 2021 IEEE Symposium on Security and Privacy. pp. 1074–1091. IEEE Computer Society Press (May 2021). <https://doi.org/10.1109/SP40001.2021.00056>
35. Yang, K., Sarkar, P., Weng, C., Wang, X.: QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In: Vigna, G., Shi, E. (eds.) *ACM CCS 2021*. pp. 2986–3001. ACM Press (Nov 2021). <https://doi.org/10.1145/3460120.3484556>

A Open problems

While we give the first construction of plausibly post-quantum verifiable MPC from generic assumptions (the existence of OLE and a random oracle) and in fact the first MPC protocol that uses `VOLEitH` for MACs, there are multiple shortcomings which may lead to interesting future work.

No support for binary circuits. Our construction relies on an MPC scheme which is only secure for fields $\mathbb{F}$ of size exponential in the security parameter. While this is an improvement over [8], it would be interesting to extend our techniques to polynomial-size fields or even $\mathbb{F}_2$. Note that our one-time linear commitment

scheme can be adapted to this setting by using techniques from [7] and the dependence on exponential field size mainly stems from the [30] protocol.

Client-server setting with one message. One interesting property of [8] is that it allows a client-server MPC use, where clients can give input to the MPC protocol and must only send one message to the servers. This crucially relies on the clients being able to check commitment openings for correctness, which is not possible with our *hint* mechanism in just one round. It would therefore be interesting if our protocol can be adapted to the client-server setting with one-round client communication.

Identifiable abort. An interesting, related property to public verifiability is to let MPC parties deduce who cheated in a MPC protocol in case of abort. This is also called *identifiable abort*. In this work, we focus on public verifiability as the main security goal. We believe that our construction could be modified to also permit identifiable abort, e.g. by including an identifiable triple generation protocol. However, this is beyond the scope of this work.

B Extended Preliminaries

B.1 Universal Hash functions

A universal hash function family $\mathcal{H}$ is an efficiently samplable set of functions such that, upon choosing two different inputs x, y it holds that $h(x) = h(y)$ with bounded probability for a randomly chosen $h \in \mathcal{H}$.

Definition 1 (Universal Hash function). Let $\mathcal{H} = \{h : S \rightarrow T\}$, where S, T are finite sets, then $\mathcal{H}$ is a family of ϵ -universal hash functions if

$$\forall x, y \in S, x \neq y : \Pr_{h \in_R \mathcal{H}} [h(x) = h(y)] \leq \epsilon$$

The family of $1/p$ -universal hash functions used in the work is defined as:

Proposition 2. Let p be a prime and l be a positive integer. Then for every input $r = (r_1, \dots, r_{n+1}) \in \mathbb{F}_p^{l+1}$, the function

$$h_{\mathbf{s}} : \mathbb{F}_p^l \rightarrow \mathbb{F}_p, \quad (r_1, \dots, r_{l+1}) \mapsto r_{n+1} + \sum_{i=1}^l r_i \dot{s}_i$$

is a $1/p$ -universal hash function according to Definition 1.

Proof. See e.g. [33, Example 8.37].

In this work, we use universal hash functions that are hiding.

Definition 2 (Hiding). A matrix $\mathbf{H} \in \mathbb{F}_p^{r \times (n+h)}$ is *n-hiding* if the distribution of $\mathbf{H}\mathbf{v}$ is independent from $\mathbf{v}_{[1..n]}$ when $\mathbf{v}_{[n+1..n+h]}$ is chosen uniformly at random from $\mathbb{F}_p^h$. A hash family $\mathcal{H}$ is *n-hiding* if every $\mathbf{H} \in \mathcal{H}$ is *n-hiding*.

The construction in Proposition 2 fulfills this property for $r = h = 1$ if the value $\mathbf{v}_{n+1}$ is chosen uniformly at random.

B.2 The UC Framework

We use a version of the universal composability (UC) framework [16] that respects global subroutines [3] for analyzing security of our protocols.

In UC, protocols are run by interactive Turing Machines (iTM) called *parties* which we denote with a $\mathcal{P}$. The *adversary* $\mathcal{A}$, which is also an iTM, can actively corrupt any of these parties and then gains full control over it, seeing messages that it obtains and choosing any messages that the corrupted party or parties send. In this work we only consider static corruptions, meaning that the choice of corruption is fixed from the beginning of the protocol.

The parties can communicate with each other directly and with so-called *ideal functionalities* (which themselves are iTMs). These ideal functionalities are denoted by $\mathcal{F}$ and abstractly describe cryptographic protocols with certain guarantees. As usual, we define security with respect to an iTM $\mathcal{Z}$ called the *environment*. The environment provides inputs to the parties, receives outputs from them and controls $\mathcal{A}$.

To define security, let $\pi^{\mathcal{F}_1, \dots} \circ \mathcal{A}$ be the distribution of the output of an arbitrary $\mathcal{Z}$ when interacting with $\mathcal{A}$ in a real protocol instance π using ideal functionalities $\mathcal{F}_1, \dots$. Furthermore, let Sim denote an *ideal world adversary* and $\mathcal{F} \circ \text{Sim}$ be the distribution of the output of $\mathcal{Z}$ when interacting with parties which run with $\mathcal{F}$ instead of π and where Sim takes care of adversarial behavior.

Definition 3. We say that π UC-securely implements $\mathcal{F}$ if for every iTM $\mathcal{A}$ there exists an iTM Sim (with black-box access to $\mathcal{A}$) such that for every environment $\mathcal{Z}$, the outputs of $\mathcal{Z} \circ \pi^{\mathcal{F}_1, \dots} \circ \mathcal{A}$ and $\mathcal{Z} \circ \mathcal{F} \circ \text{Sim}$ are identical except with negligible probability.

UC with global subroutines. While UC allows to analyze security of protocols under arbitrary composition, it falls short of modeling global routines that maintain a shared state for multiple sessions - such as random oracles or bulletin boards. Therefore, we analyze our work in the *Universal Composition with Global Subroutines* (UCGS) model, proposed in [3].

Let π be a protocol that makes use of a global subroutine $\mathcal{G}$ (or more such subroutines). UCGS describes how to transform $\pi, \mathcal{G}$ into a standard UC-protocol $M[\pi, \mathcal{G}]$. This gives $\pi, \mathcal{F}$ and $\mathcal{Z}$ access to $\mathcal{G}$ such that it UC-emulates $M[\mathcal{F}, \mathcal{G}]$. The global subroutine $\mathcal{G}$ has to fulfill the *regularity* condition: a subroutine is

regular if only previously registered machines can speak with it and if it does not activate other machines outside of π . Moreover, the subroutine is not allowed to spawn any new instances of the protocol calling it.

Definition 4. We say that π implements $\mathcal{F}$ in the presence of a global subroutine $\mathcal{G}$ if for any $\mathcal{A}$ there exists an *i*TM $\mathcal{S}$ such that for every environment $\mathcal{Z}$ the outputs of an execution $EXEC_{\pi, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}}$ are identical to an execution $EXEC_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}}$ except with negligible probability.

More details, as well as a composition theorem for UCGS and further examples, can be found in [3].

C Communication Complexity

We estimate the communication complexity and divide the communication cost in offline and online phase. Communication is measured on a per-sender basis. As sending messages to $\mathcal{F}_{\text{GBB}}$ is different from sending peer-to-peer messages we classify communication cost according to the used channel. We further assume, that all n parties send their messages simultaneously and (if possible) each party will bundle their messages to one message instead of sending separate messages in consecutive rounds. We leave out *compressible protocol management information* that we currently send along through $\mathcal{F}_{\text{GBB}}$ - this includes terms like *chall*, commitment IDs and indices of functions on commitments such as f_{id} . As before, we assume that the circuit has in input gates and m multiplication gates.

Broadcast Offline Phase. Each pair of parties has to use **Init** in $\mathcal{F}_{\text{WVole}}$ both as sender and receiver where a total of $\text{mc} = \text{in} + 6\text{m} + 1$ commitments must be supported. Towards instantiating $\mathcal{F}_{\text{HOTCom}}$ we assume that $k_{\mathcal{C}} = n_{\mathcal{C}} = \mathcal{O}(\lambda)$ and we set $N = \mathcal{O}(\lambda)$ asymptotically as well. During initialization, Π_{IndWVole} broadcasts $(n-1) \cdot 2\lambda$ bits (for H for each receiver) while $\Pi_{\text{CodeWVole}}$ sends

$$(n-1) \log(p) \cdot ((2\ell+1)(n_{\mathcal{C}} - k_{\mathcal{C}}) + n_{\mathcal{C}} + k_{\mathcal{C}})$$

to communicate $\mathbf{C}, \tilde{\mathbf{U}}, \tilde{\mathbf{V}}$ where $\ell = \lceil \text{mc}/k_{\mathcal{C}} \rceil$. This is achieved in 3 broadcast rounds. During the offline protocol all input values as well as the a, b values of the Beaver triples can be generated and committed to simultaneously. Together with the initial broadcast of the circuit $\mathcal{C}$ and the finalization of the triple generation our offline phase can be instantiated with 3 broadcast rounds. In them, each party broadcasts

$$(n-1) \cdot \text{mc} \cdot \log(p)$$

bits to commit to the triples as well as input blindings and the blinding used in the consistency check of $\Pi_{\text{HOTCommit}}^{1-n}$. Table 2 gives an overview of the amount of bits that has to be send over broadcast as well as the number of rounds for each of these operations.

Broadcast Online Phase. The online phase includes (triple) verification, the input of secret values for all parties, the circuit evaluation and the output, which includes verification and opening of $\mathcal{F}_{\text{WVole}}$. We assume that in the circuit $\mathcal{C}$ a maximum of $\mathcal{W}$ multiplications can be performed in parallel and the depth $\mathcal{D}$ is the maximum number of multiplications that have to be performed consecutively. The online phase can be completed in $4 + \mathcal{D}$ rounds, the details for each phase and number of bits can be found in Table 3.

To verify the triples, each party has to communicate 4 openings which amounts to $4m \log(p)$ bits of broadcast. During the input phase, each party broadcasts $(n-1) \cdot \text{in} \cdot 2\lambda$ bits for commitments used in private openings. To then evaluate the circuit, each party must broadcast $2m \log(p)$ bits to perform multiplications using Beaver's circuit randomization technique. During the output phase the final value y is sent to the future using **Delayed Hint** for which a total of $(n-1) \cdot 2\lambda + 2\lambda$ have to be sent via $\mathcal{F}_{\text{GBB}}$. The consistency check that is performed in $\Pi_{\text{HOTCommit}}^{1-n}$ before the release of previously opened values amounts to only $\log(p)$ bits. For the final release of y a total of $(n-1) \cdot 2 \log(p) + \lambda$ bits are sent to the broadcast channel.

More dominant is the communication part of **Open** in $\Pi_{\text{HOTCommit}}^{1-1}$, where each instance must broadcast $A_{\text{pub}}, A_{\text{delay}}, \mathbf{S}, \mathbf{D}$ which communicates

$$(n-1) \cdot \log(p) \cdot (\ell \cdot k_{\mathcal{C}} + \ell \cdot n_{\mathcal{C}} + (\text{in} + 5m)) + n \cdot 2\lambda$$

bits via the broadcast channel. In $\mathcal{F}_{\text{WVole}}$ the main communication stems from **Deliver** and **InitDone** run by Π_{IndWVole} where we broadcast the $\mathbf{t}_{i,j}, \mathbf{r}_{i,j}, c_{i,j}$ values which amount to

$$(n-1) \cdot 2n_{\mathcal{C}} \cdot (\lambda \cdot N + (N-1) \cdot \ell \cdot \log(p))$$

bits of communication. The overall amount of bits during **Output** sum up to

$$\begin{aligned} & (2n-1) \cdot 2\lambda + 3\lambda + \log(p) + (n-1) \cdot 2 \log(p) \\ & + (n-1) \cdot \log(p) \cdot (\ell \cdot k_{\mathcal{C}} + \ell \cdot n_{\mathcal{C}} + (\text{in} + 5m)) \\ & + (n-1) \cdot 2n_{\mathcal{C}} \cdot (\lambda \cdot N + (N-1) \cdot \ell \cdot \log(p)). \end{aligned}$$

Peer-to-peer communication. All communication that is peer-to-peer comes from $\mathcal{F}_{\text{OLE}}$ and private openings done in $\mathcal{F}_{\text{HOTCom}}$. As our construction is flexible in which schemes to use for implementing $\mathcal{F}_{\text{OLE}}$ an exact number of rounds as well as the length of messages can not be given. In the offline phase each pair of parties uses **Extend** in $\mathcal{F}_{\text{OLE}}$ twice, where they extend $u, x \in \mathbb{F}^{\ell}$ to $v, w \in \mathbb{F}^{\ell}$. OLE can be implemented in communication sublinear in ℓ [15] in a constant number of rounds. The opening of the in input commitments via **Hint** sends a total of $\text{in} \cdot \log(p)$ bits with another $\text{in} \cdot (\log(p) + 2\lambda)$ bits during **Open**.

Cost of auditability. To examine the costs of adding public audibility to the underlying MPC protocol, we compare our work to [30]. Their construction is secure

in the dishonest majority setting and supports dynamic switching of committees. For the comparison we assume that no switching is performed. This paper is a prime candidate for comparison as the offline phase in our construction is quite similar to the one in [30]. The first source of additional cost is the need for a bulletin board rather than a standard broadcast channel. In addition, there are three further sources of overhead. First, the matrices for the linear codes used in $\mathcal{F}_{\text{HOTCom}}$ must be communicated. Their dimensions are at most $\ell \cdot n_C = O(|C|)$. Second, instead of using a global MAC key for all required values (as in [30]), our construction requires a separate commitment for each value. Third, compared to the [30] approach, where private openings are sent only to the receiving party, each party must additionally post a commitment to the bulletin board, which also applies to delayed openings in a similar way.

In terms of triple verification [30] and our construction use different approaches but both require 6 openings in total. In their approach the $c = a \cdot b$ value has to be authenticated using one opening and every gate in the circuit is evaluated once using the actual value and once using a randomized value, consuming two triples for each multiplication. In our approach we use the opening procedure from [28]. [30] can be realized in different ways, combining their preprocessing with the SPDZ implementation of ([25], [24]) where triples are verified during the offline phase yielding the most efficient online communication. When using this implementation the communication during the preprocessing phase are $O(|C|) + O(n \log(|C|))$ field elements, while the communication during the online phase can be as low as $2|C|$ field elements. This however ignores that the preprocessing phase uses programmable PCGs whose communication is dependent on the security parameter (λ). When the triple verification is not performed in the offline phase and their dynamic online phase is used (allowing one switch of committees between offline and online phase) the communication complexity in the preprocessing phase is lower with $O(n \log |C|)$ field elements, while the online phase communication is higher with $6|C|$ field elements. Also ignoring the programmable PCGs implementation.

Our approach requires $O(n \cdot \lambda \cdot |C|)$ bits to be communicated per party during the preprocessing phase while our triple verification requires $n(3 \cdot m)$ field elements. The remainder of our online phase requires roughly $|C| \cdot n$ field elements and additional $O(n \cdot \lambda^2 \cdot |C|)$ bits where most of the overhead is due to the commitment openings. This can trivially be lowered to $O(n \cdot \lambda \cdot \log(\lambda) \cdot |C|)$ by generating the $\mathbf{t}_{i,j}, r_{i,j}$ in Π_{IndWVole} as output of a GGM tree and only revealing seeds of the opened paths, as done in [7,6].

D Constructing $\mathcal{F}_{\text{WVole}}$ for Theorem 1

We will now outline how to construct $\mathcal{F}_{\text{WVole}}^{p,N,C}$ with parameters as required for $\mathcal{F}_{\text{HOTCom}}$. As in [7] we will achieve this in two steps: first, we build an intermediate $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_C}}$ functionality where $k_C = n_C$, i.e. all columns of $\mathbf{U}$ are random and independent. Then, using $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_C}}$ we build the linear-code $\mathcal{F}_{\text{WVole}}^{p,N,C}$ by letting

Offline	Total bits	Big-O (bits)	Rounds
$\mathcal{C}, H$	$ \mathcal{C} + (n-1) \cdot 2\lambda$	$n \cdot \lambda$	1
$\mathbf{C}, \tilde{\mathbf{U}}, \tilde{\mathbf{V}}, r_{cc}$	$(n-1) \log(p) \cdot ((2\ell+1)(n_C - k_C) + n_C + k_C + 1)$	$n \cdot \log(p) \cdot \ell \cdot \lambda$	2
$\{r_j\}_{j \in \text{in}}, \{a_k, b_k\}_{k \in 2m}$	$(\text{in} + 4m) \cdot (n-1) \cdot \log(p)$	$n(\text{in} + m) \cdot \log(p)$	1
$\{c_k\}_{k \in 2m}$	$(2m) \cdot (n-1) \cdot \log(p)$	$n \cdot m \cdot \log(p)$	1

Table 2: Overview of offline phase communication through $\mathcal{F}_{\text{GGB}}$, where only $\mathcal{P}_1$ sends $\mathcal{C}$. Bits are counted for one party, the commitment of r_{cc} is used for the consistency check in $\Pi_{\text{HOTCommit}}^{1-n}$.

Online	Total bits	Big-O (bits)	Rounds
Triple Verify	$4(n-1) \cdot \log(p) + 2(n-1) \cdot 2\lambda$	$n(\log(p) + \lambda)$	1
Input	$\text{in} \cdot \log(p)$	$\text{in} \cdot \log(p)$	1
Multiply	$\mathcal{W} \cdot 2n(\log(p))$	$(\mathcal{W} \cdot n) \cdot \log(p)$	$\mathcal{D}$
Output	$(2n-1) \cdot 2\lambda + 3\lambda + \log(p) + (n-1) \cdot 2\log(p)$ $(n-1) \cdot \log(p) \cdot (\ell \cdot k_C + \ell \cdot n_C + (\text{in} + 5m))$ $+(n-1) \cdot 2n_C \cdot (\lambda \cdot N + (N-1) \cdot \ell \cdot \log(p))$	$n \cdot \lambda(\mathcal{C} + \lambda^2)$ $+\lambda \cdot \ell \cdot \log(p)$	2

Table 3: Overview of online phase communication through $\mathcal{F}_{\text{GGB}}$. Bits are counted for one party.

$\mathcal{P}_S$ correlate the columns of $\mathbf{U}$ according to $\mathcal{C}$ row-wise and using a consistency check to guarantee that the correlation is correct.

D.1 Building $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$

We now describe the protocol Π_{IndWVole} , which implements $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$. Our construction is very simple and uses the random oracle model to construct commitments, where we build subspace VOLE from a batch vector commitment. It also is highly unoptimized with respect to communication, as we mainly focus on making the protocol from [7] auditable. Any optimizations for smaller vector commitments and openings from [7] and other works also apply to our protocol.

We let $N = \text{poly}(\lambda)$, $N < p$. Let n_c be given. The protocol runs between $\mathcal{P}_S, \mathcal{P}_R, \mathcal{P}_A$ and has access to global functionalities $\mathcal{F}_{\text{GPRO}}, \mathcal{F}_{\text{GBB}}$. Let $\mathcal{T} = \{0, 1\}^{2\lambda}$ be the challenge space.

Init On $(\text{Init}, \text{sid}, \ell)$, $\mathcal{P}_S$ does the following:

1. For $i \in [N], j \in [n_c]$, sample $\mathbf{t}_{i,j} \xleftarrow{\$} \mathbb{F}_p^\ell$ and $r_{i,j} \xleftarrow{\$} \{0, 1\}^\lambda$.
2. For each $j \in [n_c]$ compute $\mathbf{u}_j := \sum_{i \in [N]} \mathbf{t}_{i,j}$ and $\mathbf{v}_j := -\sum_{i \in [N]} i \cdot \mathbf{t}_{i,j}$.
3. For each $i \in [N], j \in [n_c]$ send $(\text{Query}, \text{sid}, \mathbf{t}_{i,j} \| r_{i,j}, \{0, 1\}^{2\lambda})$ to $\mathcal{F}_{\text{GPRO}}$ and obtain $c_{i,j}$. Then send $(\text{Query}, \text{sid}, \ell \| (c_{i,j})_{i \in [N]}^{j \in [n_c]}, \{0, 1\}^{2\lambda})$ to $\mathcal{F}_{\text{GPRO}}$ and obtain H and send $(\text{Query}, \text{sid}, \ell \| H, \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ and obtain $h \in \mathcal{T}$.
4. Set $\mathbf{U} = [\mathbf{u}_1 \| \dots \| \mathbf{u}_{n_c}]$ and $\mathbf{V} = [\mathbf{v}_1 \| \dots \| \mathbf{v}_{n_c}]$. Then broadcast (ℓ, H) and output $(\mathbf{U}, \mathbf{V}, h)$.

Upon seeing (ℓ, H) on $\mathcal{F}_{\text{GBB}}$, $\mathcal{P}_R$ sends $(\text{Query}, \text{sid}, \ell \| H, \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ and denotes the response as $h \in \mathcal{T}$. Then it outputs $(\text{Init}, \text{sid}, \ell, h)$.

Init Done Upon input $(\text{InitDone}, \text{sid}, m)$, $\mathcal{P}_S$ does the following:

1. $\mathcal{P}_S$ broadcasts $(m, (c_{i,j})_{i \in [N]}^{j \in [n_c]})$.
2. $\mathcal{P}_R$ sends $(\text{Query}, \text{sid}, \ell \| (c_{i,j})_{i \in [N]}^{j \in [n_c]}, \{0, 1\}^{2\lambda})$ to $\mathcal{F}_{\text{GPRO}}$. If the result differs from H then it aborts.
3. Both $\mathcal{P}_S$ and $\mathcal{P}_R$ send $(\text{Query}, \text{sid}, m \| (c_{i,j})_{i \in [N]}^{j \in [n_c]}, [N]^{n_c})$ to $\mathcal{F}_{\text{GPRO}}$ and denote the response as Δ . Each party then outputs $(\text{Done}, \text{sid}, \Delta)$.

Deliver On input $(\text{Deliver}, \text{sid})$, $\mathcal{P}_S$ does as follows:

1. Broadcast $\mathbf{t}_{i,j}, r_{i,j}$ for $i \in [N] \setminus \{\Delta[i]\}, j \in [n_c]$.
- Upon observing the broadcast message $\mathbf{t}_{i,j}, r_{i,j}$ for $i \in [N] \setminus \{\Delta[i]\}, j \in [n_c]$ from $\mathcal{P}_S$ and if $\ell, H, m, (c_{i,j})_{i \in [N]}^{j \in [n_c]}$ is broadcast by $\mathcal{P}_S$ as well, $\mathcal{P}_R$ acts as follows:
 1. Send $(\text{Query}, \text{sid}, \mathbf{t}_{i,j} \| r_{i,j}, \{0, 1\}^{2\lambda})$ to $\mathcal{F}_{\text{GPRO}}$ for all $i \in [N] \setminus \{\Delta[j]\}, j \in [n_c]$ and check that the response is $c_{i,j}$. If any $c_{i,j}$ is not correct, abort.
 2. Compute $\mathbf{q}_j := \sum_{i \in [N] \setminus \{\Delta[j]\}} \mathbf{t}_{i,j} (\Delta[j] - i)$ for each $j \in [n_c]$.
 3. Set $\mathbf{Q} = [\mathbf{q}_1 \| \dots \| \mathbf{q}_{n_c}]$ and output $(\text{Corr}, \text{sid}, \mathbf{Q})$.

Audit On input $(\text{Audit}, \text{sid})$ act as follows:

1. Check that the messages $\ell, H, m, (c_{i,j})_{i \in [N]}^{j \in [n_c]}$ and $\mathbf{t}_{i,j}, r_{i,j}$ for $i \in [N] \setminus \{\Delta[i]\}, j \in [n_c]$ were broadcast in this order. If not, then abort.
2. Send $(\text{Query}, \text{sid}, \ell \| H, \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ and denote the response as h .
3. Send $(\text{Query}, \text{sid}, \ell \| (c_{i,j})_{i \in [N]}^{j \in [n_c]}, \{0, 1\}^{2\lambda})$ to $\mathcal{F}_{\text{GPRO}}$. If the result differs from H then abort.
4. Send $(\text{Query}, \text{sid}, m \| (c_{i,j})_{i \in [N]}^{j \in [n_c]}, [N]^{n_c})$ to $\mathcal{F}_{\text{GPRO}}$ and obtain Δ .
5. Compute $\mathbf{Q}$ as $\mathcal{P}_R$ in **Deliver** and output $(\ell, h, m, \Delta, \mathbf{Q})$. Abort if it aborts.

Challenge If ℓ, H has been broadcast send $(\text{Query}, \text{sid}, \ell \| H, \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$, obtain $h \in \mathcal{T}$ and output $(\text{Challenge}, \text{sid}, h)$. Otherwise output $(\text{Challenge}, \text{sid}, \perp)$.

Fig. 18: Protocol Π_{IndWVole}

Theorem 5. *The protocol Π_{IndWVole} , given in Figure 18, securely realizes $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_c}}$ using the global subroutines $\mathcal{F}_{\text{GpRO}}, \mathcal{F}_{\text{GBB}}$. Any malicious adversary making Q calls to $\mathcal{F}_{\text{GpRO}}$ can distinguish with probability at most $Q/2^\lambda + Q/N^{n_c} + Q^2/2^{2\lambda}$.*

As can be seen from the protocol Π_{IndWVole} , all columns of $\mathbf{U}$ are independent and so are all indices of Δ chosen. Thus, we only give the security proof for $n_c = 1$ as the generalization is obvious.

The following equation demonstrates why the protocol is indeed correct:

$$\mathbf{q} = \sum_{i \in [N] \setminus \{\Delta\}} \mathbf{t}_i(\Delta - i) = \sum_{i \in [N]} \mathbf{t}_i(\Delta - i) = \sum_{i \in [N]} \mathbf{t}_i \Delta - \sum_{i \in [N]} i \cdot \mathbf{t}_i = \mathbf{u} \Delta + \mathbf{v} \quad (1)$$

Proof.

Malicious $\mathcal{P}_R$. The high level idea is that the adversary gets to see every $\mathbf{t}_i$, except for $\mathbf{t}_\Delta$, as well as all the r_i except r_Δ . However, unless the adversary can guess r_Δ it cannot distinguish and we can rely on $\mathbf{t}_\Delta$ being random making $\mathbf{u} = \sum_{i \in [N]} \mathbf{t}_i$ uniformly random in $\mathbb{F}_p$.

We now construct a simulator. The simulator, upon receiving $(\text{Init}, \text{sid}, \ell, h)$ from $\mathcal{F}_{\text{WVole}}$, will generate all c_i as uniformly random commitments and program $\mathcal{F}_{\text{GpRO}}$ to output the uniformly random $H \in \{0, 1\}^{2\lambda}$ on input $\ell \parallel (c_i)_{i \in [N]}$. It also programs the random oracle to output h on input H . Then it broadcasts (ℓ, H) . Upon receiving $(\text{Done}, \text{sid}, m, \Delta)$ it programs $\mathcal{F}_{\text{GpRO}}$ to output Δ on input $m \parallel (c_i)_{i \in [N]}$ and broadcasts $(m, (c_i)_{i \in [N]})$.

Upon receiving $(\text{Corr}, \text{sid}, \mathbf{q})$ it samples all $\mathbf{t}_i, i \neq \Delta$ uniformly at random from $\mathbb{F}_p^\ell$ subject to the constraint that they add up to $\mathbf{q}$ for Δ according to Equation 1. Then program $\mathcal{F}_{\text{GpRO}}$ to output c_i on input $\mathbf{t}_i \parallel r_i$ for uniformly random r_i , and broadcast all $\mathbf{t}_i, r_i$ for $i \neq \Delta$. The resulting protocol output is indistinguishable to an adversary as the $\mathbf{t}_i$ are chosen uniformly in both cases, unless it has sent a query to $\mathcal{F}_{\text{GpRO}}$ before on input $\mathbf{t}_i, r_i$ so that it cannot be programmed to output c_i . Similarly, since the c_i are chosen at random and the random oracle is programmed to output Δ before they are broadcast, $\mathcal{A}$ can only detect the programming if it queried the Δ -generating input in advance. The same reasoning also holds for H . Since r_i, c_i, H are chosen uniformly at random from $\{0, 1\}^{2\lambda}$ by the simulator and the adversary makes at most Q queries, this happens with probability at most $Q/2^{2\lambda}$.

Malicious $\mathcal{P}_S$. The simulator will react to queries to $\mathcal{F}_{\text{GpRO}}$. For all inputs to $\mathcal{F}_{\text{GpRO}}$, if they are of the form $\ell \parallel H$ where H itself is a previous output of $\mathcal{F}_{\text{GpRO}}$ on inputs $(c_i)_{i \in [N]}$ then find all $\mathcal{F}_{\text{GpRO}}$ preimages of all c_i (abort if there is more than one or one, and abort if there is more than one preimage for H). Construct

$\mathbf{u}, \mathbf{v}$ as in Equation 1, and use **Challenge Test** to generate the response h to be output by $\mathcal{F}_{\text{GpRO}}$. For repeated queries to $\mathcal{F}_{\text{GpRO}}$, respond with the previous response. If some of the (c_i) do not have preimages then emulate the RO honestly. In addition, for all inputs of the form $m \parallel (c_i)_{i \in [N]}$ when all c_i are queried and valid as mentioned above, use **Delta Test** to generate the response Δ to be output by $\mathcal{F}_{\text{GpRO}}$. For any other input to the RO, emulate the Random Oracle honestly.

The simulator, upon obtaining the $\ell \parallel H$ broadcast by the adversary, observes the random oracle queries that output c_i as outlined above. Again, if there is any collision (i.e. two inputs generate the same c_i as output) then abort. If any c_i has no valid preimage, then sample the corresponding $\mathbf{t}_i$ uniformly at random. If the resulting query to $\mathcal{F}_{\text{GpRO}}$ has not been made before that would generate h , then execute it as outlined in the previous paragraph. Construct $\mathbf{u}, \mathbf{v}$ as in Equation 1 and send them to $\mathcal{F}_{\text{WVOLE}}$ where c is chosen to be the consistent index with the previously made $\mathcal{F}_{\text{GpRO}}$ query and $\mathcal{F}_{\text{WVOLE}}$ response.

Wait for the adversary to broadcast $m, (c_i)_{i \in [N]}$ and check if $\ell \parallel (c_i)_{i \in [N]}$ hashes to H . If not then abort, otherwise send $(\text{InitDone}, \text{sid}, m)$ to $\mathcal{F}_{\text{WVOLE}}$.

Wait for the adversary to broadcast openings according to all the c_i except c_Δ , as in the protocol. If any opening is invalid then abort. If c_Δ did not have a valid preimage as a random oracle output, then also abort. Otherwise, send $(\text{Deliver}, \text{sid})$ to $\mathcal{F}_{\text{WVOLE}}$.

If all $c_i \in \{0, 1\}^{2\lambda}$ have a unique preimage which are opened correctly by the adversary, then the simulation is perfect. If the adversary finds a collision in the Random Oracle for an output c_i or for H then the simulation aborts, but the probability for this event is $Q^2/2^{2\lambda}$ by Proposition 1 as the adversary makes at most Q queries to $\mathcal{F}_{\text{GpRO}}$. The simulation also aborts if more than one c_i didn't have a preimage that could be extracted or if H did not have a preimage, but in that case the adversary will not be able to open these during the **Deliver** phase of the protocol². However, if the adversary did not generate c_Δ as a random oracle output but all other are correctly formed, then the simulation fails as the simulator aborts. However, Δ is chosen by the functionality at random after all c_i are fixed via $\mathcal{F}_{\text{GpRO}}$ in the simulation. Here, the adversary may query $\mathcal{F}_{\text{GpRO}}$ (and thus trigger **Delta Test**) before broadcasting and the probability of $\mathcal{A}$ guessing Δ correctly is $1/N$ for each such query. If there are n_C parallel instances, then the chance of this event is $1/N^{n_C}$. As the adversary can make at most Q queries to the random oracle, his success probability of achieving this event is Q/N^{n_C} as stated in the theorem.

Malicious $\mathcal{P}_S$ and $\mathcal{P}_R$. The simulator works identical to the case of the dishonest sender outlined above.

² To be precise, the adversary could make random oracle queries after broadcasting H or m, c_i to match these with valid openings, but the success probability is bounded by $(N+1)Q/2^{2\lambda}$ which is substantially smaller than the $Q^2/2^{2\lambda}$ collision probability.

D.2 From $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_C}}$ to $\mathcal{F}_{\text{WVole}}^{p,N,C}$

Let $\mathcal{C}$ be a systematic $[n_C, d_C, k_C]$ -code. We assume that it is linear, systematic and has generator matrix $\mathbf{G} \in \mathbb{F}_p^{k_C \times n_C}$.

In the construction, we use the $1/p$ -universal hash functions as defined in Definition 2. Its matrices $\mathbf{H} \in \mathbb{F}_p^{1 \times (n+1)}$ are n -hiding according to Definition 2.

The protocol follows a straightforward approach. $\mathcal{P}_S$, upon obtaining $(\bar{\mathbf{U}}, \mathbf{V})$ from $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_C}}$ considers the first k_C columns of $\bar{\mathbf{U}}$ as the output, whereas $\mathbf{V}$ remains untouched. Since we assume that $\mathcal{C}$ is systematic, we therefore have to correct the remaining $n_C - k_C$ columns of $\bar{\mathbf{U}}$ to be $\mathbf{U}\mathbf{G}$. $\mathcal{P}_S$ does this by computing the difference $[\mathbf{0} \ \mathbf{C}] = \mathbf{U}\mathbf{G} - \bar{\mathbf{U}}$ and broadcasting $\mathbf{C}$. Note that the first k_C columns of $\mathbf{U}\mathbf{G} - \bar{\mathbf{U}}$ must be zero, as they define the messages of the row codewords.

After receiving $\Delta, \bar{\mathbf{Q}}$ from $\mathcal{F}_{\text{WVole}}$, $\mathcal{P}_S$ can now correct it to $\mathbf{Q}$ by adding $[\mathbf{0} \ \mathbf{C}]\text{diag}(\Delta)$ to it. This is correct because

$$\begin{aligned} \bar{\mathbf{Q}} + [\mathbf{0} \ \mathbf{C}]\text{diag}(\Delta) &= \mathbf{V} + \bar{\mathbf{U}}\text{diag}(\Delta) + [\mathbf{0} \ \mathbf{C}]\text{diag}(\Delta) \\ &= \mathbf{V} + \bar{\mathbf{U}}\text{diag}(\Delta) + (\mathbf{U}\mathbf{G} - \bar{\mathbf{U}})\text{diag}(\Delta) \\ &= \mathbf{V} + \mathbf{U}\mathbf{G}\text{diag}(\Delta) \end{aligned}$$

However, this correction does not make sure that $\mathbf{C}$ is correctly formed. Thus in addition, $\mathcal{P}_S$ also samples an ℓ -hiding universal linear hash function $\mathbf{H}$ based on $\mathbf{C}$ and applies it both to $\mathbf{U}$ and $\mathbf{V}$. It then broadcasts the resulting $\tilde{\mathbf{U}}, \tilde{\mathbf{V}}$. Looking ahead, if $\mathbf{Q} = \mathbf{U}\mathbf{G}\text{diag}(\Delta) + \mathbf{V}$, then $\mathbf{H}\mathbf{Q} = \tilde{\mathbf{U}}\mathbf{G}\text{diag}(\Delta) + \tilde{\mathbf{V}}$. At the same time, if $\mathbf{C}$ will not correct $\mathbf{Q}$ to the aforementioned relation (or if $\tilde{\mathbf{U}}, \tilde{\mathbf{V}}$ are not well-formed), then the resulting check including Δ will fail with very high probability over the random choice of Δ . To produce ℓ output rows, we let the sender produce $\ell + 1$ correlated rows and use that $\mathbf{H}$ is ℓ -hiding as it adds the last row in the process, which is discarded after the check.

Towards generating challenges, we assume that the hybrid functionality $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_C}}$ in our protocol uses an output challenge set $\mathcal{T} = \{0, 1\}^{2\lambda}$ and itself generates the same challenge set. We then hash the challenge it generates, together with the protocol transcript, to define the challenge output of this functionality.

Theorem 6. *Let $\mathcal{C}$ be a systematic $[n_C, d_C, k_C]$ -linear code. The protocol $\Pi_{\text{CodeWVole}}$, given in Figure 19, securely realizes $\mathcal{F}_{\text{WVole}}^{p,N,C}$ in the $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_C}}$ -hybrid model using the global subroutines $\mathcal{F}_{\text{GPRO}}, \mathcal{F}_{\text{GBB}}$. Any malicious adversary making Q calls to $\mathcal{F}_{\text{GPRO}}$ or **Challenge Test** or **Delta Test** can distinguish with probability at most $Q/p^{n_C+k_C} + Q\binom{n_C}{k_C+1}/p + Q^2/2^{2\lambda}$.*

If all parties are honest, then completeness is clear from the description above. Our proof generally follows the strategy in [7].

Protocol $\Pi_{\text{CodeWVole}}$

$\mathcal{H} \subseteq \mathbb{F}_p^{1 \times (\ell+1)}$ is a family of ℓ -hiding, $1/p$ -universal linear hash functions. The protocol uses $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$, $\mathcal{F}_{\text{GPRO}}$, $\mathcal{F}_{\text{GBB}}$ as hybrid functionalities. Let $\mathcal{T} = \{0, 1\}^{2\lambda}$ be the challenge space.

Init On (Init, sid, ℓ), $\mathcal{P}_S$ runs the following protocol:

1. Send (Init, sid, $\ell + 1$) to $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$.
2. Receive $\bar{\mathbf{U}} \in \mathbb{F}_p^{(\ell+1) \times n_C}$, $\mathbf{V} \in \mathbb{F}_p^{(\ell+1) \times n_C}$ and h from $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$.
3. Let $\mathbf{U} = \bar{\mathbf{U}}^{[1..k_C]}$ be the first k_C columns of $\bar{\mathbf{U}}$.
4. Let $[\mathbf{0} \ \mathbf{C}] := \mathbf{U}\mathbf{G} - \bar{\mathbf{U}}$ where $\mathbf{C} \in \mathbb{F}_p^{(\ell+1) \times (n_C - k_C)}$ and broadcast $\mathbf{C}$.
5. Choose $\mathbf{H} \in \mathcal{H}$ as the response to the query (Query, sid, $h \parallel \mathbf{C}$, $\mathbb{F}_p^\ell$) to $\mathcal{F}_{\text{GPRO}}$ and compute $\tilde{\mathbf{U}} := \mathbf{H}\mathbf{U}$ and $\tilde{\mathbf{V}} := \mathbf{H}\mathbf{V}$.
6. Broadcast $(\tilde{\mathbf{U}}, \tilde{\mathbf{V}})$ and output $(\mathbf{U}_{[1..\ell]}, \mathbf{V}_{[1..\ell]})$.

$\mathcal{P}_R$, upon obtaining (Init, sid, ℓ , h) from $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$ and observing the broadcasts $\mathbf{C}$, $\tilde{\mathbf{U}}$, $\tilde{\mathbf{V}}$ queries $\mathcal{F}_{\text{GPRO}}$ on input (Query, sid, $h \parallel \mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}}$, $\mathcal{T}$) to obtain h' . Then it outputs (Done, sid, ℓ , h').

Init Done On input (InitDone, sid, m) $\mathcal{P}_S$ sends (InitDone, sid, $\mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}} \parallel m$) to $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$. Both $\mathcal{P}_S$ and $\mathcal{P}_R$ obtain m , Δ , and $\mathcal{P}_R$ checks that the output of $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$ is consistent with the broadcast $\tilde{\mathbf{U}}, \tilde{\mathbf{V}}, \mathbf{C}$.

Deliver On (Deliver, sid), $\mathcal{P}_S$ does the following:

1. Send (Deliver, sid) to $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$.
- $\mathcal{P}_R$ then does the following:
1. Check if $\mathcal{P}_S$ broadcast $\mathbf{C}$ first and then $(\tilde{\mathbf{U}}, \tilde{\mathbf{V}})$. If not, abort.
 2. Obtain (Corr, sid, $\bar{\mathbf{Q}}$) from $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$. By definition, $\bar{\mathbf{Q}} = \bar{\mathbf{U}}\text{diag}(\Delta) + \mathbf{V}$.
 3. Compute $\mathbf{Q} := \bar{\mathbf{Q}} + [\mathbf{0} \ \mathbf{C}]\text{diag}(\Delta)$.
 4. Recompute $\mathbf{H}$ as the response to sending (Query, sid, $h \parallel \mathbf{C}$, $\mathbb{F}_p^\ell$) to $\mathcal{F}_{\text{GPRO}}$.
 5. Abort if $\tilde{\mathbf{V}} \neq \mathbf{H}\mathbf{Q} - \tilde{\mathbf{U}}\text{diag}(\Delta)$. Otherwise output $(\mathbf{Q}_{[1..\ell]})$.

Audit On input (Audit, sid) $\mathcal{P}_A$ acts as follows:

1. Obtain $(\ell, h, \mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}} \parallel m, \Delta, \bar{\mathbf{Q}})$ by sending (Audit, sid) to $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$. If there are no broadcasts of $\mathbf{C}$ or $\tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}}$ on $\mathcal{F}_{\text{GBB}}$ then abort.
2. Send (Query, sid, $h \parallel \mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}}$, $\mathcal{T}$) to $\mathcal{F}_{\text{GPRO}}$ to obtain h' .
3. Compute $\mathbf{Q}$ as $\mathcal{P}_R$ does in **Deliver** and abort if it aborts.
4. Output $(\ell, h', m, \Delta, \mathbf{Q})$.

Challenge On input (Challenge, sid):

- Send (Challenge, sid) to $\mathcal{F}_{\text{WVole}}^{p, N, \mathbb{F}_p^{n_C}}$ and obtain h . If $h = \perp$ or $\mathbf{C}, \tilde{\mathbf{U}}, \tilde{\mathbf{V}}$ have not been broadcast then output (Challenge, sid, $\perp$).
- Query $\mathcal{F}_{\text{GPRO}}$ on input (Query, sid, $h \parallel \mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}}$, $\mathcal{T}$) to obtain h' . Then output (Challenge, sid, h').

Fig. 19: Protocol $\Pi_{\text{CodeWVole}}$ achieving subspace VOLE

Proof.

Malicious $\mathcal{P}_R$. The simulator works as follows: it honestly simulates an instance

of $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_c}}$ towards the adversary. Upon obtaining $(\text{Done}, \text{sid}, \ell, h)$ from $\mathcal{F}_{\text{WVole}}^{p,N,\mathcal{C}}$ it first samples $\bar{h} \in \{0, 1\}^{2\lambda}$ uniformly at random. Then sample $\mathbf{C} \in \mathbb{F}_p^{(\ell+1) \times (n_c - k_c)}$ uniformly at random. Choose $\mathbf{H}$ consistently with $\mathcal{F}_{\text{GpRO}}$ and choose uniformly random $\tilde{\mathbf{U}} \in \mathbb{F}_p^{k_c}, \tilde{\mathbf{V}} \in \mathbb{F}_p^{n_c}$. Program $\mathcal{F}_{\text{GpRO}}$ to output h on input $\bar{h} \parallel \mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}}$ where $\bar{h}$ was chosen as outlined above. Finally, we let $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_c}}$ output $(\text{Init}, \text{sid}, \ell + 1, \bar{h})$ and broadcast $\mathbf{C}$ as well as $(\tilde{\mathbf{U}}, \tilde{\mathbf{V}})$.

Upon obtaining $(\text{InitDone}, \text{sid}, m, \Delta)$ from $\mathcal{F}_{\text{WVole}}^{p,N,\mathcal{C}}$ let $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_c}}$ deliver $(\text{Done}, \text{sid}, \mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}} \parallel m, \Delta)$.

Upon obtaining $(\text{Corr}, \text{sid}, \mathbf{Q})$ from $\mathcal{F}_{\text{WVole}}^{p,N,\mathcal{C}}$ set $\bar{\mathbf{Q}}_{[1..\ell]} = \mathbf{Q} - [\mathbf{0} \ \mathbf{C}]_{[1..\ell]} \text{diag}(\Delta)$ and $\bar{\mathbf{Q}}_{\ell+1} = \tilde{\mathbf{V}} + \tilde{\mathbf{U}} \mathbf{G} \text{diag}(\Delta) - \mathbf{H}^{[1..\ell]} \mathbf{Q}_{[1..\ell]}$. Let $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_c}}$ output $(\text{Corr}, \text{sid}, \Delta, \bar{\mathbf{Q}})$.

Note that by construction, the chosen $\tilde{\mathbf{U}}, \tilde{\mathbf{V}}, \mathbf{C}$ and the $\Delta, \bar{\mathbf{Q}}$ fulfill the check in Step 5 of **Deliver** of $\Pi_{\text{CodeWVole}}$. It remains to show that all other values have the same distribution as in the protocol.

Note that $\mathbf{C}$ as generated in the protocol includes the term $\bar{\mathbf{U}}$ which is uniformly random. $\mathcal{P}_R$ never learns $\bar{\mathbf{U}}$, but only $\bar{\mathbf{Q}}$ which additionally contains the uniformly random additive term $\mathbf{V}$. Thus, $\mathbf{C}$ is identically distributed in both cases. Moreover, by the ℓ -hiding property of the universal hash function family $\mathcal{H}$ and the fact that $\mathbf{U}_{\ell+1}, \mathbf{V}_{\ell+1}$ are uniformly random in $\mathbb{F}_p^{k_c}$ and $\mathbb{F}_p^{n_c}$ respectively, $\tilde{\mathbf{U}}$ and $\tilde{\mathbf{V}}$ also have the same distribution in both settings. Finally, the value Δ is consistent. The rows $\bar{\mathbf{Q}}$ are uniquely defined to be consistent with $\mathbf{U}, \mathbf{V}, \Delta$ of $\mathcal{F}_{\text{WVole}}^{p,N,\mathcal{C}}$ and the last row is uniquely defined by the check in Step 5. Thus, the simulation is perfectly indistinguishable except for calls of $\mathcal{A}$ to $\mathcal{F}_{\text{GpRO}}$. However, when we program $\mathcal{F}_{\text{GpRO}}$ on input $\bar{h} \parallel \mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}}$ to output h then $\mathcal{A}$ had to predict the uniformly random input $\bar{h} \in \{0, 1\}^{2\lambda}$ in order to detect the programming. Given that $\mathcal{A}$ makes Q queries to $\mathcal{F}_{\text{GpRO}}$, this happens with probability at most $Q/2^{2\lambda}$.

Malicious $\mathcal{P}_S$. The simulator again honestly simulates an instance of $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_c}}$ towards the adversary. We will take care of the adversarial calls to **Challenge Test** and **Delta Test** below.

The hybrid functionality outputs $\bar{\mathbf{U}}, \mathbf{V}$ to the adversary (or lets it choose both), who in response broadcasts $\mathbf{C}$ and thereafter $\tilde{\mathbf{U}}, \tilde{\mathbf{V}}$. Note that this is tied to one choice of h generated by $\mathcal{F}_{\text{WVole}}^{p,N,\mathbb{F}_p^{n_c}}$.

Generate candidate correlation inputs. From these broadcasts, the simulator must first obtain inputs for the ideal functionality $\mathcal{F}_{\text{WVole}}^{p,N,\mathcal{C}}$ which are consistent with the protocol messages, and send them to $\mathcal{F}_{\text{WVole}}^{p,N,\mathcal{C}}$ for **Init**.

Recompute $\mathbf{H}$ by making the query $(\text{Query}, \text{sid}, h\|\mathbf{C}, \mathbb{F}_p^\ell)$ to $\mathcal{F}_{\text{GPRO}}$ as in the protocol and set $\mathbf{U}_D := \overline{\mathbf{U}} + [\mathbf{0} \ \mathbf{C}]$. Then

$$\begin{aligned}\mathbf{Q} &= \overline{\mathbf{Q}} + [\mathbf{0} \ \mathbf{C}] \text{diag}(\Delta) \\ &= \overline{\mathbf{U}} \text{diag}(\Delta) + \mathbf{V} + [\mathbf{0} \ \mathbf{C}] \text{diag}(\Delta) \\ &= \mathbf{U}_D \text{diag}(\Delta) + \mathbf{V}\end{aligned}$$

would be obtained by the receiver. The adversary is supposed to respond with $\mathbf{H}\mathbf{U}_D$ and $\mathbf{H}\mathbf{V}$ in the consistency check. We define the differences to the actual values $\hat{\mathbf{U}}, \hat{\mathbf{V}}$ which it sends as $\hat{\mathbf{U}} = \mathbf{H}\mathbf{U}_D - \hat{\mathbf{U}}\mathbf{G}$ and $\hat{\mathbf{V}} = \mathbf{H}\mathbf{V} - \hat{\mathbf{V}}$.

For the correct responses, $\mathbf{H}\mathbf{V} = \mathbf{H}\mathbf{Q} - \mathbf{H}\mathbf{U}_D \text{diag}(\Delta)$ must hold for all Δ . If the protocol consistency check $\hat{\mathbf{V}} = \mathbf{H}\mathbf{Q} - \hat{\mathbf{U}}\mathbf{G} \text{diag}(\Delta)$ also holds, then we have that $\hat{\mathbf{V}} = -\hat{\mathbf{U}} \text{diag}(\Delta)$. Thus, for the check to pass, we can solve this system of linear equations for Δ .

If no solution for Δ exists, then the simulator aborts. This is because the adversary cannot pass the check for any choice of Δ of the ideal functionality. Otherwise, call such a solution Δ^* . There are indices $\text{Cols} \subseteq [n_C]$ corresponding to the non-zero columns of $\hat{\mathbf{U}}$ and for those $i \in \text{Cols}$ we must later have that $\Delta[i] = \Delta^*[i]$. Otherwise, the protocol consistency check will fail anyways.

Let $\mathcal{C}_{-\text{Cols}}$ be the code $\mathcal{C}$ punctured at all indices in Cols . We decode all rows of $\mathbf{U}_D$ after removing columns corresponding to Cols according to $\mathcal{C}_{-\text{Cols}}$. If any row of $\mathbf{U}_D$ (punctured at Cols) is not already in $\mathcal{C}_{-\text{Cols}}$ then abort. If $|\text{Cols}| > d_C$ then pick a random decoding that is consistent. Otherwise, let $\mathbf{U}^*$ be the decoded rows, set $\mathbf{V}^* = \mathbf{V} + (\mathbf{U}_D - \mathbf{U}^*\mathbf{G}) \text{diag}(\Delta^*)$ and send the first ℓ rows of $\mathbf{U}^*, \mathbf{V}^*$ to $\mathcal{F}_{\text{WVOLE}}^{p,N,C}$. This means that the simulator will first send $(\text{TestChallenge}, \text{sid}, \ell, \mathbf{U}_{[1..\ell]}^*, \mathbf{V}_{[1..\ell]}^*)$ and then $(\text{Sender}, \text{sid}, \mathbf{U}_{[1..\ell]}^*, \mathbf{V}_{[1..\ell]}^*, 0)$ to $\mathcal{F}_{\text{WVOLE}}^{p,N,C}$.

Correctness of the extracted inputs. If the functionality $\mathcal{F}_{\text{WVOLE}}^{p,N,C}$ delivers output, then upon fixing Δ it holds that

$$\begin{aligned}\mathbf{V}^* &= \mathbf{V} + \mathbf{U}_D \text{diag}(\Delta) - \mathbf{U}^*\mathbf{G} \text{diag}(\Delta) \\ &= \mathbf{Q} - \mathbf{U}^*\mathbf{G} \text{diag}(\Delta)\end{aligned}$$

This is consistent on all columns not in Cols as $\mathbf{U}_D - \mathbf{U}^*\mathbf{G}$ vanishes, and for those in Cols it is correct if $\Delta[i] = \Delta^*[i]$, which holds if the consistency check in the protocol passed. This also means that the outputs of $\mathcal{F}_{\text{WVOLE}}$ as well as the protocol fulfill the same required correlation.

Note that we abort the simulation in two cases: either if no solution Δ^* could be found, or if some rows in the punctured $\mathbf{U}_D$ do not lie in $\mathcal{C}_{-\text{Cols}}$. For the first event, this implies that the consistency check will fail and can be discarded. Therefore, we must bound the probability of the second event. For this, let $\mathcal{K}$ be the collection of sets of columns of $\mathbf{G}$ such that every $c \in \mathcal{K}$ is linearly dependent, while all proper subsets of c are linearly independent. By construction, $|\mathcal{K}| =$

$\binom{n_c}{\dim+1}$. Hence, every element in $\mathcal{K}$ is a proper sub-code of $\mathcal{C}$ with minimal distance 1.

If the consistency check of the protocol passes, then

$$\tilde{\mathbf{V}} = \mathbf{H}\mathbf{Q} - \tilde{\mathbf{U}}\mathbf{G}\text{diag}(\Delta) = \mathbf{H}\mathbf{V} + \mathbf{H}\mathbf{U}_D\text{diag}(\Delta) - \tilde{\mathbf{U}}\mathbf{G}\text{diag}(\Delta).$$

This requires that $\mathbf{H}\mathbf{U}_D$ is in $\mathcal{C}$ due to the free choice of Δ . Therefore, the indices of $\mathbf{H}\mathbf{U}_D$ that are not in Cols must be in the punctured code $\mathcal{C}_{-\text{cols}}$. We now bound the probability that one (or more rows) of $\mathbf{U}_D$ punctured at Cols are not in $\mathcal{C}_{-\text{cols}}$, while $\mathbf{H}\mathbf{U}_D$ punctured at Cols is in $\mathcal{C}_{-\text{cols}}$.

Towards this, let $\mathbf{P}_c$ be the parity check matrix of the code $\mathcal{C}$. $\mathbf{H}\mathbf{U}_D$ is in $\mathcal{C}$ if and only if $\mathbf{H}\mathbf{U}_D\mathbf{P}_c = \mathbf{0}$. Note that every $c \in \mathcal{K}$ induces a code $\mathcal{C}_c$ of length $k_c + 1$ and dimension k_c . Thus, for every $c \in \mathcal{K}$ we can define a parity check vector $\mathbf{p}_c \in \mathbb{F}_p^{k_c+1}$ for $\mathcal{C}_c$ so that $\forall \mathbf{x} \in \mathbb{F}_p^{k_c+1} : \mathbf{x} \in \mathcal{C}_c \leftrightarrow \langle \mathbf{x}, \mathbf{p}_c \rangle = 0$.

In general, for any $\mathbf{x} \in \mathbb{F}_p^{n_c}$ we have that if for all $c \in \mathcal{K}$ it holds that³ $\langle \mathbf{x}_c, \mathbf{p}_c \rangle = 0$, then it holds that $\mathbf{x}\mathbf{P}_c = \mathbf{0}$. Therefore, in order to bound the probability that a matrix $\mathbf{U}_D$ is not in $\mathcal{C}$ (or a submatrix not in the punctured code) while $\mathbf{H}\mathbf{U}_D$ being in $\mathcal{C}$, we can look at the individual event for each code $\mathcal{C}_c$ and the chosen $\mathbf{H}$ and then apply a union bound over all $c \in \mathcal{K}$. Due to the uniform choice of $\mathbf{H}$ after $\mathbf{U}_D$ is fixed, the probability of $\forall c \in \mathcal{K} : \mathbf{H}(\mathbf{U}_c\mathbf{p}_c^\top) = \mathbf{0}$ can be bounded by $\binom{n_c}{k_c+1}/p$. As the adversary may make up to Q queries to the Random Oracle using different choices of $\mathbf{C}$, each resulting in a different $\mathbf{H}$, its success probability is $Q\binom{n_c}{k_c+1}/p$.

Identifying the correct inputs to the hybrid functionality. The aforementioned argument requires that the simulator can identify the correct $\bar{\mathbf{U}}, \mathbf{V}$ for the input extraction, which is only possible if they were previously provided to the simulated $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ by $\mathcal{A}$. However, since the query that generates $\mathbf{H}$ also includes the input $h \in \mathcal{T}$ that is chosen uniformly at random by the simulator only after it obtained the inputs $\bar{\mathbf{U}}, \mathbf{V}$ from the adversary, the attacker must essentially find two inputs to $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ which generate a collision on h values. The probability of this event is at most $Q^2/2^{2\lambda}$ due to Proposition 1.

Simulating h' . In order to simulate the $\mathcal{F}_{\text{GpRO}}$ queries that generate h' , note that if $\mathcal{A}$ has sent $\bar{\mathbf{U}}, \mathbf{V}$ corresponding to $\mathbf{C}, \tilde{\mathbf{U}}, \tilde{\mathbf{V}}$ to $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ then it has obtained h , which was chosen uniformly at random. Thus, whenever we see an $\mathcal{F}_{\text{GpRO}}$ input query with a h that we generated for an input, we can perfectly simulate the query by computing $\mathbf{U}, \mathbf{V}$ for the **Challenge Test** interface (as well as ℓ) of $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathcal{C}}$ as outlined in *Generate candidate correlation inputs* above. Given its output h' we then program $\mathcal{F}_{\text{GpRO}}$ to give the response h' that was obtained from $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathcal{C}}$. For all other h that were not generated by $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ we respond with random outputs. However, this fails if the wrong input is extracted or if the

³ Here, $\mathbf{x}_c$ is the subvector of $\mathbf{x}$ punctured at all indices except c .

adversary guesses a h that is later generated by $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$. In the former case, this probability is at most $Q \binom{n_c}{k_c+1}/p$ over Q queries to $\mathcal{F}_{\text{GPRO}}$. For the latter event, this happens with probability at most $Q^2/2^{2\lambda}$ due to Proposition 1.

Simulating Challenge Test and Delta Test queries. For any **Challenge Test** query with input $\bar{\mathbf{U}}, \mathbf{V}$ to $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ output a uniformly random value h . For any **Delta Test** query $(\ell + 1, h, \mathbf{C} \parallel \tilde{\mathbf{U}} \parallel \tilde{\mathbf{V}} \parallel m)$ to $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ where h was output by $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ before, we can use the **Challenge Test** interface of $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ as in the h' simulation case to compute the corresponding h' . We can then send (ℓ, h', m) to **Delta Test** of $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ and obtain Δ . Finally, we let $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ output this value Δ . If h was not generated by **Challenge Test** before then generate h' as a random value and use it for the **Delta Test** query to $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$. This fails if we extract the incorrect input to **Challenge Test** of $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$, which happens for all Q queries with probability at most $Q \binom{n_c}{k_c+1}/p$. Alternatively, this fails if different inputs were provided to $\mathcal{F}_{\text{WVOLE}}^{p,N,\mathbb{F}_p^{n_c}}$ which generate the same h . This event happens with probability at most $Q^2/2^{2\lambda}$ due to Proposition 1. In addition, we can have that we program the output of **Delta Test** for a previously unqueried input h that is only later generated by **Challenge Test**. This event happens with probability at most $Q/2^{2\lambda}$.

Malicious $\mathcal{P}_S$ and $\mathcal{P}_R$. The proof is identical to the setting of the dishonest $\mathcal{P}_S$ above, as $\mathcal{P}_R$ does not do anything in the protocol and the output of $\mathcal{P}_A$ is identical to that of an honest $\mathcal{P}_R$.

E Proof of Theorem 1 (Single receiver Commitments)

For the proof, we will provide three simulators: $\mathcal{S}_{\text{HOT-1-HonestSender}}$ against a dishonest receiver, $\mathcal{S}_{\text{HOT-1-HonestReceiver}}$ against a dishonest sender and $\mathcal{S}_{\text{HOT-1-BothCorrupted}}$ where both the sender and receiver can be dishonest. This is sufficient, as corruptions are static.

Proof.

$\mathcal{S}_{\text{HOT-1-HonestSender}}$. The simulator simulates $\mathcal{F}_{\text{WVOLE}}$ and has access to $\mathcal{F}_{\text{GPRO}}, \mathcal{F}_{\text{GGB}}$. Whenever $\mathcal{P}_R$ broadcasts $(\text{Abort}, \text{sid})$, $\mathcal{S}_{\text{HOT-1-HonestSender}}$ sends $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}$.

Init Upon receiving message $(\text{Init}, \text{sid}, \text{mc})$ from $\mathcal{F}_{\text{HOTCom}}$ $\mathcal{S}_{\text{HOT-1-HonestSender}}$ simulates the protocol and all functionalities honestly.

Commit Upon obtaining $(\text{Receipt}, \text{sid}, (\text{cid}_j)_{j \in [k]}, \mathbf{ch})$ choose all $d_j \in \mathbb{F}_p$ uniformly at random and program $\mathcal{F}_{\text{GPRO}}$ to output $\mathbf{ch}$ on input $(\text{Query}, \text{sid}, \overline{\mathbf{ch}} \parallel \text{mp}_{\text{com}})$. If the programming fails, then abort. Otherwise, run the protocol.

Add and Linear Follow the protocol honestly.

Hint Upon receiving $(\text{Hint}, \text{sid}, (\text{cid}_j, x_j)_{j \in [k]}, 0)$ from $\mathcal{F}_{\text{HOTCom}}$, broadcast $(\text{cid}_j, x_j)_{j \in [k]}$ and otherwise follow the protocol. Upon receiving $(\text{Hint}, \text{sid}, (\text{cid}_j, x_j)_{j \in [k]}, 1)$ from $\mathcal{F}_{\text{HOTCom}}$, sample uniformly random $c_j \in \{0, 1\}^{2\lambda}$. Then broadcast $(\text{cid}_j, c_j)_{j \in [k]}$ and send $(\text{cid}_j, x_j)_{j \in [k]}$ to $\mathcal{P}_R$.

Delayed Hint Upon receiving $(\text{DelHint}, \text{sid}, (\text{cid}_j)_{j \in [k]})$ from $\mathcal{F}_{\text{HOTCom}}$, sample uniformly random $c_j \in \{0, 1\}^{2\lambda}$. Then broadcast $(\text{cid}_j, c_j)_{j \in [k]}$.

Open Upon receiving the first $(\text{Open}, \text{sid})$ from $\mathcal{F}_{\text{HOTCom}}$, the simulator chooses Δ_1 uniformly at random. It computes $\mathbf{S}, \mathbf{D}$ according to the protocol, computes Γ as the verifier would compute it in the protocol and sets all g_j for opened messages such that $g_j = f_j^{(0)} + f_j^{(1)}(\mathbf{s}'_1, \dots, \mathbf{s}'_\ell) - m_j \Delta_1$, which is always uniquely possible by construction. Moreover, it samples r_j for B_{priv} uniformly at random, which now fully defines $A_{\text{pub}}, A_{\text{priv}}, A_{\text{delay}}, B_{\text{priv}}$. The simulator reprograms $\mathcal{F}_{\text{GPRO}}$ to output Δ_1 on input $\text{ch} \| A_{\text{pub}} \| A_{\text{priv}} \| A_{\text{delay}}$. If the adversary sent this input to $\mathcal{F}_{\text{GPRO}}$ before, though, then the simulator aborts. The simulator also reprograms the random oracle to output c_j on input $m_j \| g_j \| r_j$ for each c_j generated during **Private Hint**. Again, if the adversary queried $m_j \| g_j \| r_j$ to $\mathcal{F}_{\text{GPRO}}$ before then abort. Finally, it broadcasts $\mathbf{S}, \mathbf{D}, A_{\text{pub}}, A_{\text{delay}}$ and sends B_{priv} to $\mathcal{P}_R$.

Delayed Open Upon receiving the second $(\text{Open}, \text{sid}, (\text{cid}_j, m_j)_{j \in [k]})$ from $\mathcal{F}_{\text{HOTCom}}$, the simulator computes the remaining $g_j = f_j^{(0)} + f_j^{(1)}(\mathbf{s}'_1, \dots, \mathbf{s}'_\ell) - m_j \Delta_1$. It then samples r_j for $A_{\text{delay,open}}$ uniformly at random, and reprograms the Random Oracle to output c_j on input $m_j \| g_j \| r_j$ for each c_j generated during **Delayed Hint**. If the adversary queried $m_j \| g_j \| r_j$ to $\mathcal{F}_{\text{GPRO}}$ before then abort. Finally, it broadcasts $A_{\text{delay,open}}$.

Verify, Audit and Challenges Simulate $\mathcal{F}_{\text{WVole}}$ honestly.

We now argue indistinguishability of transcripts via a series of hybrids.

Let $\mathcal{H}_0$ be the distribution of all the messages towards the adversary and honest outputs as in the protocol $\Pi_{\text{HOTCommit}}^{1-1}$.

Let $\mathcal{H}_1$ be defined like $\mathcal{H}_0$, except that we now sample the commitments c_j uniformly at random as in the simulation and only later program the random oracle so that the consistency checks pass. The commitments c_j obtained during **Hint** and **Delayed Hint** are outputs of $\mathcal{F}_{\text{GPRO}}$ in the protocol. As $\mathcal{F}_{\text{GPRO}}$ is a random oracle, the c_j are indeed indistinguishably distributed given that they are computed in the protocol using a uniformly random string $r_j \in \{0, 1\}^\lambda$ only revealed later to $\mathcal{P}_R$. Note that the simulator aborts if it fails to program the Random Oracle on inputs that lead to c_j . But to query on an input that leads to c_j , $\mathcal{A}$ had to guess a string r_j of length λ bits correctly. As the adversary makes Q queries to the Random Oracle, his success probability in guessing r_j (and thus aborting the protocol and distinguishing between $\mathcal{H}_0$ and $\mathcal{H}_1$) is at most $Q/2^\lambda$.

Let $\mathcal{H}_2$ be defined like $\mathcal{H}_1$, except that we choose d_j obtained in **Commit** uniformly at random as in the simulation, but then change the g_j according to m_j . We also now program $\mathcal{F}_{\text{GpRO}}$ to output challenge values h accordingly. This is perfectly indistinguishable, as m_j is blinded in the protocol by an element from $\mathbf{U}_1$ that is chosen uniformly at random by $\mathcal{F}_{\text{WVOLE}}$, but may fail as we program the random oracle. Note that the adversary never learns anything about $\mathbf{U}_1$ throughout the protocol except a sum involving $\mathbf{U}_2$ where $\mathbf{U}_2$ is also uniformly random. To address the abort probability, note that the adversary makes at most Q queries to $\mathcal{F}_{\text{GpRO}}$, where during each query it might have guessed d_j correctly, so the error probability is Q^2/p .

Let $\mathcal{H}_3$ be defined like $\mathcal{H}_2$, except that we now also choose Δ_1 uniformly at random as in the simulation and have the simulated g_j fulfill the equation like the real g_j do in the protocol. Note that $\mathbf{S}, \mathbf{D}$ are computed as in the protocol and therefore have the same distribution. Also, this sampling fixes $A_{\text{pub}}, A_{\text{delay}}$ and B_{priv} to have the same distribution in the protocol and in the simulation. $\mathcal{H}_3$ is identical to the distribution of the simulator.

Note that the simulator again aborts if it fails to program the Random Oracle on the input that leads to Δ_1 . If the last **Hint** before the call **Open** had $i = 1$, then due to the random choice of c_j sent to $\mathcal{P}_R$ the abort probability trivially follows by observing that the revealed r_j is chosen uniformly at random during the simulation. If the last message was **Hint** with $i = 0$, then note that we send A_{pub} during **Open** that contains g_j which is chosen uniformly at random, leading to the same abort probability of most $Q/2^\lambda$. Finally, if the last message was a **Delayed Hint** then we could have chosen Δ_1 at that point where also a uniformly random $c_j \in \{0, 1\}^{2^\lambda}$ was generated by the simulator and that is part of the query that generates Δ_1 .

$\mathcal{S}_{\text{HOT-1-HonestReceiver}}$. The simulator simulates $\mathcal{F}_{\text{WVOLE}}$ and has access to $\mathcal{F}_{\text{GpRO}}$ and $\mathcal{F}_{\text{GBB}}$. Whenever $\mathcal{P}_S$ broadcasts (Abort, sid), $\mathcal{S}_{\text{HOT-1-HonestReceiver}}$ sends (Abort, sid) to $\mathcal{F}_{\text{HOTCom}}$. Note that at any point of the protocol, the adversary can send (Query, sid, $\overline{\mathbf{ch}}$ | ...) to $\mathcal{F}_{\text{GpRO}}$ whose response we compute by looking up which query on $\mathcal{F}_{\text{WVOLE}}$ generated the output $\overline{\mathbf{ch}}$ and then translating the other inputs of the RO query into a **Challenge Test** query at $\mathcal{F}_{\text{HOTCom}}$. The output of this query will become the response of $\mathcal{F}_{\text{GpRO}}$, i.e. we program the random oracle on this point. The programming only fails if the adversary ever predicts $\overline{\mathbf{ch}}$ from $\mathcal{F}_{\text{WVOLE}}$ and makes a random oracle query that we respond to without querying $\mathcal{F}_{\text{HOTCom}}$, which only happens with probability at most $Q^2/2^{2\lambda}$ by Proposition 1. We now proceed with the protocol simulation:

Init $\mathcal{S}_{\text{HOT-1-HonestReceiver}}$ simulates the protocol and $\mathcal{F}_{\text{WVOLE}}$ honestly. If $\mathcal{P}_S$ sent the correct (Init, sid, 2ℓ) to $\mathcal{F}_{\text{WVOLE}}$ then calculate mc and send (Init, sid, mc) in its name to $\mathcal{F}_{\text{HOTCom}}$. Store the matrices $\mathbf{U}, \mathbf{V}$ that $\mathcal{P}_S$ obtained from (or supplied to) $\mathcal{F}_{\text{WVOLE}}$ and the challenge $\overline{\mathbf{ch}}$ generated by it.

Commit Upon obtaining all d_j via broadcast from $\mathcal{P}_S$, recompute $m_j = d_j + \mathbf{U}[x + 1, y + 1]$ for x, y as computed in the protocol based on the stored

matrix $\mathbf{U}$ for $\overline{\mathbf{ch}}$. Then send $(\text{Commit}, \text{sid}, (\text{cid}_{i_j}, m_j)_{j \in [k]})$ to $\mathcal{F}_{\text{HOTCom}}$ and otherwise follow the protocol.

Add and Linear Follow the protocol honestly.

Hint Upon receiving the broadcast $(\text{cid}_j, x_j)_{j \in [k]}$ from $\mathcal{P}_S$ send $(\text{Hint}, \text{sid}, (\text{cid}_j, x_j)_{j \in [k]}, 0)$ to $\mathcal{F}_{\text{HOTCom}}$ and otherwise follow the protocol. If any x_j is different from the m_j $\mathcal{P}_S$ should have sent based on the values extracted during **Commit** then set the flag $\text{cheated1} \leftarrow \top$.

Upon receiving the broadcast $(\text{cid}_j, c_j)_{j \in [k]}$ and the private message (cid_j, x_j) from $\mathcal{P}_S$ send $(\text{Hint}, \text{sid}, (\text{cid}_j, x_j)_{j \in [k]}, 1)$ to $\mathcal{F}_{\text{HOTCom}}$ and otherwise follow the protocol. If any c_j was not the output of $\mathcal{F}_{\text{GpRO}}$ on input x_j or if any x_j preimage is different from the m_j $\mathcal{P}_S$ should have sent based on the values extracted during **Commit** then set the flag $\text{cheated1} \leftarrow \top$.

Delayed Hint Upon receiving the broadcast $(\text{cid}_j, c_j)_{j \in [k]}$ from $\mathcal{P}_S$, find the preimage x_j for each c_j under the random oracle. If any c_j does not have a preimage under $\mathcal{F}_{\text{GpRO}}$ or there are multiple preimages then let x_j be a random value in $\mathbb{F}_p$. If $x_j \neq m_j$ where m_j is the value of the commitment cid_j based on what was extracted during **Commit** then set $\text{cheated2} \leftarrow \top$. Send $(\text{DelHint}, \text{sid}, (\text{cid}_j, x_j)_{j \in [k]})$ to $\mathcal{F}_{\text{HOTCom}}$ and otherwise follow the protocol.

Open Upon receiving the broadcast $A_{\text{pub}}, A_{\text{delay}}, \mathbf{S}^*, \mathbf{D}^*$ as well as the private B_{priv} do the following:

1. Run the same steps as $\mathcal{P}_R$ runs in the protocol for **Open**. If any of the c_j of **Hint** did not have a preimage before the call or is incorrect, then set $\text{cheated1} \leftarrow \top$. If $\text{cheated1} = \top$ or there is any $\mathcal{F}_{\text{GpRO}}$ output queried by $\mathcal{A}$ that has more than two inputs (a collision) then send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}$.
2. Perform the checks in **Verify**. If any of them fails then send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}$. Otherwise send $(\text{Open}, \text{sid})$.

Delayed Open Upon receiving the broadcast $A_{\text{delay, open}}$ do the following:

1. Run the same steps as $\mathcal{P}_R$ runs in the protocol for **Delayed Open**. If any of the c_j for **Delayed Open** did not have a preimage before the call or is incorrect, then set $\text{cheated2} \leftarrow \top$. If $\text{cheated2} = \top$ or there is any $\mathcal{F}_{\text{GpRO}}$ output queried by $\mathcal{A}$ that has more than two inputs (a collision) then send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}$.
2. Perform the checks in **Verify**. If any of them fails then send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}$. Otherwise send $(\text{Open}, \text{sid})$.

Verify, Audit and Challenges Send the command to $\mathcal{F}_{\text{HOTCom}}$ and output what it outputs.

The simulation and the protocol can only be distinguished if $\mathcal{S}_{\text{HOT-1-HonestReceiver}}$ aborts when $\mathcal{P}_R$ would not in the real protocol. This happens when preimages of $\bar{\Delta}_1$ or some c_j are undefined but nevertheless opened correctly later, or if the adversary found any collision on a random oracle output, or if any x_j revealed during **Hint** or **Delayed Hint** was different from the previously extracted m_j . For the random oracle collisions or preimages that did not exist, we can bound the probability of this event by Proposition 1 as $Q^2/2^{2\lambda}$ if Q random oracle calls are made by $\mathcal{A}$. Now, assume that the adversary was able to open $x_j \neq m_j$ and make the protocol **Verify** be correct, while $\mathcal{S}_{\text{HOT-1-HonestReceiver}}$ aborts.

For the sake of argument, assume that $\mathbf{S}^* = \mathbf{S}$ that $\mathcal{P}_S$ should have sent in the protocol. Since all the g_j are fixed before Δ_1 is uniformly chosen by the random oracle (and since $\mathbf{S}$ is correct), the chance that Δ_1 is chosen such that the check passes is $1/p$ by Lemma 1. However, note that $\mathcal{A}$ may perform up to Q queries to $\mathcal{F}_{\text{GPRO}}$ with different choices of g_j before outputting any response to $\mathcal{S}_{\text{HOT-1-HonestReceiver}}$. Thus, the probability that it finds an accepting Δ_1 during these calls is Q/p by a union bound.

However, it could be that $\mathbf{S}^* \neq \mathbf{S}$ or $\mathbf{D}^* \neq \mathbf{D}$. Let $\mathbf{S}_E = \mathbf{S}^* - \mathbf{S}$ and $\mathbf{D}_E = \mathbf{D}^* - \mathbf{D}$. Since $\mathbf{D} = \mathbf{Q}_2 + \mathbf{Q}_1 \Delta_1 - \mathcal{C}(\mathbf{S}) \cdot \text{diag}(\Delta_0)$ and $\mathbf{D}^* = \mathbf{Q}_2 + \mathbf{Q}_1 \Delta_1 - \mathcal{C}(\mathbf{S}^*) \cdot \text{diag}(\Delta_0)$ we have that $\mathbf{D}_E^* = -\mathcal{C}(\mathbf{S}_E^*) \cdot \text{diag}(\Delta_0)$. Since $\mathbf{S}_E^* \neq 0$ by assumption, $\mathcal{C}(\mathbf{S}_E^*)$ is non-zero in at least d_C positions, in which $\mathcal{A}$ would have had to predict the entries in Δ_0 to fulfill the equation. Δ_0 is chosen as an unpredictable output of **Delta Test** or **Init Done** (whose input m uniquely fixes the transcript) to a query $\mathcal{A}$ might have made and that includes $\mathbf{S}^*, \mathbf{D}^*$, and for each query the chance of having guessed Δ_0 in the right coordinates correctly is $1/N^{d_C}$ for each attempt. Overall, the adversary can cheat in this check with probability Q/N^{d_C} by a union bound over all queries made to **Delta Test**.

Finally, since **Audit** outputs what **Verify** outputs, it will also only be incorrect when the real protocol succeeds but the simulation aborts. This has been analyzed above. **Challenges** is computed identically as how they are generated during **Commit** making them consistent between protocol and simulation.

$\mathcal{S}_{\text{HOT-1-BothCorrupted}}$. The simulator can trivially be constructed from $\mathcal{S}_{\text{HOT-1-HonestReceiver}}$. It again runs $\mathcal{F}_{\text{WVOLUME}}$ towards both the dishonest sender and receiver and the same techniques for input extraction and consistency checking still apply as communication only happens from $\mathcal{P}_S$ to $\mathcal{P}_R$ via $\mathcal{F}_{\text{GBB}}$, privately or through $\mathcal{F}_{\text{WVOLUME}}$. The simulator can extract all openings except **Hint** with $i = 1$ but since the output of it is not verifiable by $\mathcal{P}_A$ anyways, this does not pose a problem.

The security argument follows the same way as above, and we must only focus on **Audit**. However, the verification algorithm of **Audit** is identical to **Verify** (except that it skips private outputs to $\mathcal{P}_R$). So the argument concerning the unpredictability of Δ_0 still applies: $\mathcal{P}_S$ and $\mathcal{P}_R$ only learn Δ_0 after all other messages defining $A_{\text{pub}}, A_{\text{priv}}, A_{\text{delay}}, \Delta_1, \mathbf{S}, \mathbf{D}$ have been broadcast and $\mathbf{U}, \mathbf{V}$ have been fixed, which makes the transcript verifiable by $\mathcal{P}_A$.

F Multi-Receiver commitment protocol and proof

We describe the protocol $\Pi_{\text{HOTCommit}}^{1-n}$ in Figures 20 and 21.

F.1 Proof of Theorem 2

We prove Theorem 2 by providing two simulators. The simulator $\mathcal{S}_{\text{HOTCorruptSender}}$ shown in Figure 22 and Figure 23 assumes that the sender $\mathcal{P}_S$ and a subset of the receivers $\mathcal{P}_R$ are corrupt, while the simulator in $\mathcal{S}_{\text{HOTHonestSender}}$ shown in Figure 24 assumes the sender is honest and some or all $\mathcal{P} \in \mathcal{P}_R$ are corrupt.

Proof.

Corrupt sender. When the sender $\mathcal{P}_S$ is corrupt the adversary can cheat in two ways. Either by using hints that are different to the commitment values or by sending inconsistent values during **Commit** to the different instances of $\mathcal{F}_{\text{HOTCom}}^i$.

During **Initialize** $\mathcal{S}$ will start local instances of $\mathcal{F}_{\text{HOTCom}}^i$ to simulate communication with corrupt parties and send init to $\mathcal{F}_{\text{HOTCom}}$ with which honest parties will communicate. In **Commit** the sender will send one message to each instance of $\mathcal{F}_{\text{HOTCom}}^i$ for every $i \in [|\mathcal{P}_R|]$. If $\mathcal{P}_S$ is cheating by sending inconsistent values $\mathcal{S}$ will set $\text{fl}_{\text{cheated}} = \top$. Although $\mathcal{S}$ detects cheating during **Commit**, it only aborts the execution based on that during **Open**, in order to maintain consistency with $\Pi_{\text{HOTCommit}}^{1-n}$. The simulator will take the values sent to $\mathcal{F}_{\text{HOTCom}}^1$ as reference for the remainder of the simulation (assuming that these are the extracted inputs) as we are only interested if $\mathcal{P}_S$ cheated or not. When commit in each $\mathcal{F}_{\text{HOTCom}}^i$ instance is completed it will output a random value $\overline{\text{ch}}^i$. The value is sampled uniformly at random for each commitment and thus unpredictable. In $\Pi_{\text{HOTCommit}}^{1-n}$ these values are then combined to ch by querying $\mathcal{F}_{\text{GpRO}}$ on message $\overline{\text{ch}}^1 \parallel \dots$. To be consistent with $\mathcal{F}_{\text{HOTCom}}$ the simulator has to program $\mathcal{F}_{\text{GpRO}}$ such that a query with $\overline{\text{ch}}^1 \parallel \dots$ will output the same as $\mathcal{F}_{\text{HOTCom}}$. However, an adversary may have queried this input before the $\overline{\text{ch}}^i$ are output by $\mathcal{F}_{\text{HOTCom}}^i$ and then programming is not possible, making the simulator abort and the simulation distinguishable. The probability of this event is $\frac{Q^2}{2^{2\lambda}}$ as given by Proposition 1, because each $\overline{\text{ch}}^i$ comes from $\{0, 1\}^{2\lambda}$.

When **Hint** is executed $\mathcal{P}_S$ will send the value to $\mathcal{F}_{\text{GBB}}$ and all $\mathcal{F}_{\text{HOTCom}}^i$ instances. $\mathcal{S}$ will compare the value that is sent to $\mathcal{F}_{\text{GBB}}$ with the value the $\mathcal{F}_{\text{HOTCom}}^i$ instances output and abort if these do not match. The value that was sent via $\mathcal{F}_{\text{GBB}}$ is given to $\mathcal{F}_{\text{HOTCom}}$.

For **Delayed Hint** we extract the values from the commitment com that the adversary broadcasts. Delayed values are opened in **Delayed Open** when the commitment is opened and the opening verified. Here the adversary can cheat by finding another preimage to $\text{com} \in \{0, 1\}^{2\lambda}$, which it can do in Q queries to

In the protocol, a sender $\mathcal{P}_S$ interacts with a set of receivers $\mathcal{P}_R$ and auditor $\mathcal{P}_A$. All parties have access to an instance of $\mathcal{F}_{\text{GPRO}}$ and $\mathcal{F}_{\text{GBB}}$. $\mathcal{F}_{\text{HOTCom}}$ is used in the $|\mathcal{P}_R| = 1$ setting. The $\mathcal{F}_{\text{HOTCom}}$ instance between $\mathcal{P}_S$ and a party $\mathcal{P}_i \in \mathcal{P}_R$ is denoted $\mathcal{F}_{\text{HOTCom}}^i$. Let $\mathcal{T} = \{0, 1\}^{2\lambda}$ for $\mathcal{F}_{\text{HOTCom}}$.

Init On input $(\text{Init}, \text{sid}, \text{mc})$ to all parties, initialize one instance of $\mathcal{F}_{\text{HOTCom}}$ for every party $\mathcal{P}_i \in \mathcal{P}_R$.

1. Send $(\text{Init}, \text{sid}, \text{mc} + 1)$ to each instance of $\mathcal{F}_{\text{HOTCom}}$.
2. $\mathcal{P}_S$ samples $r \leftarrow \mathbb{F}_p$ at random.
3. Send $(\text{Commit}, \text{sid}, (\text{cid}_{\text{mc}+1}, r))$ to each instance $\mathcal{F}_{\text{HOTCom}}^i$ for $i \in [|\mathcal{P}_R|]$.

Commit On input $(\text{Commit}, \text{sid}, (\text{cid}_j, m_j)_{j \in [k]})$, $\mathcal{P}_S$ sends $(\text{Commit}, \text{sid}, (\text{cid}_j, m_j)_{j \in [k]})$ to all instances of $\mathcal{F}_{\text{HOTCom}}^i$ for $i \in [|\mathcal{P}_R|]$. It then obtains $\overline{\text{ch}}^i$ that was returned by each $\mathcal{F}_{\text{HOTCom}}^i$ instance.

1. Each party $\mathcal{P}_j$ obtains all $\overline{\text{ch}}^i$ that were returned by each $\mathcal{F}_{\text{HOTCom}}^i$ instance
2. Every party sends $(\text{Query}, \text{sid}, \overline{\text{ch}}^1 \parallel \dots \parallel \overline{\text{ch}}^{|\mathcal{P}_R|}, \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ and outputs the returned value ch .

Hint On input $(\text{Hint}, \text{sid}, (\text{cid}_j)_{j \in [k]}, i)$ to $\mathcal{P}_S$ where each (cid_j, m_j) was sent to **Commit** before: If $i \neq 0$ then $\mathcal{P}_S$ sends $(\text{Hint}, \text{sid}, (\text{cid}_j)_{j \in [k]}, 1)$ to $\mathcal{F}_{\text{HOTCom}}^i$. If $i = 0$ then:

1. $\mathcal{P}_S$ sends $(\text{Broadcast}, (\text{cid}_j, m_j)_{j \in [k]})$ to $\mathcal{F}_{\text{GBB}}$.
2. Additionally, $\mathcal{P}_S$ sends $(\text{Hint}, \text{sid}, (\text{cid}_j)_{j \in [k]}, 0)$ to all instances of $\mathcal{F}_{\text{HOTCom}}^i$ for $i \in [|\mathcal{P}_R|]$.
3. Each receiver $\mathcal{P}_i$ compares $(\text{cid}_j, m_j)_{j \in [k]}$ that was sent to $\mathcal{F}_{\text{GBB}}$ with the value $(m'_j)_{j \in [k]}$ that is the output of **Hint**. If $m_j \neq m'_j$ then broadcast $(\text{Abort}, \text{sid})$ and abort the protocol.

Delayed Hint On input $(\text{DelHint}, \text{sid}, (\text{cid}_j)_{j \in [k]})$, $\mathcal{P}_S$ does the following:

1. Sample $r \leftarrow \{0, 1\}^\lambda$ uniformly at random.
2. Send $(\text{Query}, \text{sid}, (\text{cid}_j, m_j)_{j \in [k]} \parallel r, \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ and obtain com .
3. Broadcast $(\text{Broadcast}, \text{com})$ and send $(\text{DelHint}, \text{sid}, (\text{cid}_j)_{j \in [k]})$ to all $\mathcal{F}_{\text{HOTCom}}^i$ for $i \in [|\mathcal{P}_R|]$.

Open On first input $(\text{Open}, \text{sid})$, $\mathcal{P}_S$ does the following:

1. Run the procedure **Consistency Check**.
2. $\mathcal{P}_S$ sends $(\text{Open}, \text{sid})$ to every instance of $\mathcal{F}_{\text{HOTCom}}^i$ for $i \in [|\mathcal{P}_R|]$.

Every $\mathcal{P}_i \in \mathcal{P}_R$ sends $(\text{Verify}, \text{sid}, \text{cid}_h, h)$ to $\mathcal{F}_{\text{HOTCom}}^i$. If the check passes then it continues, otherwise it broadcasts $(\text{Broadcast}, \text{Abort})$ and aborts.

Delayed Open On second input $(\text{Open}, \text{sid})$, $\mathcal{P}_S$ does the following:

1. Send $(\text{Broadcast}, (\text{cid}_j, m_j)_{j \in [k]}, r)$ for the cid_j sent in **Delayed Hint**.
2. $\mathcal{P}_S$ sends $(\text{Open}, \text{sid})$ to every instance of $\mathcal{F}_{\text{HOTCom}}^i$ for $i \in [|\mathcal{P}_R|]$.

Thereupon, each party in $\mathcal{P}_R$ sends $(\text{Query}, \text{sid}, (\text{cid}_j, m_j)_{j \in [k]} \parallel r, \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ and compares the result with com . If it is not identical, then broadcast $(\text{Broadcast}, \text{abort})$ and abort.

Fig. 20: Protocol $\Pi_{\text{HOTCommit}}^{1-n}$ implementing multi-receiver one-time commitments.

$\Pi_{\text{HOTCommit}}^{1-n}$ (Part 2)

Add On input $(\text{Add}, \text{sid}, (\text{cid}_j)_{j \in [3]})$, forward the message to every instance $\mathcal{F}_{\text{HOTCom}}^i$ for $i \in [\mathcal{P}_R]$.

Linear On input $(\text{Linear}, \text{sid}, \text{cid}_1, \text{cid}_2, \alpha, \delta)$, forward the message to every instance $\mathcal{F}_{\text{HOTCom}}^i$ for $i \in [\mathcal{P}_R]$.

Verify On input $(\text{Verify}, \text{sid}, L)$ to any $\mathcal{P}_i \in \mathcal{P}_R$, send $(\text{Verify}, \text{sid}, L)$ to $\mathcal{F}_{\text{HOTCom}}^i$ and output what it outputs.

Audit On input $(\text{Audit}, \text{sid}, (\text{cid}_j, f_j)_{j \in [k]})$ to $\mathcal{P}_A$ where f_j are $\mathbb{F}_p$ -linear functions created by **Add** and **Linear**:

1. Send $(\text{Challenges}, \text{sid})$ to each $\mathcal{F}_{\text{HOTCom}}^i$. Let $\overline{\text{ch}}^i$ be the last challenge on each returned list. Then send $(\text{Query}, \text{sid}, \overline{\text{ch}}^1 \parallel \dots \parallel \overline{\text{ch}}^{|\mathcal{P}_R|}, \mathcal{T})$ to $\mathcal{F}_{\text{GpRO}}$ and obtain the response ch .
2. Send $(\text{Query}, \text{sid}, \text{ch}, \mathbb{F}_p^{\text{mc}})$ to get $c_1, \dots, c_{\text{mc}}$.
3. Send $(\text{Audit}, \text{sid}, \text{cid}_j, \sum_{j \in [k]} c_j \cdot \text{cid}_j + \text{cid}_{\text{mc}+1})$ to every instance $\mathcal{F}_{\text{HOTCom}}^i$. If they return different messages $(\text{Audit}, \text{sid}, L)$ then return $(\text{AuditFailed}, \text{sid})$.
4. Send $(\text{Audit}, \text{sid}, (\text{cid}_j, f_j)_{j \in [k]})$ to every $\mathcal{F}_{\text{HOTCom}}^i$ instance for $i \in [\mathcal{P}_R]$. If all output the same $(\text{Audit}, \text{sid}, L)$ then output $(\text{Audit}, \text{sid}, L)$, otherwise $(\text{AuditFailed}, \text{sid})$.

Challenges On input $(\text{Challenges}, \text{sid})$ to $\mathcal{P}_A$:

1. For each $i \in \mathcal{P}_R$ send $(\text{Challenges}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}^i$ and obtain L_i .
2. Compute each ch_i as any receiver computes it in **Commit** and output all such values.

Procedure Consistency Check Subroutine to verify that commitments and their openings are consistent.

1. Let ch be the most recent challenge. Send $(\text{Query}, \text{sid}, \text{ch}, \mathbb{F}_p^{\text{mc}})$ to $\mathcal{F}_{\text{GpRO}}$ and denote the output as $c_1, \dots, c_{\text{mc}}$.
2. $\mathcal{P}_S$ computes $cc = r + \sum_{j=1}^{\text{mc}} c_j \cdot m_j$ and sends $(\text{Broadcast}, cc)$ to $\mathcal{F}_{\text{GpB}}$.
3. Compute the function on the previous steps on every $\mathcal{F}_{\text{HOTCom}}^i$, using cid_r and $\text{cid}_r, \dots, \text{cid}_{\text{mc}}$. Denote the resulting commitment ID as cid_{cc} .
4. $\mathcal{P}_S$ sends $(\text{Hint}, \text{sid}, \text{cid}_{cc}, 0)$ to every instance of $\mathcal{F}_{\text{HOTCom}}^i$.
5. Each $\mathcal{P}_i \in \mathcal{P}_R$ verifies that m_{cc} received from $\mathcal{F}_{\text{HOTCom}}^i$ equals cc . If $m_{cc} \neq cc$ send $(\text{Broadcast}, \text{Abort})$ to $\mathcal{F}_{\text{GpB}}$ and output 0.

Fig. 21: Protocol $\Pi_{\text{HOTCommit}}^{1-n}$ implementing multi-receiver one-time commitments.

RO with probability at most $\frac{Q^2}{2^{2\lambda}}$. For **Add**, **Linear** the simulation is straight forward as the simulator follows the steps of the protocol.

Open the parties perform a consistency check that uses outputs from $\mathcal{F}_{\text{GpRO}}$ and the universal hash function defined in Proposition 2. Note that the challenges $c_1, \dots, c_{\text{mc}}$ used in the **Consistency Check** are chosen based on all $\overline{\text{ch}}^i$, which are only generated once the commitments (which may be different for each $\mathcal{F}_{\text{HOTCom}}^i$) have been sent. The probability that $\mathcal{A}$ has made the query to $\mathcal{F}_{\text{GpRO}}$

that generates the challenges $c_1, \dots, c_{mc}$ before the $\overline{\mathbf{ch}}^i$ are chosen is bounded by $\frac{Q^2}{2^{2\lambda}}$ as $\overline{\mathbf{ch}}^i \in \{0, 1\}^{2\lambda}$. During the consistency check $\mathcal{P}_S$ will open the value m_{cc} which, if commitments are consistent throughout the $\mathcal{F}_{\text{HOTCom}}^i$ instances, is equal to the value h that was broadcast. The simulator will always abort if the commitments were inconsistent, while the protocol continues if $\mathcal{A}$ found a collision for the universal hash function. The universal hash function is fixed by the random challenge, and the probability of a collision is $\frac{1}{p}$ for each query. This gives a $\frac{Q}{p}$ distinguishing advantage for $\mathcal{Z}$ on this event, or $\frac{Q}{p} + \frac{Q^2}{2^{2\lambda}}$ in total.

Sender and all receivers corrupt. When all parties including the sender are dishonest the simulator will follow the steps of the *dishonest sender simulator* to extract the inputs of the sender. In the simulation, the simulator will always abort (and not extract inputs) if $\mathcal{A}$ attempts to cheat. In the protocol, the parties will output (VerifyFailed, sid) if $\mathcal{A}$ cheats by sending inconsistent values and the consistency check fails, which reduces to the same distinguishing advantage as mentioned above.

Honest sender. When the sender is honest and a subset or all $\mathcal{P}_i \in \mathcal{P}_R$ are corrupt $\mathcal{S}$ will simulate the outputs of every $\mathcal{F}_{\text{HOTCom}}^i$ instance. During **Init** $\mathcal{S}$ will follow the steps of $\Pi_{\text{HOTCommit}}^{1-n}$ such that there exists an instance of $\mathcal{F}_{\text{HOTCom}}^i$ for every $\mathcal{P}_i \in \mathcal{C}$. When $\mathcal{S}$ receives (Receipt, sid, $(\text{cid}_j)_{j \in [k]}, \mathbf{ch}$) they send a commitment message to each $\mathcal{F}_{\text{HOTCom}}^i$ using 0 as a dummy value. This is indistinguishable for any $\mathcal{P}_i \in \mathcal{C}$ as their instance will only output the commitment ID. To replicate the challenge value $\mathbf{ch}$ (which will be the output of $\mathcal{F}_{\text{HOTCom}}$) the simulator chooses random $\overline{\mathbf{ch}}^i$ values to $\mathcal{F}_{\text{GBB}}$ and programs $\mathcal{F}_{\text{GPRO}}$ such that it outputs $\mathbf{ch}$ when queried on $\overline{\mathbf{ch}}^1 \parallel \dots$. Since each $\overline{\mathbf{ch}}^i$ is in $\{0, 1\}^{2\lambda}$ and the adversary makes at most Q queries to $\mathcal{F}_{\text{GPRO}}$, the programming can fail with probability at most $Q^2/2^{2\lambda}$. For **Add**, **Linear**, **Verify** and **Audit** the simulation is straight forward as they are non-interactive. $\mathcal{S}$ simply lets every $\mathcal{F}_{\text{HOTCom}}^i$ instance output what was sent by $\mathcal{F}_{\text{HOTCom}}$. For **Hint** $\mathcal{S}$ has to broadcast the value received from $\mathcal{F}_{\text{HOTCom}}$ first. For **Delayed Hint** it broadcasts a random commitment. When **Open** is executed $\mathcal{S}$ will first perform the consistency check and open cc to the value sent to $\mathcal{F}_{\text{GBB}}$. When the sender is honest the functionality and protocol are not distinguishable by any environment $\mathcal{Z}$. The value cc is indistinguishable from the protocol because the chosen universal hash function is hiding as per Definition 2. During **Delayed Open** the simulator equivocates the simulated commitment to the real openings L by programming the random oracle on a random input. During Q queries to the random oracle before programming (thus leading to an abort), the adversary could have guessed r with probability at most $Q/2^\lambda$.

$S_{\text{HOTComCorruptSender}}$ (Part 1)

$\mathcal{H}$ denotes the honest parties and $\mathcal{C}$ the corrupt parties. Every $\mathcal{P}_i$ is connected to one instance of $\mathcal{F}_{\text{HOTCom}}^i$. $\mathcal{S}$ will simulate all $\mathcal{F}_{\text{HOTCom}}^i$ for $\mathcal{P}_i \in \mathcal{H}$. Let m denote the number of honest parties. Note that **Verify**, **Challenges** and **Audit** are non-interactive protocols and need no simulation.

On input $(\text{Query}, \text{sid}, \overline{\text{ch}}^1 \parallel \dots \parallel \mathcal{T})$ to $\mathcal{F}_{\text{GPRO}}$ check if each $\overline{\text{ch}}^i$ was output by $\mathcal{F}_{\text{HOTCom}}^i$. If so, if the same $(\text{cid}_j, m_j)_{j \in [k]}$ were sent to all $\mathcal{F}_{\text{HOTCom}}^i$ instances and if all $\overline{\text{ch}}^i$ were output to the same input by all $\mathcal{F}_{\text{HOTCom}}^i$ then send all $(\text{cid}_j, m_j)_{j \in [k]}$ to **Challenge Test** of $\mathcal{F}_{\text{HOTCom}}$ and let $\mathcal{F}_{\text{GPRO}}$ output what it outputs. Otherwise simulate $\mathcal{F}_{\text{GPRO}}$ honestly.

- Init**
1. Set $\text{fl}_{\text{cheated}} = \perp$.
 2. Start local instance of $\Pi_{\text{HOTCommit}}^{1-n}$, such that there exists one instance of $\mathcal{F}_{\text{HOTCom}}^i$ for every party.
 3. Run all steps of **Init** in $\Pi_{\text{HOTCommit}}^{1-n}$ and simulate honest parties $\mathcal{P} \in \mathcal{H}$.
 4. Send $(\text{Init}, \text{sid}, \text{mc})$ to $\mathcal{F}_{\text{HOTCom}}^i$ if **Init** in $\Pi_{\text{HOTCommit}}^{1-n}$ finishes.

Commit $\mathcal{P}_S$ will send one message for each instance of $\mathcal{F}_{\text{HOTCom}}^i$. Let M^i be the message $(\text{Commit}, \text{sid}, n, (\text{cid}_j, m_j)_{j \in [k]})$ intended for $\mathcal{P}_i \in \mathcal{P}_R$.

1. Wait until $\mathcal{P}_S$ sends all commit messages $M^1, \dots, M^{|\mathcal{P}_R|}$, where M^1 is intended for $\mathcal{F}_{\text{HOTCom}}^i$.
2. For every message $M_{i>1}$ that was sent by $\mathcal{P}_S$ to $\mathcal{F}_{\text{HOTCom}}^i$, compute $\delta = M^1 - M^i$. If $\delta \neq 0$ for any of the messages, set $\text{fl}_{\text{cheated}} = \top$.
3. Send M^1 to $\mathcal{F}_{\text{HOTCom}}^i$ and wait for the challenge output **ch**. Program $\mathcal{F}_{\text{GPRO}}$ such that it will output **h** when queried on $\overline{\text{ch}}^1 \parallel \dots \parallel \overline{\text{ch}}^{|\mathcal{P}_R|}$. If the programming fails, then abort.

Hint On input $(\text{Hint}, \text{sid}, \text{cid}_j)_{j \in [k]}, 0)$ by $\mathcal{P}_S$ to each $\mathcal{F}_{\text{HOTCom}}^i$:

1. Wait for $\mathcal{P}_S$ to broadcast $(\text{cid}_j, x_j)_{j \in [k]}$.
2. Wait for input $(\text{Hint}, \text{sid}, (m_j)_{j \in [k]})$ sent by $\mathcal{A}$ to each $\mathcal{F}_{\text{HOTCom}}^i$ with $\mathcal{P}_i \in \mathcal{H}$.
3. If any $m_j \neq x_j$ send $(\text{abort}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}^i$ and $(\text{Broadcast}, \text{abort})$ to $\mathcal{F}_{\text{GBB}}$.
4. Send $(\text{Hint}, \text{sid}, (\text{cid}_j)_{j \in [k]}, 0)$ and $(\text{Hint}, \text{sid}, (x_j)_{j \in [k]})$ to $\mathcal{F}_{\text{HOTCom}}^i$.

On input $(\text{Hint}, \text{sid}, \text{cid}_j)_{j \in [k]}, 1)$ by $\mathcal{P}_S$ to $\mathcal{F}_{\text{HOTCom}}^i$ where $i \in \mathcal{H}$:

1. Wait for input $(\text{Hint}, \text{sid}, (m_j)_{j \in [k]})$ sent by $\mathcal{A}$ to $\mathcal{F}_{\text{HOTCom}}^i$.
2. Send $(\text{Hint}, \text{sid}, (\text{cid}_j)_{j \in [k]}, i)$ to $\mathcal{F}_{\text{HOTCom}}^i$.
3. Input $(m^j)_{j \in [k]}$ to $\mathcal{F}_{\text{HOTCom}}^i$.

Fig. 22: Simulator of $\mathcal{F}_{\text{HOTCom}}$ with parameter $|\mathcal{P}_R| > 1$ and corrupt sender.

$\mathcal{S}_{\text{HOTCorruptSender}}$ (Part 2)

Delayed Hint On input $(\text{DelHint}, \text{sid}, (\text{cid}_j)_{j \in [k]})$ by $\mathcal{P}_S$ to each $\mathcal{F}_{\text{HOTCom}}^i$:

1. Wait for the broadcast $(\text{Broadcast}, \text{com})$ on $\mathcal{F}_{\text{GBB}}$. Let $(\text{cid}_j, m_j)_{j \in [k]}$ be the preimage under $\mathcal{F}_{\text{GPRO}}$. If any value is undefined, then let m_j be uniformly random.
2. Wait for input $(\text{Hint}, \text{sid}, (m_j)_{j \in [k]})$ sent by $\mathcal{A}$ to each $\mathcal{F}_{\text{HOTCom}}^i$ with $\mathcal{P}_i \in \mathbb{H}$.
3. Send $(\text{DelHint}, \text{sid}, (\text{cid}_j)_{j \in [k]})$ and $(\text{Hint}, \text{sid}, (m_j)_{j \in [k]})$ to $\mathcal{F}_{\text{HOTCom}}$.

Open On first input $(\text{Open}, \text{sid})$ to each $\mathcal{F}_{\text{HOTCom}}^i$ by $\mathcal{P}_S$:

1. Follow the **Consistency Check** procedure in $\Pi_{\text{HOTCommit}}^{1-n}$. If $m_h \neq h$ abort.

Delayed Open On second input $(\text{Open}, \text{sid})$ to each $\mathcal{F}_{\text{HOTCom}}^i$ by $\mathcal{P}_S$: Wait for $\mathcal{P}_S$ to broadcast L', r' and recompute com . If any message in L' does not coincide with the messages committed in com or $r' \neq r$ then send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{HOTCom}}$, otherwise send $(\text{Open}, \text{sid})$.

Add Follow the steps described in $\Pi_{\text{HOTCommit}}^{1-n}$.

Linear Follow the steps described in $\Pi_{\text{HOTCommit}}^{1-n}$.

Verify Follow the steps described in $\Pi_{\text{HOTCommit}}^{1-n}$, if $\text{fl}_{\text{cheated}} = \top$ output $(\text{VerifyFailed}, \text{sid})$.

Fig. 23: Simulator of $\mathcal{F}_{\text{HOTCom}}$ with parameter $|\mathcal{P}_R| > 1$ and corrupt sender.

In the following the sender $\mathcal{P}_S$ is honest and the set of all receivers is denoted as $\mathcal{P}_R$. $\mathbf{H}$ denotes honest receivers and $\mathbf{C}$ the corrupt receivers. If any dishonest party broadcasts (**Broadcast**, *abort*), then send (**Abort**, *sid*) to $\mathcal{F}_{\text{HOTCom}}$. Note that **Verify**, **Challenges** and **Audit** are non-interactive and require no simulation.

Init Upon (**Init**, *sid*, *mc*) from $\mathcal{F}_{\text{HOTCom}}$:

1. Start local instance of $\Pi_{\text{HOTCommit}}^{1-n}$ with which the corrupt parties will communicate. Therefore, each $\mathcal{P} \in \mathbf{C}$ will have access to its own instance of $\mathcal{F}_{\text{HOTCom}}^i$ where the $\mathcal{P}_S$ is simulated by $\mathcal{S}$. The instance has $mc + 1$ commitments.
2. Simulate honest uncorrupted receivers towards the adversary.

Commit Upon message (**Receipt**, *sid*, $(\text{cid}_j)_{j \in [k]}$, **ch**) from $\mathcal{F}_{\text{HOTCom}}$ $\mathcal{S}$ sends (**Commit**, *sid*, $(\text{cid}_j, 0)_{j \in [k]}$) to each $\mathcal{F}_{\text{HOTCom}}^i$ instance. It picks uniformly random $\overline{\text{ch}}^i$ for each $\mathcal{F}_{\text{HOTCom}}^i$ to output, and programs $\mathcal{F}_{\text{GPRO}}$ to output **ch** on input (**Query**, *sid*, $\overline{\text{ch}}^1 \parallel \dots \parallel \mathcal{T}$). If the programming fails, then abort.

Hint On input (**Hint**, *sid*, $(\text{cid}_j, x_j)_{j \in [k]}$, *i*) from $\mathcal{F}_{\text{HOTCom}}$, if $i = 0$:

1. Send (**Broadcast**, $x_j)_{j \in [k]}$) to $\mathcal{F}_{\text{GBB}}$.
2. Make each $\mathcal{F}_{\text{HOTCom}}^i$ output the value x_j for ID cid_j .

If $i \neq 0$, then let the instance of $\mathcal{F}_{\text{HOTCom}}^i$ where $\mathcal{P}_i$ is the receiver output the message received from $\mathcal{F}_{\text{HOTCom}}$. Also respond to verification queries to $\mathcal{F}_{\text{HOTCom}}^i$ with the obtained value.

Delayed Hint Upon message (**DelHint**, *sid*, $(\text{cid}_j)_{j \in [k]}$) from $\mathcal{F}_{\text{HOTCom}}$:

1. Let $com \in \{0, 1\}^{2\lambda}$ be uniformly random. Broadcast (**Broadcast**, *com*) on $\mathcal{F}_{\text{GBB}}$.
2. Let each $\mathcal{F}_{\text{HOTCom}}^i$ instance output (**DelHint**, *sid*, $(\text{cid}_j)_{j \in [k]}$).

Open Upon first message (**Open**, *sid*) from $\mathcal{F}_{\text{HOTCom}}$, run the protocol honestly.

Delayed Open Upon second message (**Open**, *sid*) from $\mathcal{F}_{\text{HOTCom}}$:

1. Obtain (**Open**, *sid*, *L*) from $\mathcal{F}_{\text{HOTCom}}$.
2. Sample a uniformly random $r \in \{0, 1\}^\lambda$ and program $\mathcal{F}_{\text{GPRO}}$ to output *com* on input $L \parallel r$. If the programming fails, then abort.
3. Broadcast (**Broadcast**, $L \parallel r$) on $\mathcal{F}_{\text{GBB}}$ and respond to all verification queries on $\mathcal{F}_{\text{HOTCom}}^i$ for *L* accordingly.

Add Wait for $\mathcal{F}_{\text{HOTCom}}$ to send (**Add**, *sid*, $(\text{cid}_i)_{[3]}$) and send (**Add**, *sid*, $(\text{cid}_j)_{[3]}$) to each instance of $\mathcal{F}_{\text{HOTCom}}^i$.

Linear Wait for $\mathcal{F}_{\text{HOTCom}}$ to send (**Linear**, *sid*, $\text{cid}_1, \text{cid}_2, \alpha, \delta$) and send (**Linear**, *sid*, $\text{cid}_1, \text{cid}_2, \alpha, \delta$) to each instance of $\mathcal{F}_{\text{HOTCom}}^i$.

Verify When **Verify** was send by $\mathcal{P}_i \in \mathbf{C}$ through an instance of $\mathcal{F}_{\text{HOTCom}}^i$, forward the message to $\mathcal{F}_{\text{HOTCom}}$ and let all $\mathcal{F}_{\text{HOTCom}}^i$ instances output what $\mathcal{F}_{\text{HOTCom}}$ outputs.

Fig. 24: Simulator of $\mathcal{F}_{\text{HOTCom}}$ with parameter $|\mathcal{P}_R| > 1$ and honest sender.

G Proof of Theorem 3 (Offline Protocol)

In the proof, we mainly focus on the case with the presence of at least one honest party in $\mathcal{P}$, whose simulator $\mathcal{S}_{\text{OffHonP}}$ we describe in Figure 25 and Figure 26. We then argue how the setting where every party in $\mathcal{P}$ is corrupted follows.

Proof. In the proof, we will focus on the case where there is one random value/triple and therefore drop the index l to simplify notation.

At least one honest party. During **Initialize** the simulator initializes an instance of Π_{Offline} . The simulator uses dummy values for all $\mathcal{P} \in \mathcal{H}$ throughout the protocol. For **Linear**, **Verify** the simulation is straightforward as the parties follow the steps in Π_{Offline} .

For every party $\mathcal{P}_c \in \mathcal{C}$ the simulator stores the share r_c that the party inputs into $\mathcal{F}_{\text{HOTCom}}^c$ during **Input** and forwards them to $\mathcal{F}_{\text{Offline}}$.

During the **Triples** protocol, the adversary $\mathcal{A}$ can add two types of errors to the honest sharings. The first is a multiplicative error, denoted by δ_a^h and δ_b^h , which can differ for every honest party $\mathcal{P}_h$. The second is an additive term Δ_c for all dishonest parties.

During Π_{Offline} δ_a^h, δ_b^h are introduced when one or more of the dishonest parties $\mathcal{P}_c$ send inputs to $\mathcal{F}_{\text{OLE}}$ which differ from their additive shares a^c, b^c that they have committed to in $\mathcal{F}_{\text{HOTCom}}$ before. In the simulation this is modeled by going through each interaction of an honest party $\mathcal{P}_h$ with each party $\mathcal{P}_c \in \mathcal{C}$ and recording the difference between the values $\bar{a}^c, \bar{b}^c$ that the adversary inputs for $\mathcal{F}_{\text{OLE}}$ and the values a^c, b^c that were committed previously. Clearly, the difference $\bar{a}^c - a^c$ is multiplied with the honest b^h when using $\mathcal{F}_{\text{OLE}}$ and both parties obtain additive shares of the result, therefore it impacts the product as the aforementioned error. We therefore add this term to δ_b^h , as well as any error introduced by any other dishonest party when multiplying with b^h . The same reasoning applies when computing δ_a^h . Here, note that from the perspective of the adversary, while the shares of the honest party get multiplied with a value chosen by the attacker, making the error multiplicative makes it independent of the concrete choice of shares of the honest parties i.e. identical in simulation and the real protocol.

Furthermore, the **Triples** protocol will output a challenge value h . For every input to $\mathcal{F}_{\text{HOTCom}}^i$ the adversary will see a uniformly random challenge output. These are then combined, in the protocol via $\mathcal{F}_{\text{GpRO}}$, to a challenge to be output by the protocol. $\mathcal{S}$ will wait until $\mathcal{F}_{\text{HOTCom}}$ sends **ch** as a response to all commitments and will program $\mathcal{F}_{\text{GpRO}}$ to output **ch** when queried on $(\bar{\mathbf{ch}}^1 || \dots || \bar{\mathbf{ch}}^{|\mathcal{P}|})$. Note that this can fail if $\mathcal{F}_{\text{GpRO}}$ was already programmed to a different output, but this can only happen if $\mathcal{A}$ guessed $\bar{\mathbf{ch}}^i \in \{0, 1\}^{2\lambda}$ before it was output by $\mathcal{F}_{\text{HOTCom}}^i$. As the adversary makes at most Q queries to $\mathcal{F}_{\text{GpRO}}$, the distinguishing advantage due to this event is at most $Q^2/2^{2\lambda}$ by Proposition 1.

Simulator $\mathcal{S}_{\text{OffHonP}}$ (Part 1)

In the following honest parties are denoted as $\mathbf{H}$ and corrupt parties as $\mathbf{C}$. The simulator $\mathcal{S}_{\text{OffHonP}}$ maintains a copy of shares from all parties and updates them accordingly.

Upon query $(\text{Query}, \text{sid}, \overline{\mathbf{ch}}^1 \parallel \dots \parallel \mathbb{F}_p^*)$ to $\mathcal{F}_{\text{GPRO}}$ and if all $\overline{\mathbf{ch}}^i$ were generated by $\mathcal{F}_{\text{HOTCom}}^i$ as a last response to a commitment of shares for **Triples**, send all shares as input to **Triples** in $\mathcal{F}_{\text{Offline}}$ and afterwards send **reject**. Let $\mathcal{F}_{\text{GPRO}}$ output what $\mathcal{F}_{\text{Offline}}$ outputs. Otherwise, simulate $\mathcal{F}_{\text{GPRO}}$ honestly.

Initialize

1. Start a local instance of Π_{Offline} with which the corrupt parties communicate. Simulate parties $\mathcal{P} \in \mathbf{H}$ and functionalities $\mathcal{F}_{\text{HOTCom}}$ and $\mathcal{F}_{\text{OLE}}$.
2. Run **Initialize** for all honest parties.
3. Upon obtaining $(\text{Init}, \text{sid}, \text{in} + 3\text{m})$ from $\mathcal{F}_{\text{HOTCom}}^j$, $j \in \mathbf{C}$ send $(\text{Init}, \text{sid}, \text{in}, \text{m})$ to $\mathcal{F}_{\text{Offline}}$.

Input Run the protocol honestly for each party in $\mathbf{H}$ and commit to random shares. Send shares that $\mathcal{P}_i \in \mathbf{C}$ inputs into $\mathcal{F}_{\text{HOTCom}}^i$ to $\mathcal{F}_{\text{Offline}}$.

Triples

1. For each $l \in [m]$, $\mathcal{P}_h \in \mathbf{H}$ set $\delta_{a,l}^h = \delta_{b,l}^h = 0$.
2. Follow step 1 of the protocol honestly. For the honest a_l^h, b_l^h use the values generated by the simulator before. Let a_l^c, b_l^c to be the values inserted into $\mathcal{F}_{\text{HOTCom}}^c$ by the dishonest parties $\mathcal{P}_c \in \mathbf{C}$.
3. Simulate the interactions between each honest $\mathcal{P}_h$ and each corrupted $\mathcal{P}_c$ for each $l \in [m]$ as follows:
 - (a) Simulate calls to $\mathcal{F}_{\text{OLE}}$ of $\mathcal{P}_h$ with values a_l^h and b_l^h defined above.
 - (b) For each dishonest party $\mathcal{P}_c$, wait until $\mathcal{P}_c$ sends its multiplicative shares $(\bar{a}_{h,l}^c, \bar{b}_{h,l}^c)$ to the instances of $\mathcal{F}_{\text{OLE}}$.
 - (c) Add $\bar{a}_{h,l}^c - a_l^c$ to $\delta_{a,l}^h$ and $\bar{b}_{h,l}^c - b_l^c$ to $\delta_{b,l}^h$.
 - (d) Wait for each $\mathcal{P}_c$ to send $\bar{c}_l^c$ to $\mathcal{F}_{\text{HOTCom}}^c$.
 - (e) After the final commit wait for each $\mathcal{F}_{\text{HOTCom}}^i$ instance to
4. Compute $c_l = (\sum_h a_l^h + \sum_c \bar{a}_l^c) \cdot (\sum_h b_l^h + \sum_c \bar{b}_l^c)$ and for every honest party set $c_l^h = a_l^h b_l^h + \sum_{j=1}^n (w_{j,l}^h + v_{j,l}^h)$.
5. Set $\Delta_{c,l} = \sum_{c \in \mathbf{C}} \bar{c}_l^c - (c_l - \sum_{h \in \mathbf{H}} c_l^h)$.
6. When every interaction between all pairs of parties is completed as the last dishonest party commits to its share, input $\delta_{a,l}^h, \delta_{b,l}^h$ for each honest party, $a_l^c, b_l^c, \bar{c}_l^c$ for each dishonest party $\mathcal{P}_c$ as well as the cumulative $\Delta_{c,l}$. Then obtain h as a response.
7. Before outputting $\overline{\mathbf{ch}}^i$ program $\mathcal{F}_{\text{GPRO}}$ so that on input $(\overline{\mathbf{ch}}^1 \parallel \dots \parallel \overline{\mathbf{ch}}^{|\mathcal{P}|})$ it will output $\mathbf{ch}$ (if its not already programmed to this value). If $\mathcal{F}_{\text{GPRO}}$ cannot be reprogrammed, abort.

Linear For any $(\text{Linear}, \text{sid})$ sent by a dishonest party to $\mathcal{F}_{\text{HOTCom}}^j$ send the respective command to $\mathcal{F}_{\text{Offline}}$.

Fig. 25: Simulator for Π_{Offline} where at least one party is honest.

After all $\mathcal{F}_{\text{OLE}}$ interactions are completed, all parties commit to their additive share of c using $\mathcal{F}_{\text{HOTCom}}$. Corrupt parties $\mathcal{P}_c$ can introduce an additive error Δ_c

Simulator $\mathcal{S}_{\text{OffHonP}}$ (Part 2)

Open All openings are simulated as follows:

1. Wait for the message $(\text{Open}, \text{sid}, \text{id}, i, v)$ from $\mathcal{F}_{\text{Offline}}$. Change the committed share v_h of the simulated honest party $\mathcal{P}_h$ with the smallest index in $\mathcal{F}_{\text{HOTCom}}^h$ such that $v = \sum_j v_j$.
2. Let all simulated honest parties run the protocol honestly & set $\delta = 0$.
3. Wait for each $\mathcal{P}_c \in \mathcal{C}$ to let $\mathcal{F}_{\text{HOTCom}}^j$ release $(\text{cid}^c, \bar{v}^c)$ (either towards an honest party or all parties).
4. Set $\Delta_c = \sum_{c \in \mathcal{C}} (\bar{v}^c - v^c)$. If any $\bar{v}^c \neq v^c$ then set $\delta = 1$.
5. Input (Δ_c, δ) to $\mathcal{F}_{\text{Offline}}$.
6. If instead $\text{type} = 2$ then wait for all dishonest parties to send $(\text{DelHint}, \text{sid}, \text{id})$ as well as $(\text{Hint}, \text{sid}, v^c)$ to each $\mathcal{F}_{\text{HOTCom}}^c$, compute Δ_c, δ as before and then send $(\text{Open}, \text{sid}, \text{id}, 2, 0)$ and Δ_c, δ to $\mathcal{F}_{\text{Offline}}$.

Release Send $(\text{Release}, \text{sid})$ for each dishonest $\mathcal{P}_c$ upon first message $(\text{Open}, \text{sid})$ from $\mathcal{F}_{\text{HOTCom}}^c$.

Full Release

1. Send $(\text{Release}, \text{sid})$ for each dishonest $\mathcal{P}_c$ upon first message $(\text{Open}, \text{sid})$ from $\mathcal{F}_{\text{HOTCom}}^c$.
2. Upon obtaining $(\text{Release}, \text{sid}, \text{id}, v)$ for each delayed id adjust the share v_h of the simulated honest party $\mathcal{P}_h$ with the smallest index at $\mathcal{F}_{\text{HOTCom}}^h$ such that $v = \sum_j v_j$.
3. Simulate the protocol honestly.

Verify Follow the steps of **Verify** in Π_{Offline} .

Challenge Follow the steps of **Challenge** in Π_{Offline} .

Fig. 26: Simulator for Π_{Offline} where at least one party is honest.

to the product by committing to a share $\bar{c}^c \neq c^c$ where c^c is the value they should have committed to considering the outputs of $\mathcal{F}_{\text{OLE}}$. To compute this additional additive error, note that $\mathcal{S}_{\text{OffHonP}}$ has all shares of a, b of both the simulated honest and the dishonest parties. We use the honest shares of $\mathcal{P}_h$ and the shares $\bar{a}^c, \bar{b}^c$ that the adversary uses for $\mathcal{F}_{\text{OLE}}$ to define the product c as it would be computed in the protocol at this point, including the errors δ_a^h, δ_b^h introduced by the adversary. Thus, the output of the multiplication protocol using the inputs to $\mathcal{F}_{\text{OLE}}$ can be written as

$$c = \left(\sum_h a^h + \sum_c \bar{a}^c \right) \cdot \left(\sum_h b^h + \sum_c \bar{b}^c \right)$$

for which the adversary can provide additive shares if it followed the protocol. However, the sum $\sum_h c^h + \sum_c \bar{c}^c$ may differ from c , and this difference defines the additive error term Δ_c . Note that this computation of Δ_c removes all dependence on the a^h, b^h, c^h of simulated honest parties and is therefore independent of the choice of the simulator. Hence, adding this constant error to the actual product in $\mathcal{F}_{\text{Offline}}$ is not noticed by the adversary, and protocol and simulation are indistinguishable when the outcomes are reconstructed later.

During **Open** the simulator obtains from $\mathcal{F}_{\text{Offline}}$ the correct openings, and adjusts the output of a single instance of $\mathcal{F}_{\text{HOTCom}}$ towards $\mathcal{A}$ accordingly. It also waits for the openings from all $\mathcal{P}_c \in \mathcal{C}$. The sent values are compared to the values stored earlier, and the difference to the correct value is set as Δ_c that the simulator will input to $\mathcal{F}_{\text{Offline}}$. In addition, it can happen that $\Delta_c = 0$ but shares of dishonest parties are still not opened correctly. In this case, $\mathcal{S}_{\text{OffHonP}}$ will set $\delta = 1$ and send this to $\mathcal{F}_{\text{Offline}}$ as well. During **Full Release**, the opening is equally equivocated to the correct value v as during **Open**.

The openings that $\mathcal{S}_{\text{OffHonP}}$ makes towards $\mathcal{A}$ have shares of the same distribution as in the protocol. Towards this, note that in both cases they reconstruct to the correct secret. If only one party is honest, then by definition the revealed share has the correct distribution. If more than one party is honest, for all but one of the parties the shares are distributed exactly as they are in the protocol. This is because we simulate the shares of each honest party (except the one we use for equivocation) by exactly following the protocol. For the last party, the distribution must be identical by correctness of the reconstruction. Moreover, **Verify** will output the same both during simulation and in the real protocol: in the real protocol, it outputs if dishonest parties open a share incorrectly. In the simulation, it does the same as $\mathcal{F}_{\text{HOTCom}}^h$ for the equivocated honest party will always output that the equivocated share was valid to begin with.

All parties corrupted. In this case, the simulator only has to extract the shares for **Input** and **Triples** for input into $\mathcal{F}_{\text{Offline}}$ as it simulates all instances of $\mathcal{F}_{\text{HOTCom}}$ and otherwise just forwards messages to $\mathcal{F}_{\text{Offline}}$ for completion of protocol steps. It also has to simulate $\mathcal{F}_{\text{GPRO}}$ and performs this as in $\mathcal{S}_{\text{OffHonP}}$. The shares sent to $\mathcal{F}_{\text{HOTCom}}^i$ fully define which openings are correct and are input into $\mathcal{F}_{\text{Offline}}$, and eventually the challenge h . In comparison to $\mathcal{S}_{\text{OffHonP}}$ it can happen that a random oracle output for **Triples** may happen for **Input** shares that were not sent to $\mathcal{F}_{\text{Offline}}$ as $\mathcal{A}$ “simulates” a protocol execution and the generated Fiat Shamir responses locally. To emulate this and the expected output challenges, $\mathcal{F}_{\text{Offline}}$ allows the simulator to tentatively submit shares and then **reset** $\mathcal{F}_{\text{Offline}}$ to try different shares afterwards. We use this to simulate responses correctly, and then let the simulator choose the correct response according the commitments of shares to all $\mathcal{F}_{\text{HOTCom}}^i$ by $\mathcal{A}$. Note that while this seems to be a stronger adversarial capability, it still only leads to at most Q random oracle queries that lead to reprogramming, and thus a distinguishing advantage of at most $Q^2/2^{2\lambda}$.

Π_{Offline} shows error during **Audit** exactly when a dishonest party either didn’t open its share correctly or if the reconstructed sum is not equal to the input to the audit procedure. $\mathcal{F}_{\text{Offline}}$ computes its result the same way, as incorrect openings in the simulation are forwarded by the simulator as Δ, δ inputs during **Open** leading to the same abort during **Audit** as well.

H Online Protocol

We present the online phase of the MPC scheme in Figure 27 and Figure 28.

Π_{Online} (Part 1)

The protocol runs among parties $\mathcal{P}$ which evaluate the circuit $\mathcal{C}$ over $\mathbb{F}$, and the auditor $\mathcal{P}_A$. All parties have access to the bulletin board $\mathcal{F}_{\text{GBB}}$ and the offline functionality $\mathcal{F}_{\text{Offline}}$. The communication is synchronous. When a party does not send a message, $(\text{Abort}, \text{sid})$ is sent through $\mathcal{F}_{\text{GBB}}$ and the protocol is aborted. $\mathcal{F}_{\text{Offline}}$ uses challenge space $\mathcal{T} = \mathbb{F}_p^*$.

Initialize Given the circuit $\mathcal{C}$ let n be the number of input values and m for the number of multiplication gates. Let mp_{val} be a map that is initially empty.

1. $\mathcal{P}_1$ sends $\mathcal{C}$ to $\mathcal{F}_{\text{GBB}}$. If the circuit is incorrect, then abort.
2. All parties send $(\text{Init}, \text{sid}, n, 2m)$ to $\mathcal{F}_{\text{Offline}}$.
3. All parties then invoke $(\text{Input}, \text{sid})$ and $(\text{Triples}, \text{sid})$. Each party stores all shares it obtains in mp_{val} .
4. Use **Verify Triples** to verify that the triples are correct.

Input $\mathcal{P}_i$ wants to share input $x \in \mathbb{F}$ under identifier id_x .

1. Let id_r be the first unused id in mp_{val} .
2. Each party sends $(\text{Open}, \text{sid}, id_r, 0, \mathcal{P}_i)$ to $\mathcal{F}_{\text{Offline}}$.
3. Upon obtaining $(\text{Open}, \text{sid}, i, r)$ from $\mathcal{F}_{\text{Offline}}$, $\mathcal{P}_i$ computes $\delta = x - r$ and sends $(\text{Broadcast}, \mathcal{P}_i, \delta)$ to $\mathcal{F}_{\text{GBB}}$.
4. Upon seeing δ on $\mathcal{F}_{\text{GBB}}$, each party sends $(\text{linear}, \text{sid}, id_r, id_x, 1, \delta)$ to $\mathcal{F}_{\text{Offline}}$.

Procedure Verify Triples

1. Let τ be the challenge returned by $\mathcal{F}_{\text{Offline}}$ when generating triples.
2. Each of the parties compute $\rho = \tau \cdot x + a$ and $\sigma = y + b$ using linear operations in $\mathcal{F}_{\text{Offline}}$.
3. The parties then reconstruct ρ and σ using $\mathcal{F}_{\text{Offline}}$.
4. $\mathcal{P}_1$ broadcasts ρ, σ, τ . If the value differs from the opened value, abort.
5. All parties compute $v = \tau \cdot z - c + \sigma \cdot a + \rho \cdot b - \rho \cdot \sigma$ using linear operations in $\mathcal{F}_{\text{Offline}}$.
6. Open v by sending $(\text{Open}, \text{sid}, id_v, 1, 0)$ to $\mathcal{F}_{\text{Offline}}$. If $v \neq 0$, abort.

Fig. 27: Online protocol Π_{Online} that securely realizes $\mathcal{F}_{\text{Online}}$ (Input and Computation).

H.1 Security of the online protocol

Proof. We prove Theorem 4 by providing a simulator $\mathcal{S}_{\text{OnHonP}}$ (Figure 29) when at least one party is honest and $\mathcal{S}_{\text{OnFull}}$ (Figure 30) for the fully malicious case. The adversary can cheat by either introducing an error into the multiplication triples such that $a \cdot b + \Delta = c$ with $\Delta \neq 0$ or adding an error to a public, private or delayed opening. In both settings the simulator will first initialize $\mathcal{F}_{\text{Online}}$, start a local copy of Π_{Online} and initialize $\mathcal{F}_{\text{Offline}}$ with which all dishonest parties communicate. The simulation of **Linear** and **Multiply** is straight forward in either setting. The main difference lies in the point in time the simulator outputs that $\mathcal{A}$ cheated.

Π_{Online} (Part 2)

Compute When **Init** and **Input** have been executed before and all input gates of $\mathcal{C}$ have inputs assigned, all parties do the following for each gate in $\mathcal{C}$ with inputs id_x, id_y in mp_{val} and where the output id_z has not yet been computed:

Linear Evaluate the gate using **Linear** in $\mathcal{F}_{\text{Offline}}$. Then store id_z in mp_{val} .

Multiply

1. The parties find an unused triple id_a, id_b, id_c .
2. The parties use **Linear** of $\mathcal{F}_{\text{Offline}}$ to compute $\alpha = x - a, \beta = y - b$. Denote the outputs as id_α, id_β .
3. Every $\mathcal{P}_i \in \mathcal{P}$ sends $(\text{Open}, \text{sid}, id_\alpha, 1, i)$ and $(\text{Open}, \text{sid}, id_\beta, 1, i)$ to $\mathcal{F}_{\text{Offline}}$.
4. $\mathcal{P}_1$ sends α, β to $\mathcal{F}_{\text{GGB}}$. If the values are incorrect, then abort.
5. Upon having obtained α, β from $\mathcal{F}_{\text{Offline}}$, each party computes id_z on $\mathcal{F}_{\text{Offline}}$ where $z = \alpha \cdot b + \beta \cdot a - \alpha \cdot \beta + c$ using **Linear**.
6. Mark id_a, id_b, id_c as used.

Output After all gates are evaluated, the parties open the output as follows:

1. Let id_y be the ID of the output. All parties send $(\text{Open}, \text{sid}, id_y, 2, 0)$ to $\mathcal{F}_{\text{Offline}}$.
2. Every party sends $(\text{Release}, \text{sid})$ to $\mathcal{F}_{\text{Offline}}$.
3. Let L_o be the list of all opened $\alpha, \beta, \rho, \sigma, v$ during the computation and triple verification. Send $(\text{Verify}, \text{sid}, L_o)$ to $\mathcal{F}_{\text{Offline}}$.
4. For $\mathcal{P}_i$ let L_i list all IDs opened privately to $\mathcal{P}_i$ during **Input**. Send $(\text{Verify}, \text{sid}, L_i)$ to $\mathcal{F}_{\text{Offline}}$.
5. If **Verify** outputs **reject** then send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{GGB}}$ and abort.
6. If no party aborts then all parties send $(\text{Release}, \text{sid})$ to $\mathcal{F}_{\text{Offline}}$ and obtain y .
7. Send $(\text{Verify}, \text{sid}, id_y)$ to $\mathcal{F}_{\text{Offline}}$. If $\mathcal{F}_{\text{Offline}}$ responds with *Accept* then output y . Otherwise send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{GGB}}$ and abort.

Audit Upon input $(\text{Audit}, \text{sid}, y')$ to the auditor $\mathcal{P}_A$, it does the following:

1. Check if any party broadcast $(\text{Abort}, \text{sid})$ on $\mathcal{F}_{\text{GGB}}$. If so, then output $(\text{NoAuditPossible}, \text{sid})$.
2. Send $(\text{Challenge}, \text{sid})$ to $\mathcal{F}_{\text{Offline}}$ to obtain τ .
3. $\mathcal{P}_A$ sends $(\text{Verify}, \text{sid})$ to $\mathcal{F}_{\text{GGB}}$ to retrieve the list of all opened messages denoted by li_m which opened all $\alpha, \beta, \rho, \sigma, v$ and the output y . $\mathcal{P}_A$ changes li_m so that it also contains all the applied linear functions f_l which can be reconstructed using τ .
4. Let id_y be the output and y be the opened value on $\mathcal{F}_{\text{GGB}}$. If $y \neq y'$ then output $(\text{Reject}, \text{sid})$.
5. $\mathcal{P}_A$ sends $(\text{Audit}, \text{sid}, \text{li}_m)$ to $\mathcal{F}_{\text{Offline}}$. If $\mathcal{F}_{\text{Offline}}$ returns **reject**, output $(\text{Reject}, \text{sid})$. Otherwise output $(\text{Accept}, \text{sid})$.

Fig. 28: Online protocol Π_{Online} that securely realizes $\mathcal{F}_{\text{Online}}$

At least one honest party. When at least one party is honest, $\mathcal{S}$ has to simulate these parties in the local copy of Π_{Online} . When $\mathcal{A}$ introduces an additive error to any of the triples set $\text{fl}_{\text{cheated}} = \top$ but only abort the execution when v opens to

another value than 0 in the **Verify Triple** procedure. Additionally, when using any opening and $\mathcal{A}$ adds an error to the opened value $\mathbf{fl}_{\text{cheated}}$ will be set to $\top$. During **Output** a delayed opening of y^* is performed followed by a verification of all previously opened values, including their linear computations. Each party also includes the values that were privately opened to them in the verification process. Any honest party would abort the execution when **Verify** in $\mathcal{F}_{\text{Offline}}$ returns **reject**, so $\mathcal{S}$ aborts if $\mathbf{fl}_{\text{cheated}} = \top$ which includes all of these cases and any other case of cheating. After y^* is released the parties by reprogramming $\mathcal{F}_{\text{Offline}}$ they perform another verification, similarly honest parties would abort the execution based on the answer of $\mathcal{F}_{\text{Offline}}$. $\mathcal{S}$ will forward y^* to $\mathcal{F}_{\text{Online}}$ which will abort if $y^* \notin \{y, \perp\}$, where y is the correct value computed by $\mathcal{F}_{\text{Online}}$. **Audit** is straight forward in this setting as the protocol output is identical to the $\mathcal{F}_{\text{Online}}$ output. $\mathcal{S}$ will always abort the execution while honest parties in a protocol will not abort the execution in the case of a cheating $\mathcal{A}$, when $\mathcal{A}$ wins in the triple check despite triples being inconsistent. We discuss the probability for this scenario after we present the fully malicious setting.

Fully malicious setting. When no party is honest there is not much simulation to be done, except of $\mathcal{F}_{\text{Offline}}$ towards the corrupted parties. Moreover, $\mathcal{S}$ will not reveal cheating during **Output**, as no malicious party would abort the computation even if $\mathcal{F}_{\text{Offline}}$ returns **reject** or $v \neq 0$ (unless they explicitly abort). Only when $\mathcal{P}_A$ queries **Audit** the protocol should reject in the case of malicious behavior. $\mathcal{F}_{\text{Online}}$ will always output $(\text{Reject}, \text{sid}, y)$ if $\mathcal{A}$ cheated, but the protocol will falsely accept y in the same case that honest parties will falsely accept the computation in the previously discussed case.

False triple check. The value τ is generated by $\mathcal{F}_{\text{Offline}}$ and can be any value in $\mathbb{F} \setminus 0$. Any new choice of triples changes the value of τ and $\mathcal{A}$ can test τ when triples are sent during **Triples**. If the adversary is not satisfied with τ (i.e. the test would not pass), $\mathcal{A}$ can send **reject** and try new inputs, or reset the functionality by sending **reset** if all parties are corrupted. It is important that all triple shares of the adversary must be chosen before the adversary learns τ . For each τ the probability of a false accepting **Triple Verification** is $\frac{1}{|\mathbb{F}|-1}$ as described and proven in [28]. Hence the total distinguishing advantage for any environment $\mathcal{Z}$ is $\frac{Q}{|\mathbb{F}|-1}$, where Q equals the number of **reject** or **reset** messages that the adversary can send.

Simulator $\mathcal{S}_{\text{OnHonP}}$

In the following we denote $\mathbf{H}$ as honest and $\mathbf{C}$ as corrupt parties. Initially set $\text{fl}_{\text{cheated}} = \perp$. If at any point a party broadcasts $(\text{Abort}, \text{sid})$ then send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{Online}}$.

Initialize

1. Upon observing the broadcast of $\mathbf{C}$ and if no party aborts, send $(\text{Init}, \text{sid}, \mathcal{C})$ to $\mathcal{F}_{\text{Online}}$.
2. Start a local instance of Π_{Online} with which all $\mathcal{P} \in \mathbf{C}$ communicate. Simulate honest parties in $\mathbf{H}$.
3. Initialize a copy of $\mathcal{F}_{\text{Offline}}$ by running all steps of **Initialize**. All parties communicate with the instance of $\mathcal{F}_{\text{Offline}}$ through the simulator.
4. Follow the steps of **Initialize** in Π_{Online} . When $\mathcal{A}$ inputs any δ_a, δ_b or Δ_c that is $\neq 0$ to $\mathcal{F}_{\text{Offline}}$ during **Triple**, set $\text{fl}_{\text{cheated}} = \top$.
5. When the opening of v during the **Verify Triples** procedure is $v \neq 0$, abort the execution.

Input

1. For every $\mathcal{P}_i \in \mathbf{H}$, simulate Π_{Online} honestly using a dummy input upon receiving $(\text{Input}, \text{sid}, i, \text{id}_x)$ from $\mathcal{F}_{\text{Online}}$. If any party in $\mathbf{C}$ inputs $\Delta \neq 0$ to $\mathcal{F}_{\text{Offline}}$ during **Open**, set $\text{fl}_{\text{cheated}} = \top$.
2. For any party $\mathcal{P}_i \in \mathbf{C}$ wait until $\mathcal{P}_i$ sends δ through $\mathcal{F}_{\text{GBB}}$. Compute $x = \delta + r$ and send $(\text{Input}, \text{sid}, i, \text{id}_x, x)$ to $\mathcal{F}_{\text{Online}}$.

Compute When **Initialize** and **Input** have been executed before, evaluate $\mathcal{C}$ gate per gate. For **Linear** and **Multiply** follow the steps in Π_{Online} .

Output

1. Simulate the delayed open during **Output**. The adversary inputs Δ into $\mathcal{F}_{\text{Offline}}$.
2. Run the first $(\text{Release}, \text{sid})$ with $\mathcal{F}_{\text{Offline}}$. If $\text{fl}_{\text{cheated}} = \top$ then abort.
3. Otherwise send $(\text{Compute}, \text{sid})$ to $\mathcal{F}_{\text{Online}}$ and wait for the output y .
4. Wait for parties to send the second $(\text{Release}, \text{sid})$ and let $\mathcal{F}_{\text{Offline}}$ output $y^* = y + \Delta$. If $\Delta \neq 0$ let $\mathcal{F}_{\text{Offline}}$ output reject during **Verify** of y^* and accept if $\Delta = 0$.
5. Send $(\text{Output}, \text{sid}, y^*)$ to $\mathcal{F}_{\text{Online}}$.

Fig. 29: Simulator for the online phase where at least one party is honest.

Simulator $\mathcal{S}_{\text{OnFull}}$

In the following all parties are corrupt. Initially set $\text{fl}_{\text{cheated}} = \perp$. The simulator waits until the computation is complete and all messages are sent to $\mathcal{F}_{\text{GBB}}$ before the simulation begins. If at any point a party broadcasts $(\text{Abort}, \text{sid})$ then send $(\text{Abort}, \text{sid})$ to $\mathcal{F}_{\text{Online}}$.

Initialize

1. Initialize a copy of $\mathcal{F}_{\text{Offline}}$ with which the parties communicate.
2. Upon observing the circuit $\mathcal{C}$ being broadcast, send $(\text{Init}, \text{sid}, \mathcal{C})$ to $\mathcal{F}_{\text{Online}}$.
3. Emulate an instance of $\mathcal{F}_{\text{Offline}}$ honestly.
4. If $\mathcal{A}$ sends a malformed triple to $\mathcal{F}_{\text{Offline}}$, set $\text{fl}_{\text{cheated}} = \top$.

Input Wait for $\mathcal{P}_i$ to broadcast δ . Input $x = \delta + r$ to $\mathcal{F}_{\text{Online}}$, where r is the value opened through $\mathcal{F}_{\text{Offline}}$.

Compute If any of the openings during a multiplication is incorrect, set $\text{fl}_{\text{cheated}} = \top$.

Output

1. Upon having observed all openings of $\mathcal{F}_{\text{GBB}}$ and if $(\text{Release}, \text{sid})$ has been used as in the protocol, send $(\text{Compute}, \text{sid})$ to $\mathcal{F}_{\text{Online}}$ and obtain y . Let Δ be the value sent through **Open** when releasing the output.
2. Set $y^* = y + \Delta$ and send $(\text{Output}, \text{sid}, y^*)$ to $\mathcal{F}_{\text{Online}}$. If $y^* \notin \{y, \perp\}$, set $\text{fl}_{\text{cheated}} = \top$.
3. If $\text{fl}_{\text{cheated}} = \top$ then let **Audit** in $\mathcal{F}_{\text{Offline}}$ always fail during audit of the output y^* .

Fig. 30: Simulator for the online phase in the fully malicious setting.